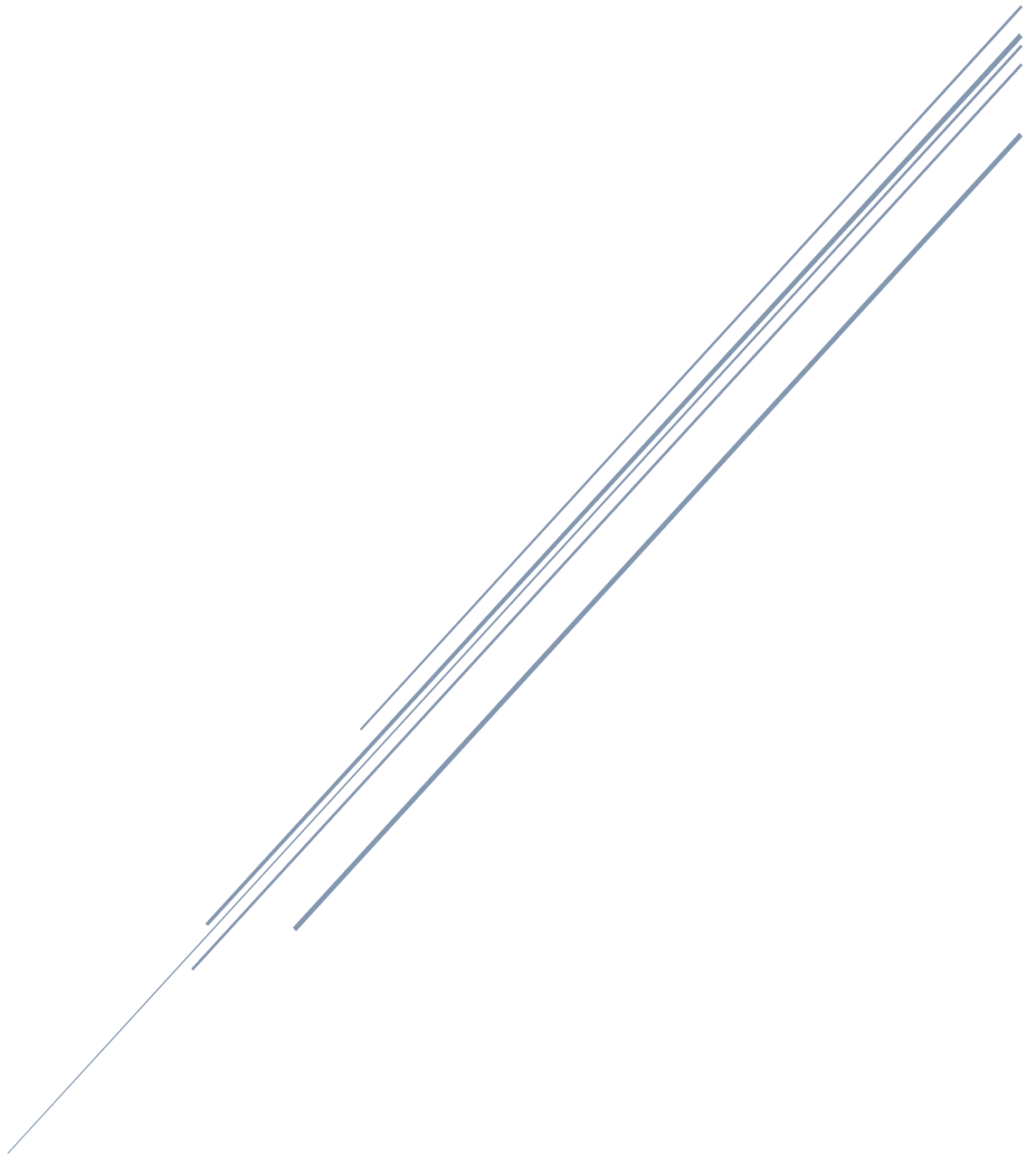


# VLSI PHYSICAL DESIGN Experiment Manual

## OpenROAD



## VLSI PHYSICAL DESIGN OpenROAD Experiment Manual

### Suggested List of Experiments:

| Contents :<br>Unit | Topics                                                                                                                                                                  | Contact<br>Hours |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| 1                  | <b>Experiment-1</b> Explore the OpenROAD flow and develop the environment to use open source tools for RTL2GDS flow                                                     | 2                |
| 2                  | <b>Experiment-2</b> Perform RTL Synthesis of the design using yosys open synthesis tool.                                                                                | 2                |
| 3                  | <b>Experiment-3</b> Develop the data base using OpenDB for physical chip design.                                                                                        | 2                |
| 4                  | <b>Experiment-4</b> Initialize the floor plan with necessary specifications.                                                                                            | 2                |
| 5                  | <b>Experiment-5</b> Develop power grid planning using PDNGEN                                                                                                            | 2                |
| 6                  | <b>Experiment-6</b> Learn the pin placement using ioPlacer                                                                                                              | 2                |
| 7                  | <b>Experiment-7</b> Place an IO ring around the boundary of a chip and connect with either wirebond pads or a bump array                                                | 2                |
| 8                  | <b>Experiment-8</b> Explore OpenSTA as a gate level static timing verifier and as a stand-alone executable to verify the timing of a design using standard file format. | 2                |
| 9                  | <b>Experiment-9</b> Perform the detailed placement analysis using OpenDP                                                                                                | 2                |
| 10                 | <b>Experiment-10</b> Analyze clock tree synthesis using TritonCTS 2.0                                                                                                   | 2                |
| 11                 | <b>Experiment-11</b> Perform global routing using TritonRoute and do analysis of design.                                                                                | 2                |
| 12                 | <b>Experiment-12</b> Do IR drop analysis using PDNSim                                                                                                                   | 2                |
| 13                 | <b>Experiment-13</b> Perform parasitic extraction analysis using OpenRCX                                                                                                | 2                |
| 14                 | <b>Experiment-14</b> Check antenna rules violations using Antenna Rule Checker and generate a report.                                                                   | 2                |
|                    | <b>Total Hours</b>                                                                                                                                                      | <b>28</b>        |

## EXPERIMENT–1

### Title:

**Explore the OpenROAD Flow and Develop the Environment to Use Open-Source Tools for RTL-to-GDSII Flow**

**Contact Hours:** 2 Hours

### 1. AIM

To explore the **OpenROAD RTL-to-GDSII flow** and set up a working environment using open-source EDA tools for digital physical design.

### 2. OBJECTIVES

After completing this experiment, the student will be able to:

1. Understand the complete **RTL-to-GDSII physical design flow**.
2. Understand the role of OpenROAD and OpenROAD-flow-scripts.
3. Set up the OpenROAD environment in Linux.
4. Verify the installation of OpenROAD and supporting tools.
5. Understand the input and output files used at different stages.
6. Execute a sample design through the OpenROAD flow.
7. Observe generated reports and physical-design results.

OpenROAD provides an open-source digital place-and-route environment, while OpenROAD-flow-scripts provides a ready-to-use RTL-to-GDSII flow controlle

### 3. SOFTWARE / TOOLS REQUIRED

| S.No. | Software / Tool        | Purpose                        |
|-------|------------------------|--------------------------------|
| 1     | Ubuntu/Linux           | Operating environment          |
| 2     | OpenROAD               | Physical design implementation |
| 3     | OpenROAD-flow-scripts  | RTL-to-GDSII flow              |
| 4     | Yosys                  | RTL synthesis                  |
| 5     | OpenSTA                | Static timing analysis         |
| 6     | OpenDB                 | Physical design database       |
| 7     | Tcl                    | Flow scripting                 |
| 8     | Nangate45 / SKY130 PDK | Technology information         |

| S.No. | Software / Tool | Purpose                         |
|-------|-----------------|---------------------------------|
| 9     | Gvim            | Editing configuration/Tcl files |

OpenROAD supports Tcl-based flow control and provides commands for reading/writing LEF, DEF, Verilog and database files.

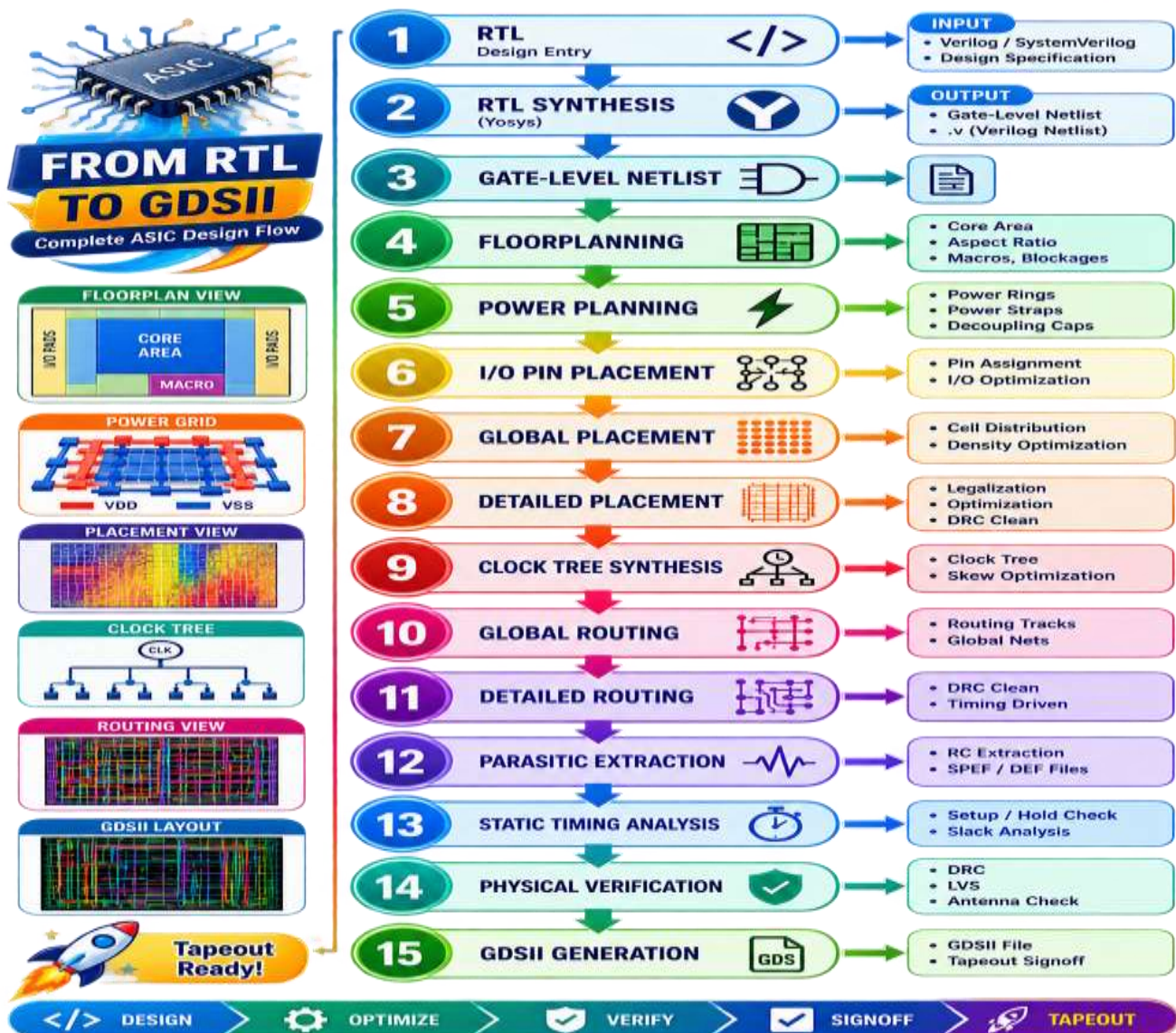
## 4. THEORY

### 4.1 Introduction to OpenROAD

**OpenROAD** is an open-source digital physical-design tool used to implement a digital circuit from synthesized RTL/netlist toward a manufacturable physical layout.

The major objective is to provide an integrated environment for the **RTL-to-GDSII** implementation flow.

The simplified flow is:



OpenROAD documentation describes the flow as an RTL-to-GDSII implementation flow containing stages such as floorplanning, I/O placement, PDN creation, placement, CTS and routing.

## 5. OPENROAD FLOW COMPONENTS

| Stage              | Open-Source Tool / Module | Function                              |
|--------------------|---------------------------|---------------------------------------|
| RTL Synthesis      | Yosys                     | Converts RTL into gate-level netlist  |
| Database           | OpenDB                    | Stores design physical information    |
| Floorplanning      | OpenROAD/IFP              | Creates die/core area                 |
| I/O Placement      | ioPlacer/PPL              | Places I/O pins                       |
| Power Planning     | PDNGEN                    | Creates power distribution network    |
| Global Placement   | RePIAce/GPL               | Determines approximate cell locations |
| Detailed Placement | OpenDP/DPL                | Legalizes cell locations              |
| CTS                | TritonCTS                 | Builds clock tree                     |
| Global Routing     | FastRoute/GRT             | Generates global routing              |
| Detailed Routing   | TritonRoute/DRT           | Performs detailed routing             |
| Extraction         | OpenRCX                   | Extracts parasitic R and C            |
| Timing             | OpenSTA                   | Performs static timing analysis       |
| IR Drop            | PDNSim                    | Performs power-grid analysis          |
| Antenna            | Antenna Checker           | Checks antenna violations             |

## 6. RTL-TO-GDSII FILES

The important files used in the flow are:

| File | Description                   |
|------|-------------------------------|
| .v   | Verilog RTL/netlist           |
| .lib | Liberty timing library        |
| .lef | Physical abstract information |
| .sdc | Timing constraints            |

| File       | Description                     |
|------------|---------------------------------|
| .def       | Physical design information     |
| .odb / .db | OpenDB design database          |
| .spef      | Extracted parasitic information |
| .gds       | Final physical layout           |
| .tcl       | OpenROAD flow script            |
| .rpt       | Analysis/report file            |

## 7. PROCEDURE

### Step 1: Open the Linux Terminal

Open the Ubuntu/Linux terminal.

**Check the current user:**

**Cd** : whoami

**Check the operating system:**

**Cd** : uname -a

**Check the current directory:**

**Cd** : pwd

### Step 2: Check OpenROAD Installation

Execute:

**Cd** : openroad -version

If OpenROAD is correctly installed, the command should display the installed version and available command-line options.

The official OpenROAD command-line interface supports options such as -help, -version, -gui, -threads, -log, and -db.

Link :<https://openroad.readthedocs.io/en/latest/main/README2.html>

## 8. CHECK OPENROAD INTERACTIVE MODE

### Step 1: Start OpenROAD

From the Linux terminal, execute:

**Cd:** openroad

**You should see:**

**OpenROAD 26Q2-1994-g065077a742**

Features included (+) or not (-): +GPU +GUI +Python

**openroad>**

### Step 2: Check Available Commands

Inside the OpenROAD interactive prompt, execute:

**Cd :**help

You can also check the available Tcl commands using:

**Cd :**info commands

### Step 3: Check OpenROAD Version

Do not use version inside the OpenROAD prompt, because it is not a valid OpenROAD Tcl command.

**Exit OpenROAD:**

**Cd :** exit

Then, from the Linux terminal, execute:

**Cd :**openroad --version

**Expected output:**

**OpenROAD 26Q2-1994-g065077a742**

### Step 4: Exit OpenROAD

Inside the OpenROAD prompt:

**Cd :**exit

You will return to the Linux terminal:

karnati\_kiran@KIRAN-MOHN55RV:~\$

**Important Note**

For the current OpenROAD version:

### **OpenROAD 26Q2-1994-g065077a742**

the following command is NOT valid inside interactive mode:

**version**

Therefore, use:

**Cd :openroad --version**

from the Linux terminal to check the OpenROAD version.

## **9. OPENROAD-FLOW-SCRIPTS**

If OpenROAD-flow-scripts is already installed:

**Cd : cd ~/OpenROAD-flow-scripts**

Check the contents:

**Cd : ls**

You should find directories such as:

- flow
- docs
- etc
- platforms
- scripts

Move to the flow directory:

**Cd: cd flow**

Check available designs:

**Cd: ls designs**

**For example:**

**Cd: ls designs/nangate45**

A sample design such as gcd may be available.

## **10. CHECK THE SAMPLE DESIGN**

Move into the sample design directory:

**Cd: cd designs/nangate45/gcd**

Check the files:

**Cd: ls**

You may find a configuration file such as:

- config.mk

The exact designs and file names depend on the version of OpenROAD-flow-scripts installed.

## **11. RUN THE SAMPLE OPENROAD FLOW**

Return to the flow directory:

**Cd:** `cd ~/OpenROAD-flow-scripts/flow`

Run the design using its configuration:

**Cd:** `make DESIGN_CONFIG=designs/nangate45/gcd/config.mk`

This launches the configured RTL-to-GDSII flow.

The flow generates intermediate files and reports for different stages.

## **12. OBSERVE THE FLOW DIRECTORIES**

After running the flow, inspect:

**Cd :** `ls`

Important directories include:

- logs/
- reports/
- results/
- objects/

You can inspect them using: **cd :**

- `ls logs/`
- `ls reports/`
- `ls results/`

The exact directory contents vary with the flow version and configuration.

## **13. RUN OPENROAD GUI**

Open the OpenROAD graphical interface using:

**Cd :** `openroad -gui`

The GUI can be used to inspect:

- Die area
- Core area
- Standard cells
- Macros
- I/O pins
- Placement

- Clock tree
- Routing
- Congestion
- Power network

## **14. BASIC OPENROAD TCL COMMANDS**

Start OpenROAD:

**Cd** :openroad

Read a LEF file:

**Cd:** read\_lef <technology.lef> or path

Read another LEF:

**Cd:** read\_lef <standard\_cell.lef> or path

**Read Liberty:**

**Cd:**read\_liberty <standard\_cell.lib> or path

**Read Verilog:**

**Cd** :read\_verilog <design.v> or path

**Link the design:**

**Cd:** link\_design <top\_module>

**Read timing constraints:**

**Cd:** read\_sdc <design.sdc> or path

**Write the database:**

**Cd** : write\_db <design.odb> or path

**Read the database:**

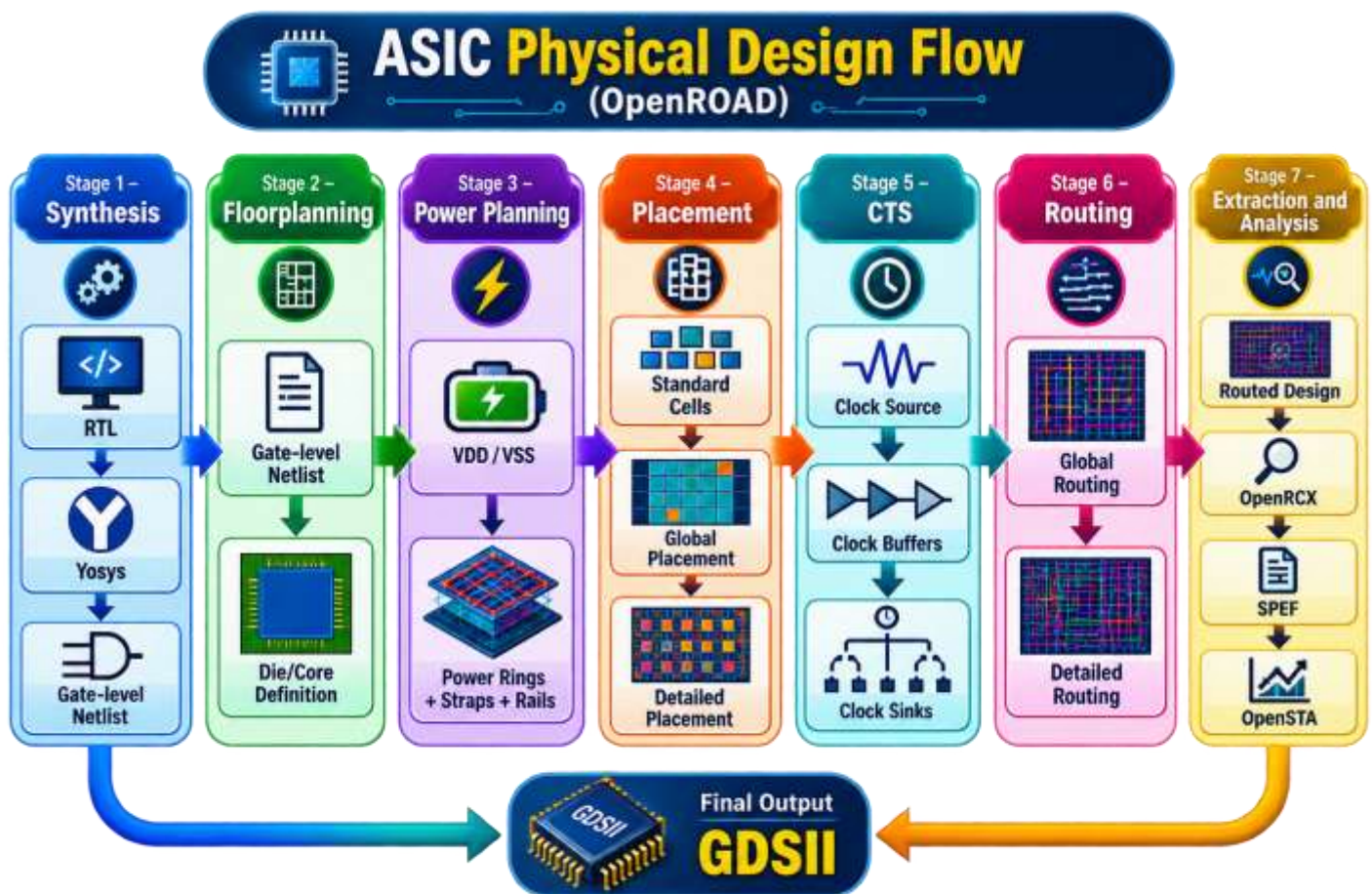
**Cd** : read\_db <design.odb> or path

**Write DEF:**

**Cd** : write\_def <design.def> or path

These are standard OpenROAD database/file commands documented by the project.

## 15. IMPORTANT OPENROAD FLOW STAGES



## 16. OBSERVATION TABLE

| Parameter           | Observation |
|---------------------|-------------|
| OpenROAD Version    |             |
| OS Version          |             |
| PDK                 |             |
| Design Name         |             |
| Top Module          |             |
| Number of Instances |             |
| Number of Nets      |             |
| Core Utilization    |             |
| Die Area            |             |
| Core Area           |             |
| WNS                 |             |
| TNS                 |             |
| Routing Status      |             |
| Final Output        |             |

## 18. RESULT

## **21. CONCLUSION**

## EXPERIMENT-2

### Title

**Perform RTL Synthesis of the Design Using Yosys Open Synthesis Tool**

**Contact Hours: 2 Hours**

### 1. AIM

To perform **RTL synthesis** of a digital design using the **Yosys open-source synthesis tool** and generate a **ngate-level netlist** suitable for further physical design implementation.

### 2. OBJECTIVES

After completing this experiment, the student will be able to:

- Understand the concept of RTL synthesis.
- Understand the role of Yosys in the RTL-to-GDSII flow.
- Read and elaborate Verilog RTL files using Yosys.
- Check the design hierarchy.
- Perform logic optimization.
- Perform technology mapping using a standard-cell library.
- Generate a synthesized gate-level Verilog netlist.
- Analyze synthesis statistics.

### 3. SOFTWARE / TOOLS REQUIRED

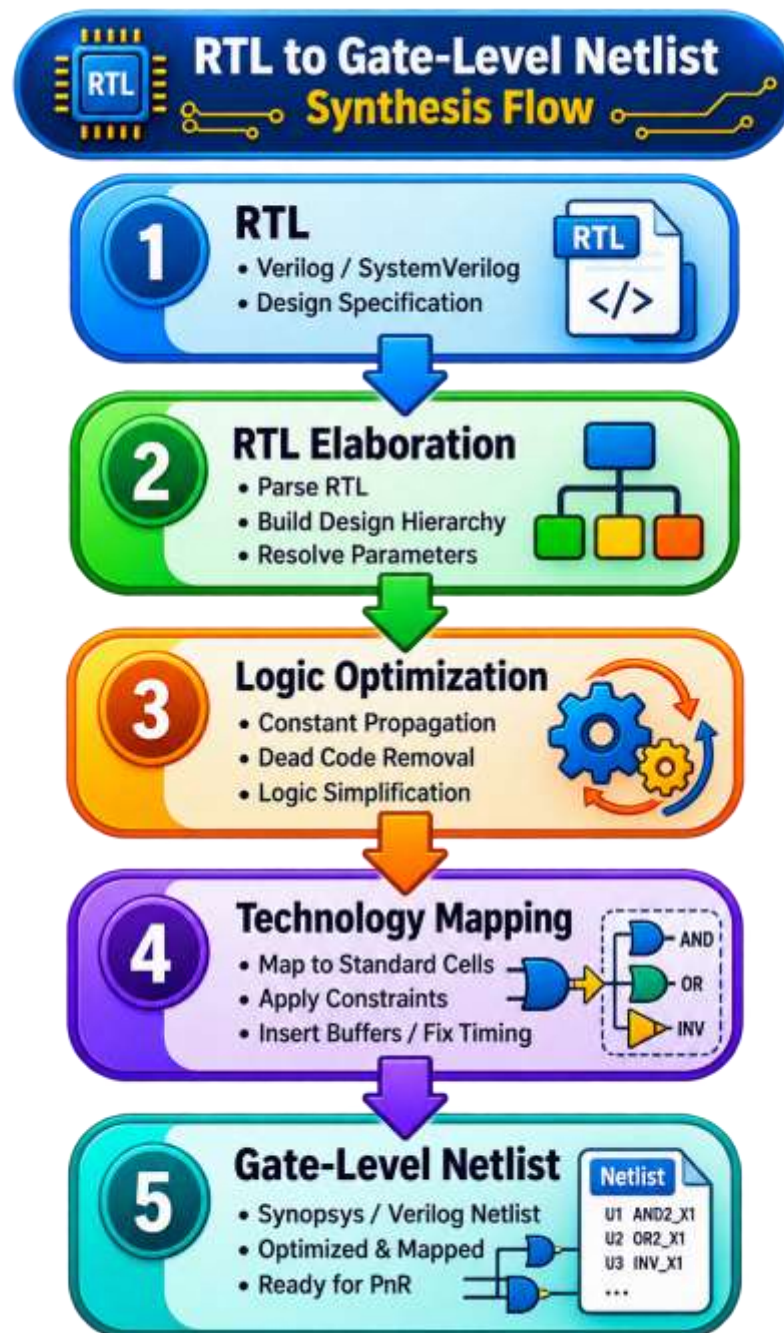
| S.No. | Software / Tool | Purpose                      |
|-------|-----------------|------------------------------|
| 1     | Ubuntu/Linux    | Operating environment        |
| 2     | Yosys           | RTL synthesis                |
| 3     | OpenROAD        | Physical design              |
| 4     | GVim            | RTL/script editing           |
| 5     | Liberty .lib    | Standard-cell timing library |
| 6     | Verilog .v      | RTL source                   |
| 7     | Tcl             | Synthesis scripting          |

## 4. THEORY

### 4.1 RTL Synthesis

RTL synthesis is the process of converting a hardware description written at the Register Transfer Level (RTL) into a gate-level representation.

The basic process is:



The synthesized netlist contains logic gates and sequential elements corresponding to the target standard-cell library.

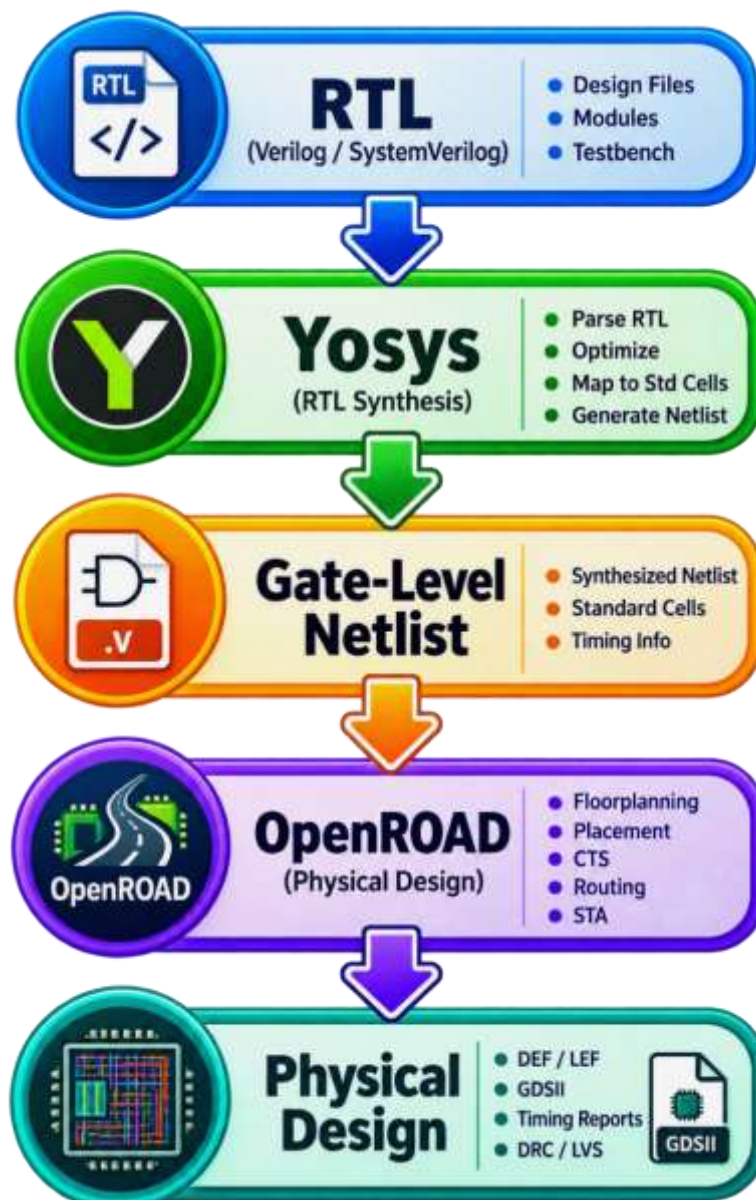
## 4.2 Role of Yosys

**Yosys** is an open-source framework for RTL synthesis.

It can:

- Read Verilog/SystemVerilog RTL
- Elaborate the design hierarchy
- Convert processes into logic
- Optimize Boolean logic
- Infer flip-flops and memories
- Perform technology mapping
- Generate gate-level netlists
- Generate synthesis statistics

In an RTL-to-GDSII flow:



## 5. INPUT FILES

The following files are required.

### 5.1 RTL File

Example:

- **design.v**

This contains the Verilog RTL description.

### 5.2 Liberty File

Example:

- **NangateOpenCellLibrary\_typical.lib**

The Liberty file contains information about:

- Cell function
- Timing
- Power
- Area
- Input capacitance
- Output characteristics

### 5.3 Optional Synthesis Script

Example:

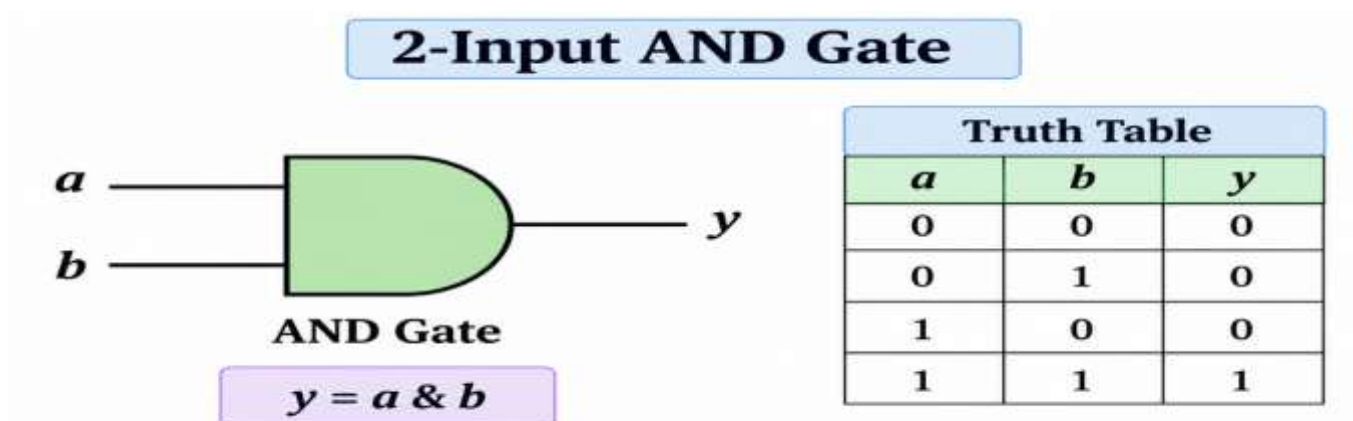
- **synth.tcl**

## 6. EXAMPLE RTL DESIGN

For this experiment, consider a simple 2-input AND circuit.

Create the Verilog file:

- **gvim and\_gate.v**



**Enter:**

```
module and_gate (  
    input a,  
    input b,  
    output y  
);
```

```
assign y = a & b;
```

```
endmodule
```

Save and exit GVim.

## 7. PROCEDURE

### Step 1: Check Yosys Installation

Open the terminal and execute:

- **cd: yosys -Version**

The installed Yosys version should be displayed.

You can also execute:

- **cd: yosys -help**

### Step 2: Create a Working Directory

- **Cd: mkdir -p ~/yosys\_exp2**

Move into the directory:

- **Cd: cd ~/yosys\_exp2**

Check the current directory:

- **Cd: pwd**

### Step 3: Create the RTL File

Create the Verilog file:

- **gvim and\_gate.v**

Enter:

```
module and_gate (  
    input a,  
    input b,  
    output y  
);
```

```
assign y = a & b;
```

```
endmodule
```

Save the file.

Check it:

- `Cd :cat and_gate.v`

## 8. START YOSYS

Run:

- `Cd: yosys`

The Yosys prompt will appear:

- `yosys>`

## 9. READ THE RTL FILE

Inside Yosys:

- `read_verilog and_gate.v or path`

This reads the Verilog RTL into Yosys.

## 10. CHECK DESIGN HIERARCHY

Execute:

- `cd :hierarchy -check -top and_gate`

The -top option specifies the top-level module.

The hierarchy check verifies that the design hierarchy is valid.

## 11. DISPLAY THE DESIGN

Execute:

- `cd:stat`

This displays synthesis-related statistics.

For example:

- Number of wires
- Number of wire bits
- Number of cells
- Number of processes

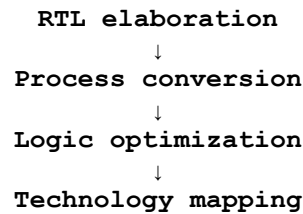
## 12. PERFORM RTL SYNTHESIS

Execute:

- `Cd :synth -top and_gate`

The `synth` command performs the main synthesis process.

The synthesis process includes operations such as:



## 13. VIEW SYNTHESIS STATISTICS

After synthesis:

`stat`

**Observe:**

- Number of wires
- Number of cells
- Cell types
- Number of inputs
- Number of outputs

## 14. TECHNOLOGY MAPPING

For technology mapping, a Liberty standard-cell library can be provided.

First read the Liberty file:

- `Cd:read_liberty -lib <standard_cell.lib>`

Then perform technology mapping:

- `Cd:abc -liberty <standard_cell.lib>`

Run:

- `Cd:stat`

again to observe the mapped cells.

**Note:** Replace `<standard_cell.lib>` with the actual Liberty file available in your PDK.

## 15. GENERATE THE GATE-LEVEL NETLIST

After synthesis, write the output netlist:

- **Cd write\_verilog -noattr synthesized\_netlist.v**

Check the generated file:

- **Cd:ls**

You should see:

- **and\_gate.v**
- **synthesized\_netlist.v**

Display the synthesized netlist:

- **cd:cat synthesized\_netlist.v**

## 16. COMPLETE YOSYS COMMAND SEQUENCE

The complete basic sequence is:

- read\_verilog and\_gate.v
- hierarchy -check -top and\_gate
- stat
- synth -top and\_gate
- stat
- write\_verilog -noattr synthesized\_netlist.v

For technology mapping:

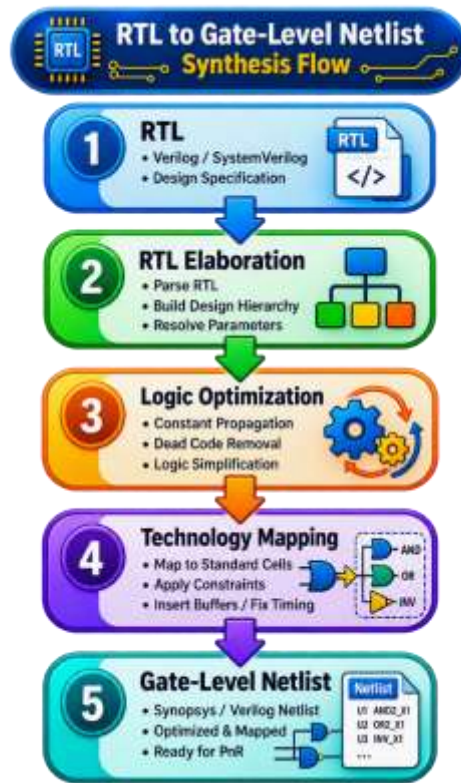
- read\_verilog and\_gate.v
- hierarchy -check -top and\_gate
- read\_liberty -lib <standard\_cell.lib>
- synth -top and\_gate
- abc -liberty <standard\_cell.lib>
- stat
- write\_verilog -noattr synthesized\_netlist.v

Exit Yosys:

- exit

## 17. SYNTHESIS FLOW

- The experiment can be represented as:



## 18. OBSERVATION TABLE

- Record the following values during the experiment:

| Parameter                      | Observation |
|--------------------------------|-------------|
| Yosys Version                  |             |
| RTL File                       |             |
| Top Module                     |             |
| Number of Inputs               |             |
| Number of Outputs              |             |
| Number of Wires                |             |
| Number of Wire Bits            |             |
| Number of Cells Before Mapping |             |
| Number of Cells After Mapping  |             |
| Standard Cell Library          |             |

| Parameter         | Observation |
|-------------------|-------------|
| Generated Netlist |             |

## 19. ANALYSIS

## 20. EXPECTED OUTPUT

## 21. RESULT

## 24. CONCLUSION

## EXPERIMENT–3

### Title

**Develop the Database Using OpenDB for Physical Chip Design**

**Contact Hours:** 2 Hours

### 1. AIM

To develop and initialize a **physical design database using OpenDB** and understand how design, technology, library, placement, routing, and connectivity information are stored for physical chip design.

### 2. OBJECTIVES

After completing this experiment, the student will be able to:

1. Understand the concept of a physical design database.
2. Understand the role of **OpenDB** in the OpenROAD flow.
3. Read technology and standard-cell LEF files.
4. Read the synthesized gate-level netlist.
5. Link the design with the standard-cell library.
6. Initialize the physical design database.
7. Inspect design instances, nets, pins and technology information.
8. Save and reload the OpenDB database.

### 3. SOFTWARE / TOOLS REQUIRED

| S.No. | Software / Tool | Purpose                         |
|-------|-----------------|---------------------------------|
| 1     | Ubuntu/Linux    | Operating environment           |
| 2     | OpenROAD        | Physical design environment     |
| 3     | OpenDB          | Physical design database        |
| 4     | Yosys           | Synthesized netlist generation  |
| 5     | GVim            | Editing Tcl/configuration files |
| 6     | LEF             | Physical library information    |
| 7     | Liberty         | Timing/library information      |
| 8     | Verilog         | Gate-level netlist              |

## 4. THEORY

### 4.1 Introduction to OpenDB

**OpenDB** is the physical design database used by OpenROAD.

It stores information required during different stages of physical implementation.

The database contains information related to:

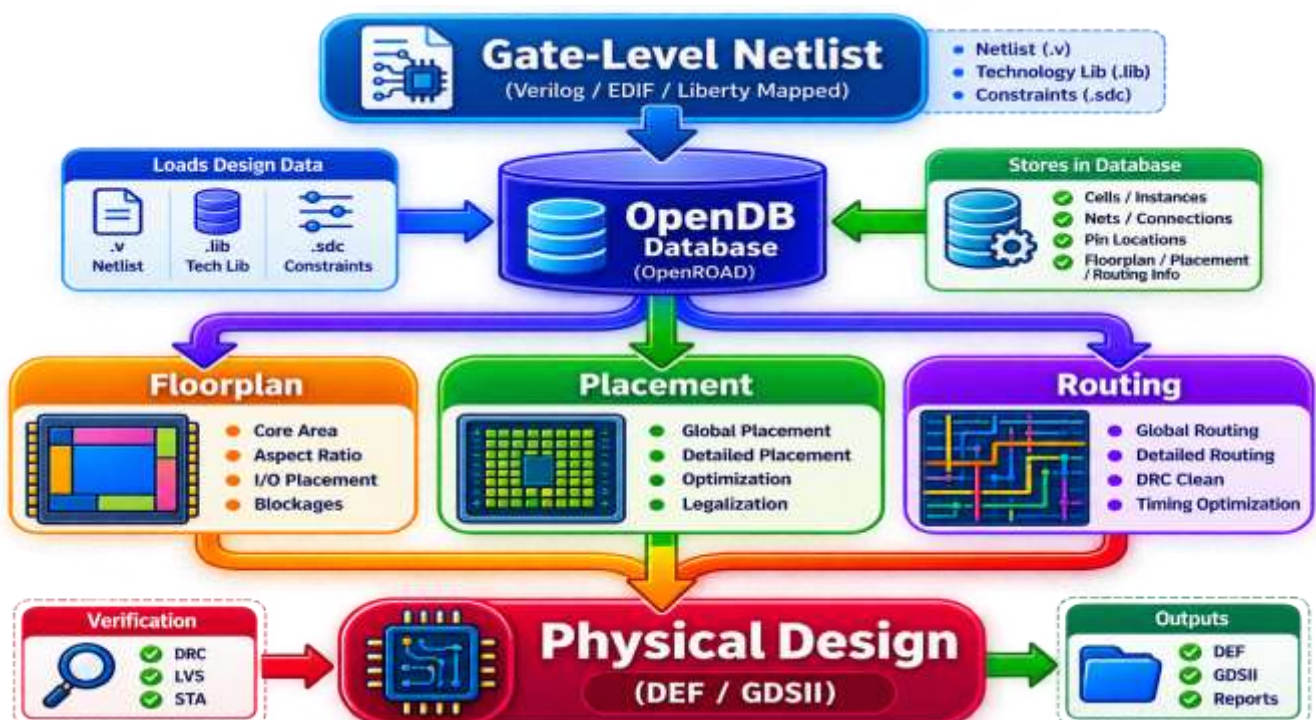
- Technology
- Standard-cell libraries
- Design blocks
- Instances
- Nets
- Pins
- Ports
- Die area
- Core area
- Placement rows
- Routing layers
- Physical shapes
- Placement
- Routing

OpenDB provides the common database infrastructure that allows different OpenROAD engines to operate on the same design representation.

Link: <https://openroad.readthedocs.io/en/latest/main/src/odb/README.html>

## 5. ROLE OF OPENDB IN OPENROAD

The OpenROAD physical-design flow can be represented as:



## 6. IMPORTANT INPUT FILES

### 6.1 LEF File

#### LEF – Library Exchange Format

LEF contains the physical abstract information of technology and cells.

It may contain:

- Metal layers
- Via information
- Cell dimensions
- Cell pins
- Pin layers
- Routing directions
- Placement sites

Example:

- technology.lef  
and
- standard\_cell.lef

### 6.2 Liberty File

The Liberty file contains logical, timing and power information.

Example:

- standard\_cell.lib

### 6.3 Verilog Netlist

The synthesized gate-level Verilog netlist contains:

- Cell instances
- Nets
- Connections
- Inputs
- Outputs

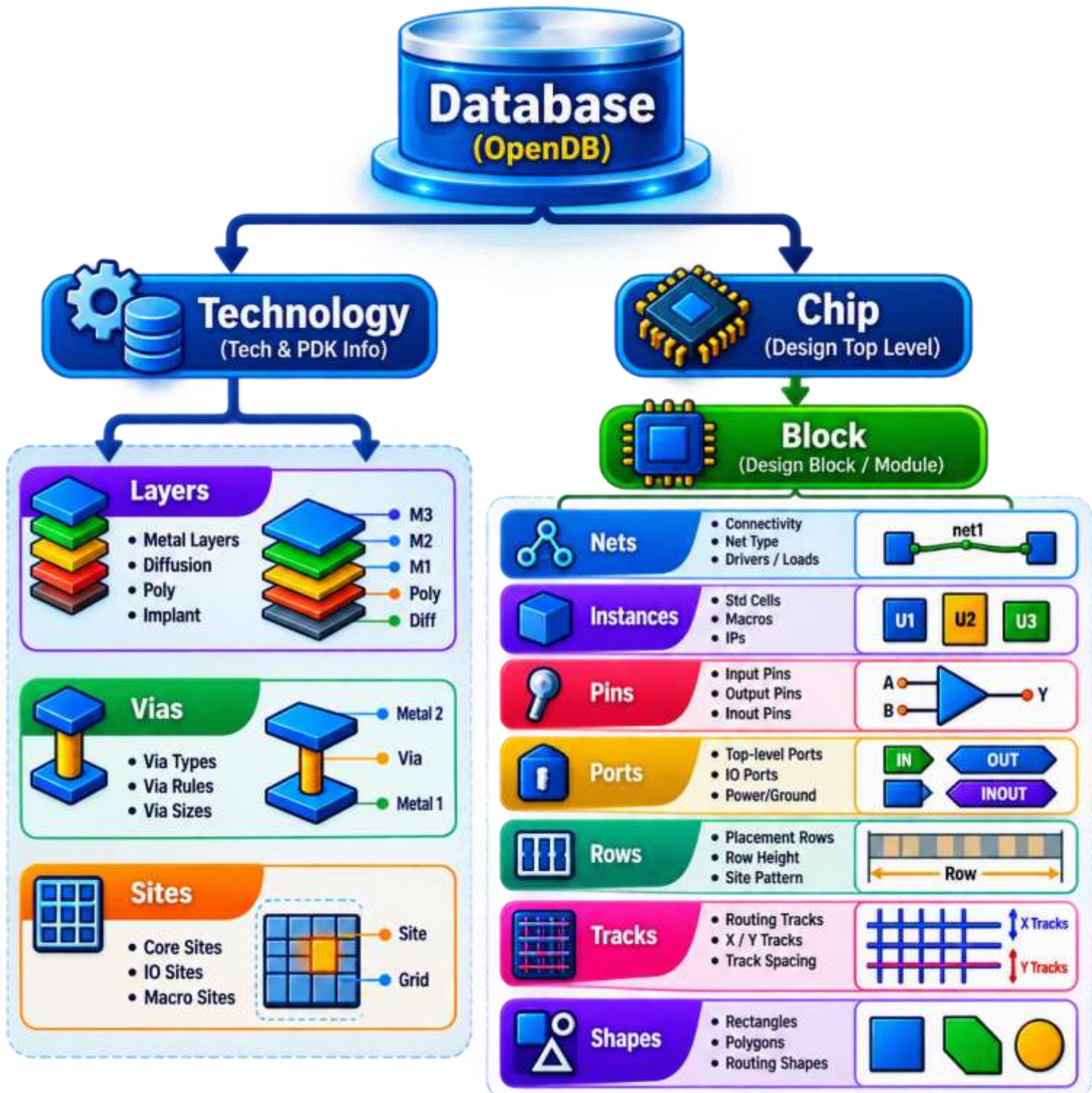
Example:

- synthesized\_netlist.v

## 7. OPENDB DATABASE HIERARCHY

- A simplified OpenDB hierarchy is:

The **Technology** portion describes the process technology, while the **Block** contains the physical design being implemented.



## 8. PROCEDURE

### Step 1: Create a Working Directory

Open the terminal:

- `Cd :mkdir -p ~/openroad_exp3`

Move into the directory:

- `Cd:cd ~/openroad_exp3`

Check the current directory:

- `Cd :pwd`

## 9. STEP 2: CHECK OPENROAD

Execute:

- `openroad -version`

Start OpenROAD:

- `openroad`

You should see:

- `OpenROAD>`

Check the available commands:

- `help`

## 10. STEP 3: READ THE TECHNOLOGY LEF

Inside OpenROAD:

- `read_lef <technology.lef>`

Example:

- `read_lef /path/to/tech.lef`

The technology LEF provides information about:

- Technology layers
- Routing layers
- Via definitions
- Manufacturing grid
- Routing rules

## 11. STEP 4: READ THE STANDARD-CELL LEF

Read the standard-cell LEF:

- `read_lef <standard_cell.lef>`

Example:

- `read_lef /path/to/NangateOpenCellLibrary.lef`

Now OpenROAD has the physical abstracts of the standard cells.

## 12. STEP 5: READ THE LIBERTY FILE

Read the timing library:

- `read_liberty <standard_cell.lib>`

Example:

- `read_liberty /path/to/NangateOpenCellLibrary.lib`

The Liberty file allows OpenROAD to associate logical cell information with the physical cell information from LEF.

## 13. STEP 6: READ THE SYNTHESIZED NETLIST

Read the gate-level netlist generated in Experiment-2:

- `read_verilog synthesized_netlist.v`

This loads:

- Instances
- Nets
- Ports
- Cell connections

into the design.

## 14. STEP 7: LINK THE DESIGN

Specify the top-level module:

- `link_design <TOP_MODULE>`

For example:

- `link_design and_gate`

The linking process connects the instances in the netlist to the corresponding library cells.

## 15. STEP 8: CHECK DESIGN INFORMATION

Run:

- `report_design_area`

You can also use:

- `report_floating_nets`

and:

- `report_design_area`

The reports help verify that the design has been correctly loaded.

## 16. STEP 9: INITIALIZE THE FLOORPLAN

To create the initial physical database, initialize the floorplan.

Example:

- `initialize_floorplan \`
- `-utilization 60 \`
- `-aspect_ratio 1.0 \`
- `-core_space 10 \`
- `-site <SITE>`

For example, if the standard-cell site is:

- `FreePDK45_38x28Site`

the command becomes:

- `initialize_floorplan \`
- `-utilization 60 \`
- `-aspect_ratio 1.0 \`
- `-core_space 10 \`
- `-site FreePDK45_38x28Site`

**Note:** The site name is technology-dependent. Use the site defined in your selected LEF.

## 17. STEP 10: CREATE ROUTING TRACKS

After floorplan initialization:

- `make_tracks`

This creates routing tracks according to the technology information.

The database now contains important physical information such as:

- Die
- Core
- Rows
- Sites
- Routing tracks

## 18. STEP 11: SAVE THE DATABASE

Save the OpenDB database:

- `write_db design.odb`

The generated file is:

- `design.odb`

Check it from Linux:

- `ls -lh design.odb`

## 19. STEP 12: SAVE THE DESIGN AS DEF

The database can also be written as a DEF file:

- `write_def design.def`

Check:

- `ls -lh design.def`

Now the working directory may contain:

- `design.odb`
- `design.def`
- `synthesized_netlist.v`

## 20. STEP 13: READ THE DATABASE AGAIN

Exit OpenROAD:

- `exit`

Start OpenROAD again:

- `openroad`

Read the saved database:

- `read_db design.odb`

Check the design:

- `report_design_area`

This demonstrates that the physical design information has been stored in the OpenDB database and can be restored.

## 21. COMPLETE COMMAND SEQUENCE

The complete experiment can be performed using:

1. `read_lef <technology.lef>`
2. `read_lef <standard_cell.lef>`
3. `read_liberty <standard_cell.lib>`
4. `read_verilog synthesized_netlist.v`
5. `link_design <TOP_MODULE>`
6. `report_design_area`
7. `initialize_floorplan \`
8. `-utilization 60 \`
9. `-aspect_ratio 1.0 \`
10. `-core_space 10 \`
11. `-site <SITE>`
12. `make_tracks`
13. `write_db design.odb`
14. `write_def design.def`

## 22. DATABASE GENERATION FLOW



## 23. OBSERVATION TABLE

| Parameter           | Observation |
|---------------------|-------------|
| OpenROAD Version    |             |
| Technology          |             |
| Technology LEF      |             |
| Standard Cell LEF   |             |
| Liberty File        |             |
| Netlist File        |             |
| Top Module          |             |
| Number of Instances |             |
| Number of Nets      |             |
| Number of Pins      |             |
| Die Area            |             |
| Core Area           |             |
| Utilization         |             |
| Aspect Ratio        |             |
| Site Name           |             |
| OpenDB File         |             |
| DEF File            |             |

## 24. EXPECTED OUTPUT

After successful execution, the following files should be generated:

- design.odb
- design.def

## **25. ANALYSIS**

## **26. RESULT**

## **29. CONCLUSION**

## EXPERIMENT– 4

### Title

### Initialize the Floor Plan with Necessary Specifications

**Contact Hours:** 2 Hours

### 1. AIM

To initialize the **floor plan of a digital chip design** using OpenROAD by defining the die area, core area, utilization, aspect ratio, placement rows, and routing tracks.

### 2. OBJECTIVES

After completing this experiment, the student will be able to:

1. Understand the concept of chip floorplanning.
2. Understand the difference between **die area and core area**.
3. Define the required core utilization.
4. Set the aspect ratio of the floorplan.
5. Create placement rows for standard cells.
6. Generate routing tracks.
7. Inspect the initialized floorplan using OpenROAD GUI.
8. Save the initialized floorplan in DEF and OpenDB formats

### 3. SOFTWARE / TOOLS REQUIRED

| S.No. | Software / Tool | Purpose                                  |
|-------|-----------------|------------------------------------------|
| 1     | Ubuntu/Linux    | Operating environment                    |
| 2     | OpenROAD        | Physical design                          |
| 3     | OpenDB          | Design database                          |
| 4     | LEF             | Technology and physical cell information |
| 5     | Liberty         | Timing/library information               |
| 6     | Verilog         | Gate-level netlist                       |
| 7     | GVim            | Editing Tcl scripts                      |

## 4. THEORY

### 4.1 Introduction to Floorplanning

**Floorplanning** is the first major physical-design stage after synthesis and database initialization.

The purpose of floorplanning is to define:

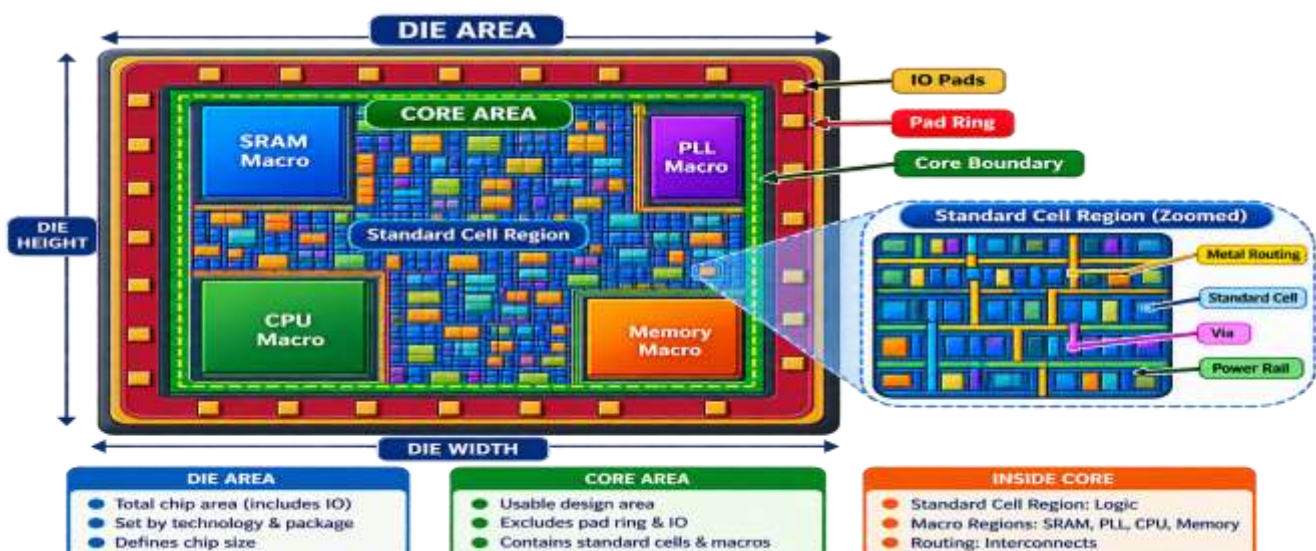
- Die dimensions
- Core dimensions
- Core utilization
- Aspect ratio
- Standard-cell placement area
- Placement rows
- Routing tracks
- Macro locations
- I/O regions
- Power planning regions

A good floorplan is important because it affects:

- Placement congestion
- Routing congestion
- Timing
- Power distribution
- Wirelength
- IR drop
- Overall chip area

## 5. DIE AREA AND CORE AREA

- A chip floorplan consists mainly of two regions:



### Die Area

The **die area** represents the complete physical chip boundary.

### Core Area

The **core area** is the region inside the die where standard cells and other core-level physical structures are placed.

## 6. CORE UTILIZATION

Core utilization represents how much of the available core area is occupied by standard-cell area.

It can be represented approximately as:

$$\text{Core Utilization} = \frac{\text{Standard Cell Area}}{\text{Core Area}} \times 100$$

For example, if the target utilization is:

60%

approximately 60% of the available placement area is targeted for standard-cell placement.

### Why not use 100% utilization?

Because routing requires free space between cells.

Very high utilization can result in:

- Placement congestion
- Routing congestion
- Timing degradation
- Difficult legalization
- Routing failures

Therefore, a reasonable target utilization is normally selected according to the design and technology.

## 7. ASPECT RATIO

The **aspect ratio** determines the shape of the core.

$$\text{Aspect Ratio} = \frac{\text{Core Height}}{\text{Core Width}}$$

For:

Aspect Ratio = 1.0

the core is approximately square.

Example:

Aspect Ratio = 1.0



Square floorplan

Whereas:

Aspect Ratio > 1



Taller core

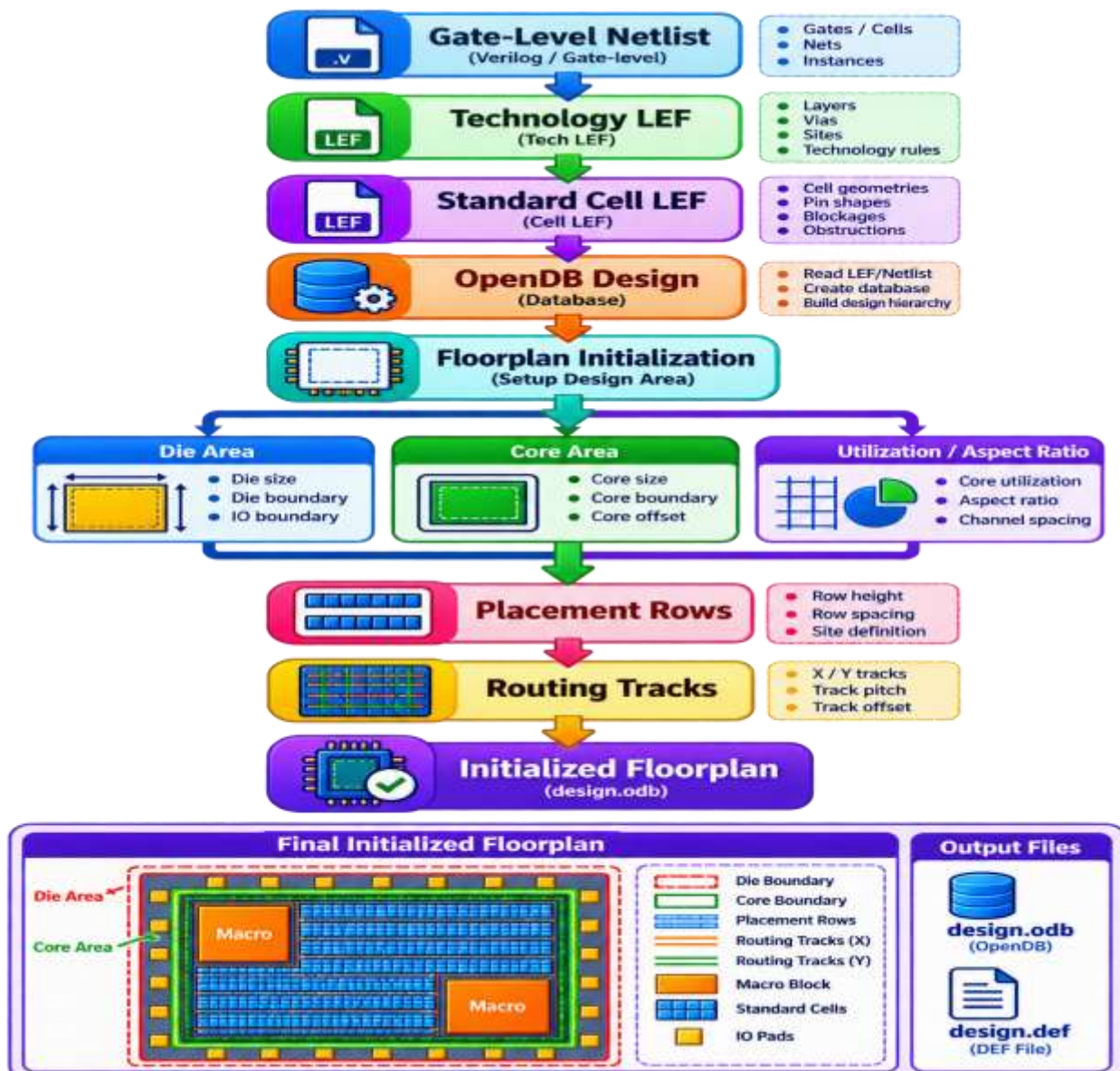
and:

Aspect Ratio < 1



Wider core

## 8. FLOORPLAN INITIALIZATION FLOW



## 9. PROCEDURE

### Step 1: Create a Working Directory

Open the terminal:

- `mkdir -p ~/openroad_exp4`

Move into the directory:

- `cd ~/openroad_exp4`

Check the current directory:

- `pwd`

## 10. STEP 2: Start OpenROAD

Execute:

- `openroad`

The OpenROAD prompt appears:

- `OpenROAD>`

Check the version:

- `version`

## 11. STEP 3: READ TECHNOLOGY LEF

Read the technology LEF:

- `read_lef <technology.lef>`

Example:

- `read_lef /path/to/tech.lef`

The technology LEF provides information about:

- Metal layers
- Via layers
- Manufacturing grid
- Routing rules
- Placement sites

## 12. STEP 4: READ STANDARD-CELL LEF

Read the standard-cell LEF:

- `read_lef <standard_cell.lef>`

Example:

- `read_lef /path/to/standard_cell.lef`

The standard-cell LEF provides:

- Cell dimensions
- Cell pins
- Pin locations
- Cell obstructions
- Placement information

### 13. STEP 5: READ LIBERTY FILE

Read the standard-cell timing library:

- `read_liberty <standard_cell.lib>`

Example:

- `read_liberty /path/to/standard_cell.lib`

### 14. STEP 6: READ THE SYNTHESIZED NETLIST

Read the synthesized netlist generated in Experiment-2:

- `read_verilog synthesized_netlist.v`

### 15. STEP 7: LINK THE DESIGN

Specify the top-level module:

- `link_design <TOP_MODULE>`

For example:

- `link_design and_gate`

Check the design:

- `report_design_area`

### 16. STEP 8: INITIALIZE THE FLOORPLAN

Use the `initialize_floorplan` command.

**Example:**

```
initialize_floorplan \
-utilization 60 \
-aspect_ratio 1.0 \
-core_space 10 \
-site <SITE>
```

**Here:**

| Parameter         | Meaning                       |
|-------------------|-------------------------------|
| -utilization 60   | Target core utilization = 60% |
| -aspect_ratio 1.0 | Square core                   |
| -core_space 10    | Core-to-die spacing           |

| Parameter    | Meaning                      |
|--------------|------------------------------|
| -site <SITE> | Standard-cell placement site |

**Important:** The site name is technology-dependent. Use the site name defined in your LEF.

## 17. STEP 9: CHECK FLOORPLAN

After initialization, inspect the floorplan.

Run:

- report\_design\_area

You can also check the design database using:

- report\_units

If available in your OpenROAD version, use:

- check\_placement

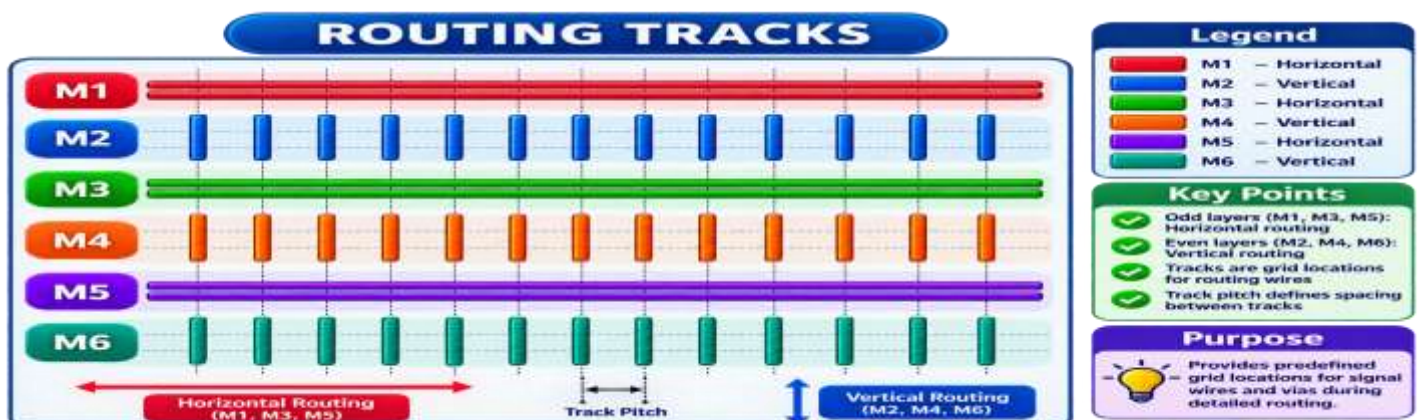
## 18. STEP 10: GENERATE ROUTING TRACKS

Execute:

- make\_tracks

Routing tracks provide legal locations for routing wires on different metal layers.

The routing structure can be visualized as:



Different layers generally have preferred routing directions.

## 19. STEP 11: OPEN THE GUI

You can inspect the floorplan using:

- `openroad -gui`

Alternatively, if OpenROAD is already running with GUI support, use the GUI from the current session.

Inspect:

- Die boundary
- Core boundary
- Placement rows
- Standard-cell region
- Routing tracks
- I/O locations if present
- Macros if present

## 20. STEP 12: SAVE THE FLOORPLAN

Write the floorplan to DEF:

- `write_def floorplan.def`

Save the OpenDB database:

- `write_db floorplan.odb`

Check the files:

- `ls -lh`

Expected output includes:

- `floorplan.def`
- `floorplan.odb`

## 21. COMPLETE OPENROAD COMMAND SEQUENCE

The complete basic sequence is:

1. `read_lef <technology.lef>`
2. `read_lef <standard_cell.lef>`
3. `read_liberty <standard_cell.lib>`
4. `read_verilog synthesized_netlist.v`
5. `link_design <TOP_MODULE>`
6. `report_design_area`
7. `initialize_floorplan \`
8. `-utilization 60 \`
9. `-aspect_ratio 1.0 \`
10. `-core_space 10 \`
11. `-site <SITE>`
12. `make_tracks`
13. `check_placement`

14. write\_def floorplan.def
15. write\_db floorplan.odg

## **22. FLOORPLAN PARAMETERS**

### **22.1 Utilization**

Example:

- -utilization 60

Target utilization:

- 60%

### **22.2 Aspect Ratio**

Example:

- -aspect\_ratio 1.0

This creates a square-like core.

### **22.3 Core Space**

Example:

- -core\_space 10

This defines spacing between the core and die boundary when using automatic floorplan sizing.

### **22.4 Explicit Floorplan Coordinates**

Instead of utilization-based sizing, explicit die/core coordinates can also be specified.

For example:

```
initialize_floorplan \  
-die_area "0 0 1000 1000" \  
-core_area "100 100 900 900" \  
-site <SITE>
```

This defines:

Die:

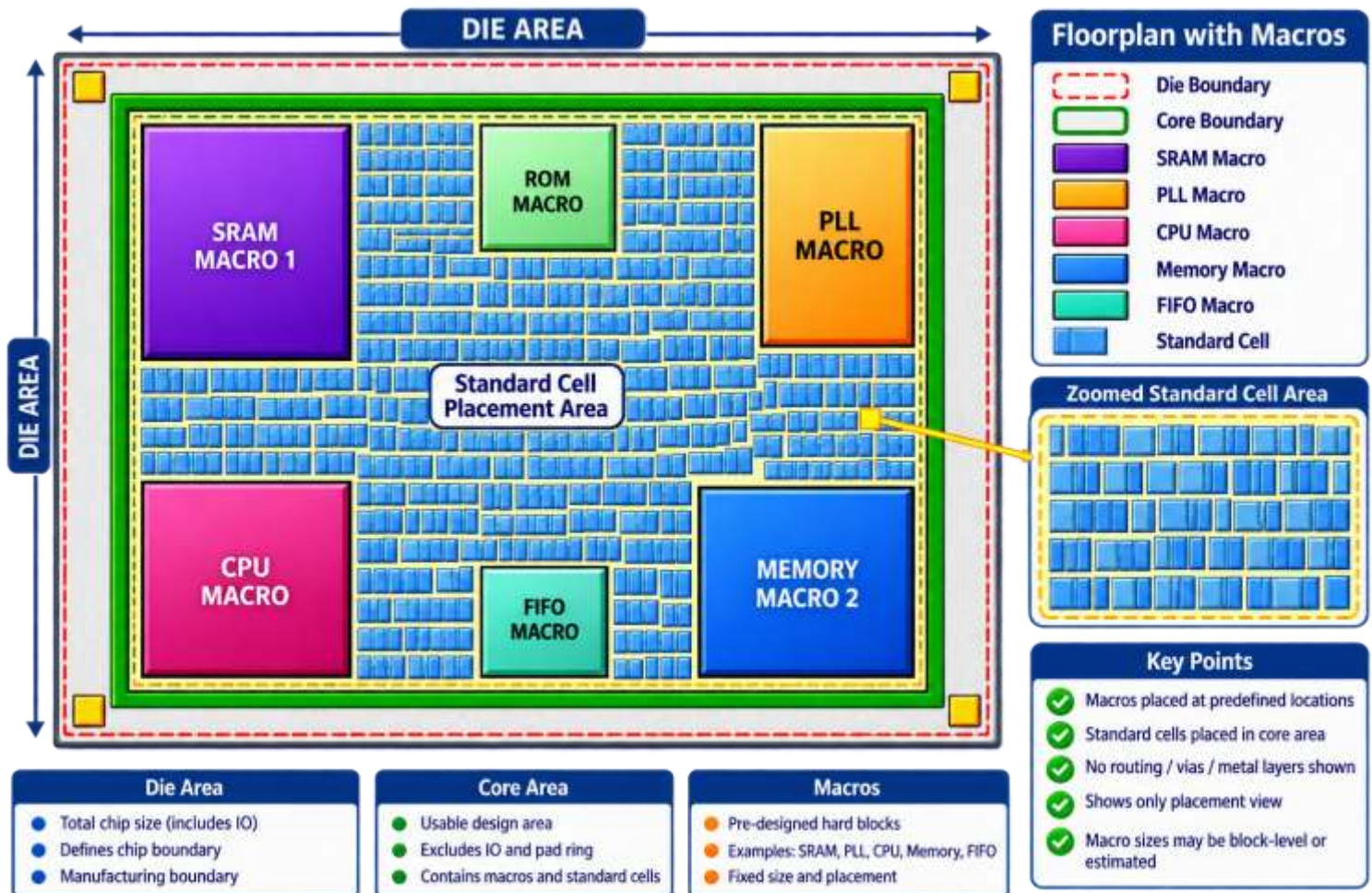
- (0,0) → (1000,1000)

Core:

- (100,100) → (900,900)

The exact values should be selected according to the target design and technology.

## 23. FLOORPLAN WITH MACROS



Macros should be placed carefully to avoid:

- Routing congestion
- Narrow channels
- Notches
- Timing problems
- Power-grid problems

## 24. OBSERVATION TABLE

Record the following values:

| Parameter             | Observation |
|-----------------------|-------------|
| OpenROAD Version      |             |
| Design Name           |             |
| Top Module            |             |
| Technology            |             |
| Standard Cell Library |             |
| Site Name             |             |
| Target Utilization    |             |
| Aspect Ratio          |             |
| Die Width             |             |
| Die Height            |             |
| Core Width            |             |
| Core Height           |             |
| Die Area              |             |
| Core Area             |             |
| Number of Rows        |             |
| Routing Tracks        |             |
| Floorplan DEF         |             |
| Floorplan ODB         |             |

## **25. ANALYSIS**

## **26. EXPECTED OUTPUT**

## **27. RESULT**

## **30. CONCLUSION**

## EXPERIMENT-5

### Title

**Develop Power Grid Planning Using PDNGEN**

**Contact Hours:** 2 Hours

### 1. AIM

To develop a **Power Distribution Network (PDN)** for a digital chip using **PDNGEN in OpenROAD**, and to analyze the connectivity of VDD and VSS through power rings, straps, rails, and vias.

### 2. OBJECTIVES

After completing this experiment, the student will be able to:

1. Understand the concept of power planning in physical design.
2. Understand the structure of a Power Distribution Network.
3. Understand VDD and VSS power connections.
4. Create global power and ground connections.
5. Define a voltage domain.
6. Generate power rings.
7. Generate power straps.
8. Generate standard-cell power rails.
9. Connect different metal layers using vias.
10. Verify the generated PDN using OpenROAD.

### 3. SOFTWARE / TOOLS REQUIRED

| S.No. | Software / Tool | Purpose                             |
|-------|-----------------|-------------------------------------|
| 1     | Ubuntu/Linux    | Operating environment               |
| 2     | OpenROAD        | Physical design                     |
| 3     | PDNGEN          | Power-grid generation               |
| 4     | OpenDB          | Design database                     |
| 5     | LEF             | Technology and physical information |
| 6     | Liberty         | Timing/library information          |
| 7     | DEF             | Physical design information         |
| 8     | Verilog         | Gate-level netlist                  |
| 9     | GVim            | Tcl/script editing                  |

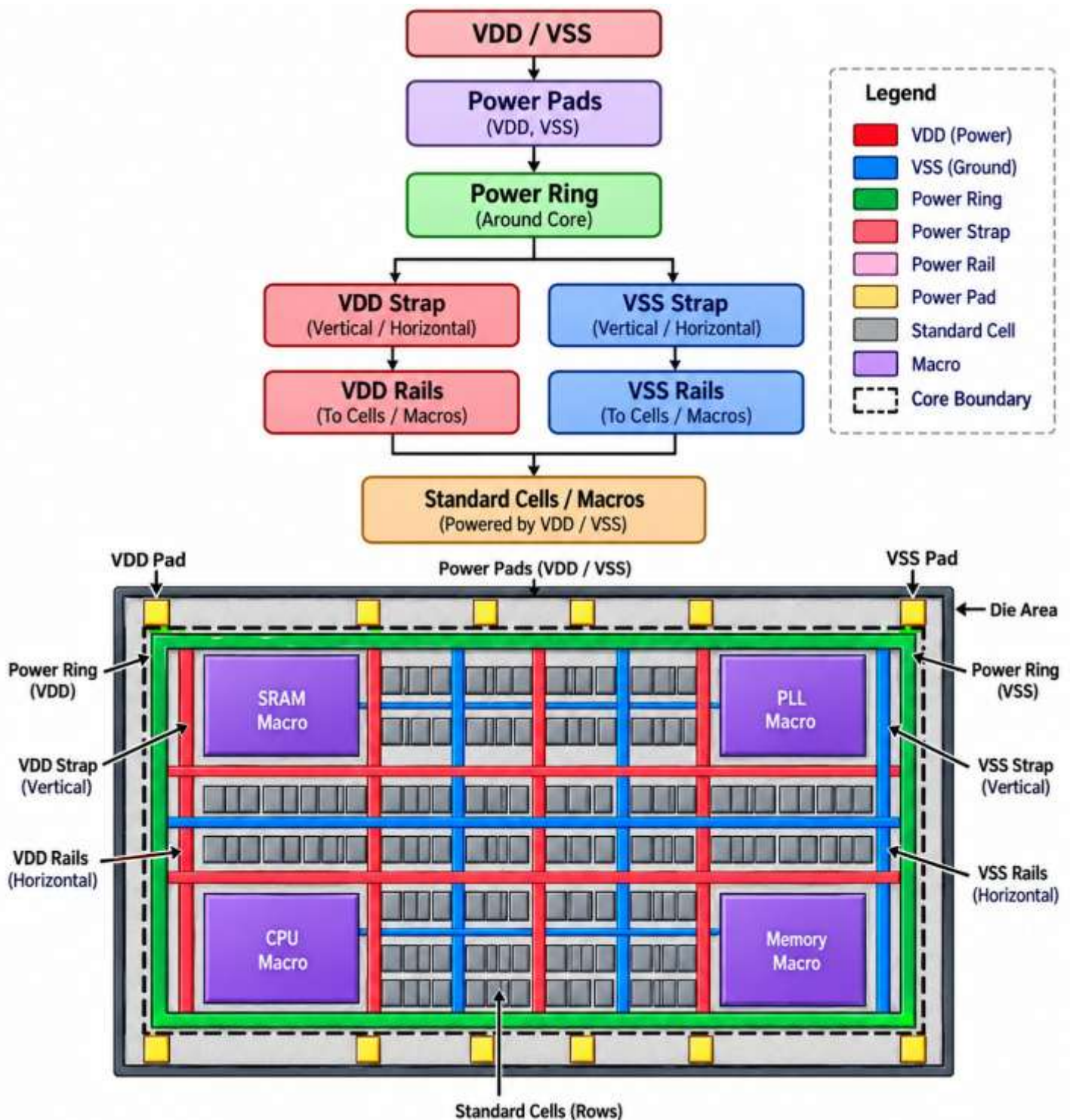
## 4. THEORY

### 4.1 Introduction to Power Planning

**Power planning** is an important physical-design stage in which the power and ground network of the chip is created.

The main purpose of the power distribution network is to deliver a stable supply voltage to all standard cells, macros and other circuit blocks.

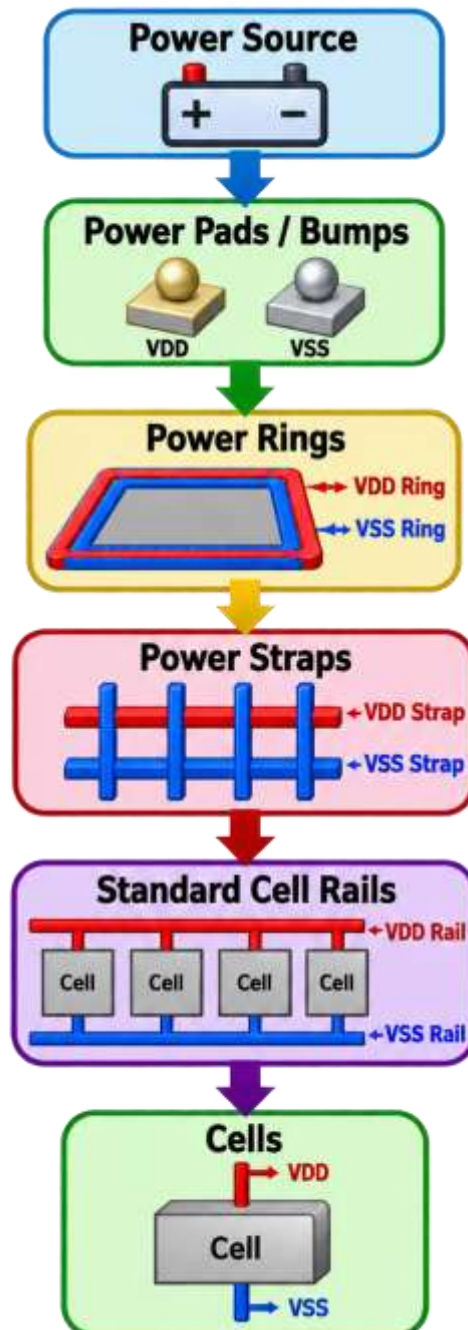
A simplified power network is:



## 5. WHAT IS PDN?

**PDN** stands for **Power Distribution Network**.

It provides electrical connections between:



The two primary supply nets are:

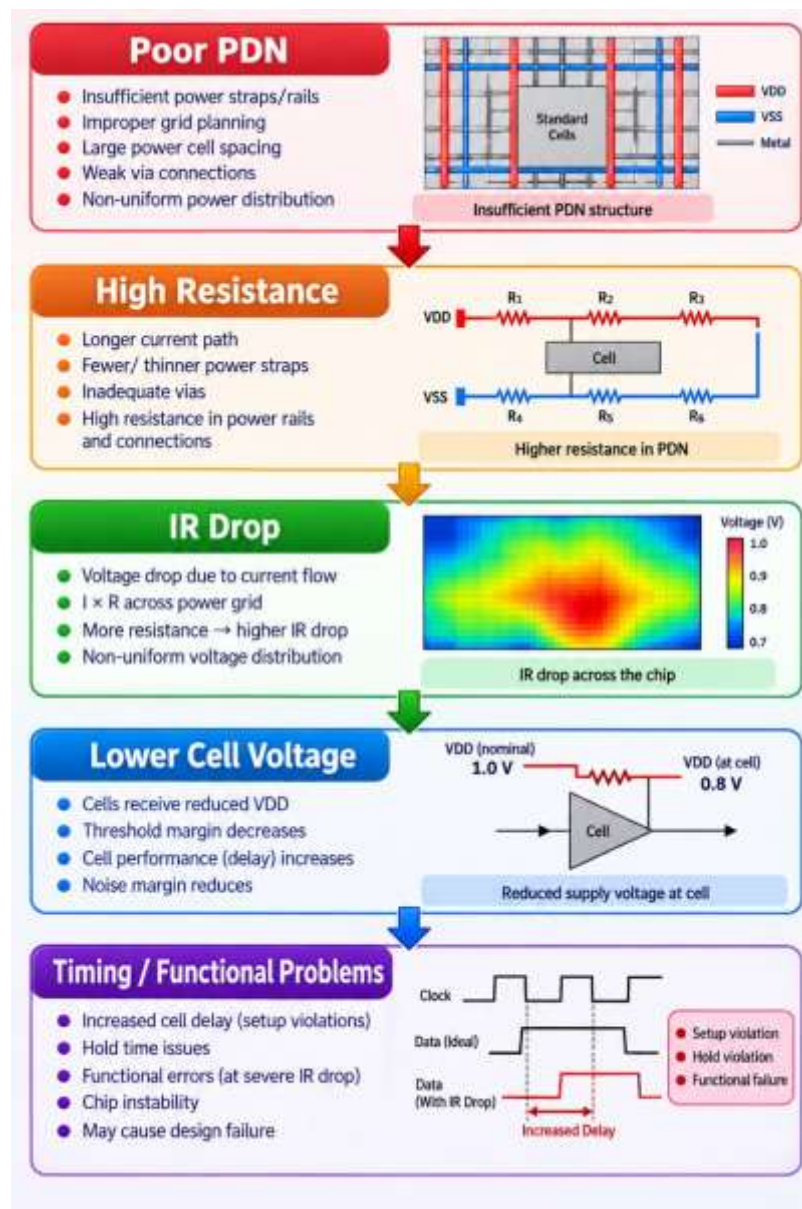
- **VDD** – Power supply
- **VSS** – Ground

## 6. IMPORTANCE OF POWER PLANNING

A properly designed PDN should:

- Deliver sufficient voltage to all cells.
- Minimize IR drop.
- Reduce electromigration risk.
- Provide low-resistance power paths.
- Maintain VDD/VSS connectivity.
- Support high switching activity.
- Provide adequate current capacity.

Poor power planning can cause:



## 7. POWER DISTRIBUTION NETWORK STRUCTURE

A typical PDN consists of:

### 1. Power Pads / Bumps

Provide external VDD/VSS connections.

### 2. Power Rings

Large metal structures around the core.

### 3. Power Straps

Horizontal and vertical metal lines that distribute power across the core.

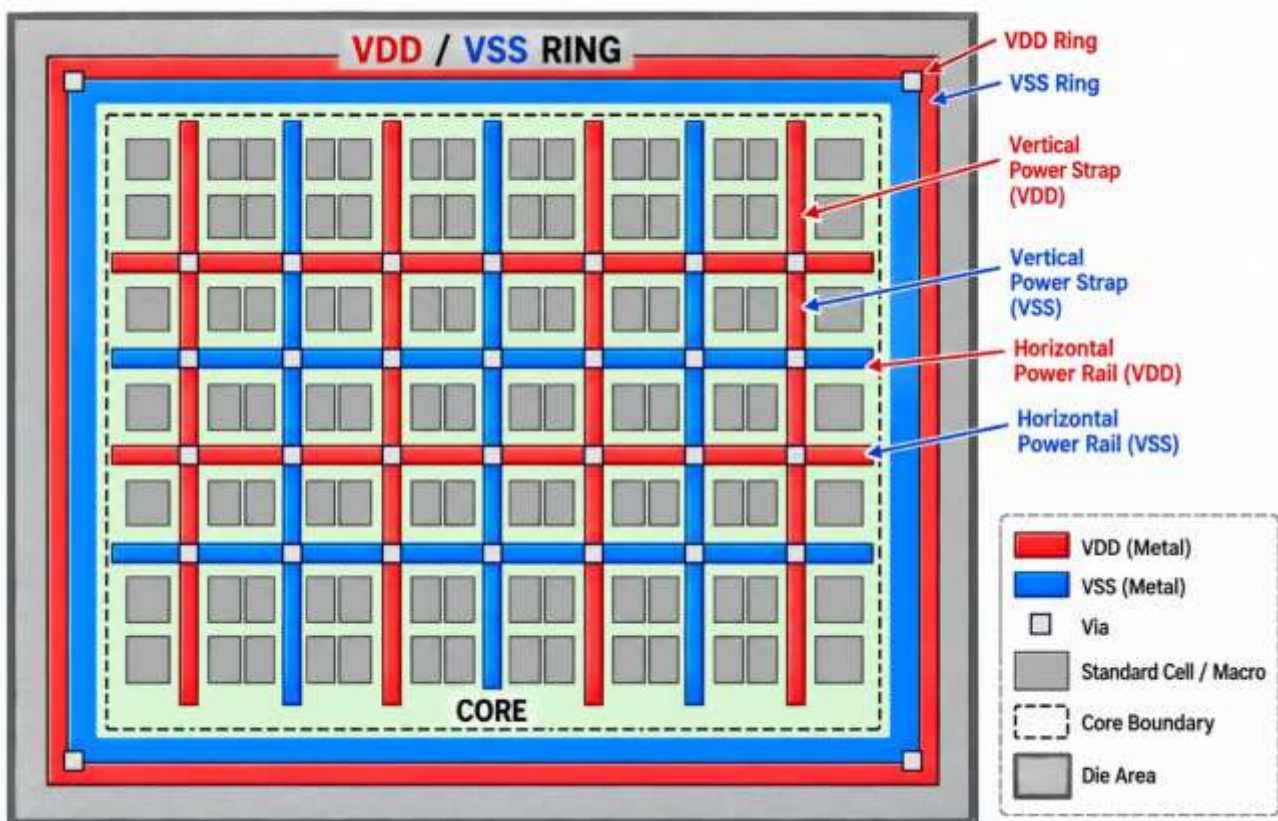
### 4. Follow-Pin Rails

Power rails that follow standard-cell rows.

### 5. Vias

Connect power structures between different metal layers.

## 8. TYPICAL PDN STRUCTURE



The vertical and horizontal structures form a mesh that distributes VDD and VSS.

## 9. PDNGEN

**PDNGEN** is the OpenROAD power-grid generation module.

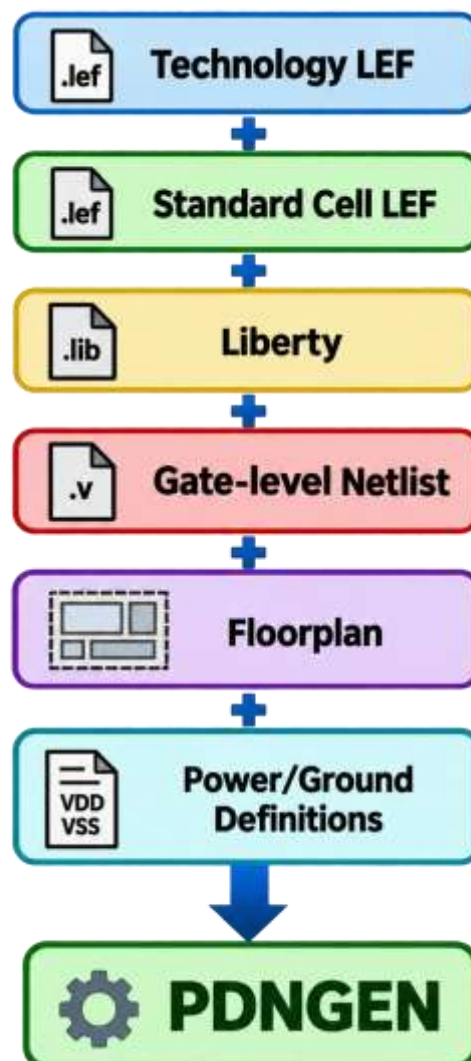
It generates the physical power network according to a specified PDN configuration.

PDNGEN can create:

- Rings
- Straps
- Follow-pin rails
- Layer connections
- Power-grid structures

## 10. INPUTS REQUIRED

The experiment requires:



The floorplan generated in **Experiment-4** is used as the starting point.

## 11. PROCEDURE

### Step 1: Create Working Directory

Open the terminal:

- `Cd: mkdir -p ~/openroad_exp5`

Move into it:

- `Cd: cd ~/openroad_exp5`

### 12. STEP 2: START OPENROAD

Run:

- `openroad`

Check the version:

- `version`

### 13. STEP 3: LOAD THE FLOORPLAN DATABASE

If the database generated in Experiment-4 is available:

- `read_db floorplan.odb`

Alternatively, load the required files:

- `read_lef <technology.lef>`
- `read_lef <standard_cell.lef>`
- `read_liberty <standard_cell.lib>`
- `read_verilog <synthesized_netlist.v>`
- `link_design <TOP_MODULE>`

Then initialize the required floorplan if it has not already been saved.

### 14. STEP 4: CHECK THE DESIGN

Run:

- `report_design_area`

Check placement:

- `check_placement`

Make sure the floorplan is valid before creating the PDN.

### 15. STEP 5: CREATE GLOBAL POWER CONNECTION

The standard cells must be connected to the global VDD net.

**Example:tcl**

```
add_global_connection \  
-net VDD \  
-pin_pattern {^VDD$} \  

```

-power

This tells OpenROAD that pins matching VDD should connect to the global power net.

## 16. STEP 6: CREATE GLOBAL GROUND CONNECTION

Create the VSS connection:

Tcl:

```
add_global_connection \  
-net VSS \  
-pin_pattern {^VSS$} \  
-ground
```

Now the design has:

- VDD → Power
- VSS → Ground

Note: Standard-cell power-pin names are technology/library dependent. In your library they may be VDD, VSS, VPWR, VGND, or other names. Use the actual names from your LEF/Liberty.

## 17. STEP 7: DEFINE VOLTAGE DOMAIN

Define the core voltage domain:

Tcl:

```
set_voltage_domain \  
-power VDD \  
-ground VSS
```

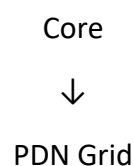
The voltage domain identifies the power and ground nets used by the core.

## 18. STEP 8: DEFINE PDN GRID

Create a PDN grid:

- define\_pdn\_grid -name Core

Here:



The grid will contain the power structures used to distribute VDD and VSS.

### 19. STEP 9: ADD POWER RING

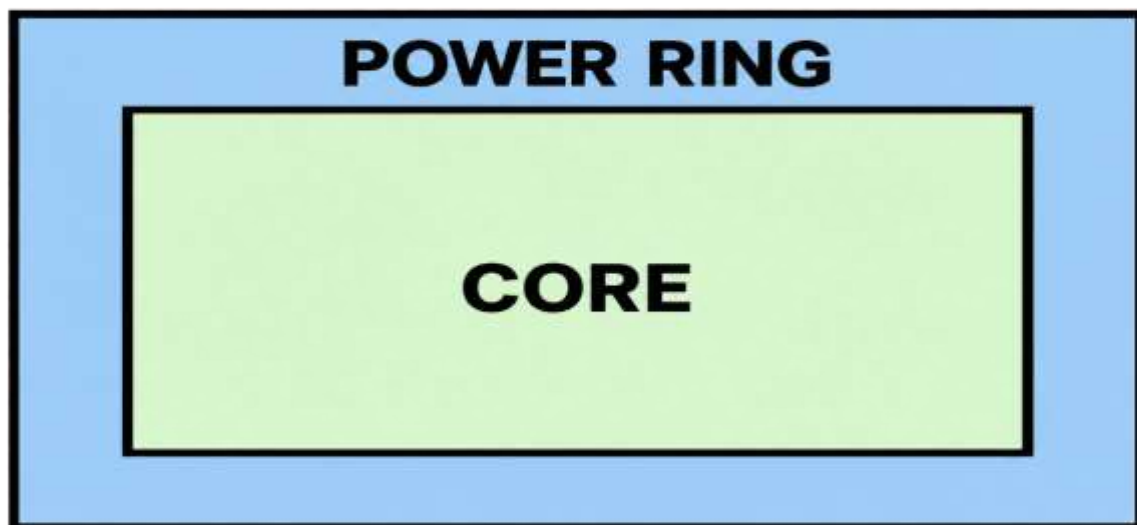
A power ring is created around the core.

**Example:**

```
add_pdn_ring \  
-grid Core \  
-layers {M8 M9} \  
-widths {5.0 5.0} \  
-spacings {2.0 2.0} \  
-core_offsets {4.5 4.5}
```

The exact metal layers, widths, spacings and offsets are technology dependent.

Conceptually:



### 20. STEP 10: ADD FOLLOW-PIN RAILS

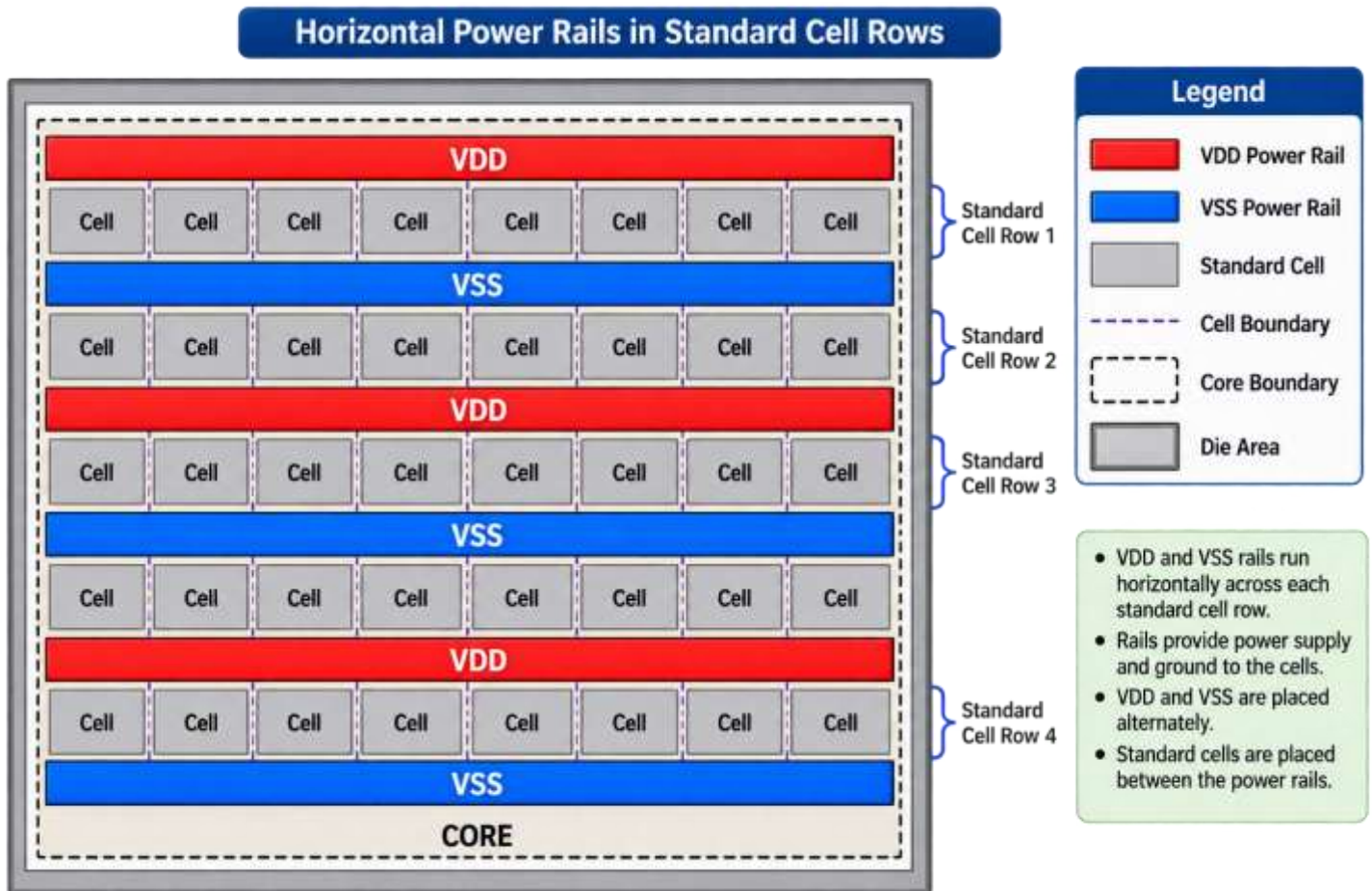
Standard cells require power and ground rails.

**Example:**

```
add_pdn_stripe \  
-grid Core \  
-followpins \  
-layer M1
```

This creates follow-pin power rails along standard-cell rows.

Conceptually:



## 21. STEP 11: ADD POWER STRAPS

Add vertical/horizontal straps on higher metal layers.

Example:

Tcl:

```

add_pdn_stripe \
-grid Core \
-layer M4 \
-width 0.48 \
-pitch 56.0 \
-offset 2.0

```

Another layer can be added:

Tcl:

```

add_pdn_stripe \
-grid Core \
-layer M5 \
-width 0.48 \

```

-pitch 56.0 \

-offset 2.0

The straps form the power mesh.

## **22. STEP 12: CONNECT DIFFERENT METAL LAYERS**

Power structures on different layers must be electrically connected.

**Example: tcl:**

```
add_pdn_connect \  
-grid Core \  
-layers {M1 M4}
```

Another connection:

**tcl:**

```
add_pdn_connect \  
-grid Core \  
-layers {M4 M5}
```

Vias are automatically generated according to the PDN configuration and technology rules.

## **23. STEP 13: GENERATE THE PDN**

Execute:

- pdngen

PDNGEN now generates the specified:

- Power Rings
- Power Straps
- Follow-Pin Rails
- Vias
- VDD Connections
- VSS Connections

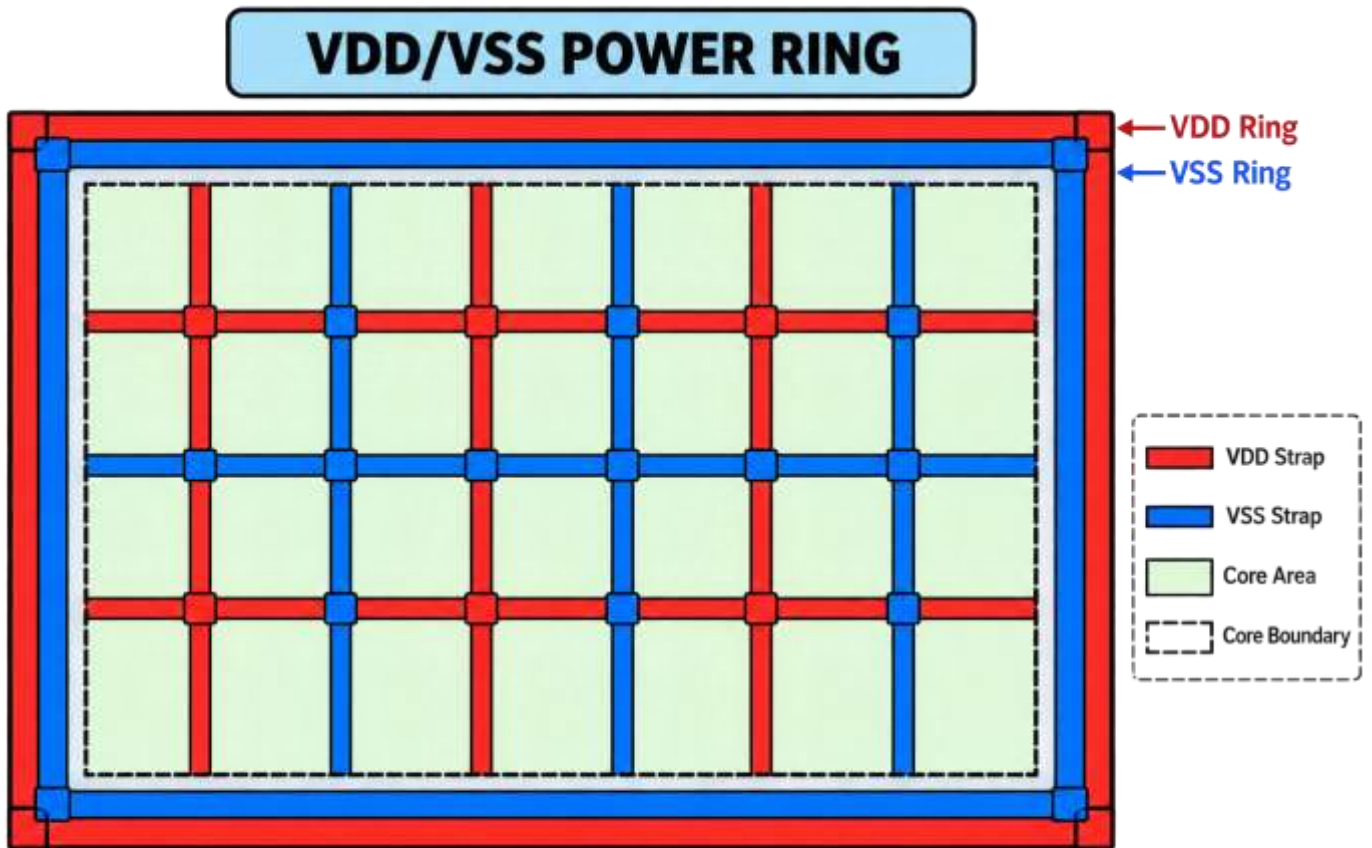
## **24. STEP 14: CHECK POWER GRID**

Inspect the generated PDN in the OpenROAD GUI.

Start GUI:

- openroad -gui

The PDN should look conceptually like:



## 25. STEP 15: CHECK FOR FLOATING PDN STRIPES

Run the PDN analysis/checking commands supported by your OpenROAD version.

A typical verification is:

- `check_power_grid`

If the command is not available in your installed version, use the PDN/IR analysis commands provided by that version and inspect the generated PDN visually and through the logs.

The objective is to identify:

- Floating stripes
- Missing connections
- Disconnected rails
- Incorrect VDD/VSS connections

## 26. STEP 16: SAVE THE PDN DATABASE

Save the database:

- `write_db pdn.odb`

Write the DEF:

- `write_def pdn.def`

Check the generated files:

- ls -lh
- Expected:
- pdn.odb
- pdn.def

## 27. COMPLETE PDNGEN COMMAND SEQUENCE

A representative sequence is: **tcl:**

```
read_db floorplan.odb
```

```
add_global_connection \  
-net VDD \  
-pin_pattern {^VDD$} \  
-power
```

```
add_global_connection \  
-net VSS \  
-pin_pattern {^VSS$} \  
-ground
```

```
set_voltage_domain \  
-power VDD \  
-ground VSS
```

```
define_pdn_grid -name Core
```

```
add_pdn_ring \  
-grid Core \  
-layers {M8 M9} \  
-widths {5.0 5.0} \  
-spacings {2.0 2.0} \  
-core_offsets {4.5 4.5}
```

```
add_pdn_stripe \  
-grid Core \  
-followpins \  
-layer M1
```

```
add_pdn_stripe \  
-grid Core \  
-layer M4 \  
-width 0.48 \  

```

-pitch 56.0 \  
-offset 2.0

add\_pdn\_stripe \  
-grid Core \  
-layer M5 \  
-width 0.48 \  
-pitch 56.0 \  
-offset 2.0

add\_pdn\_connect \  
-grid Core \  
-layers {M1 M4}

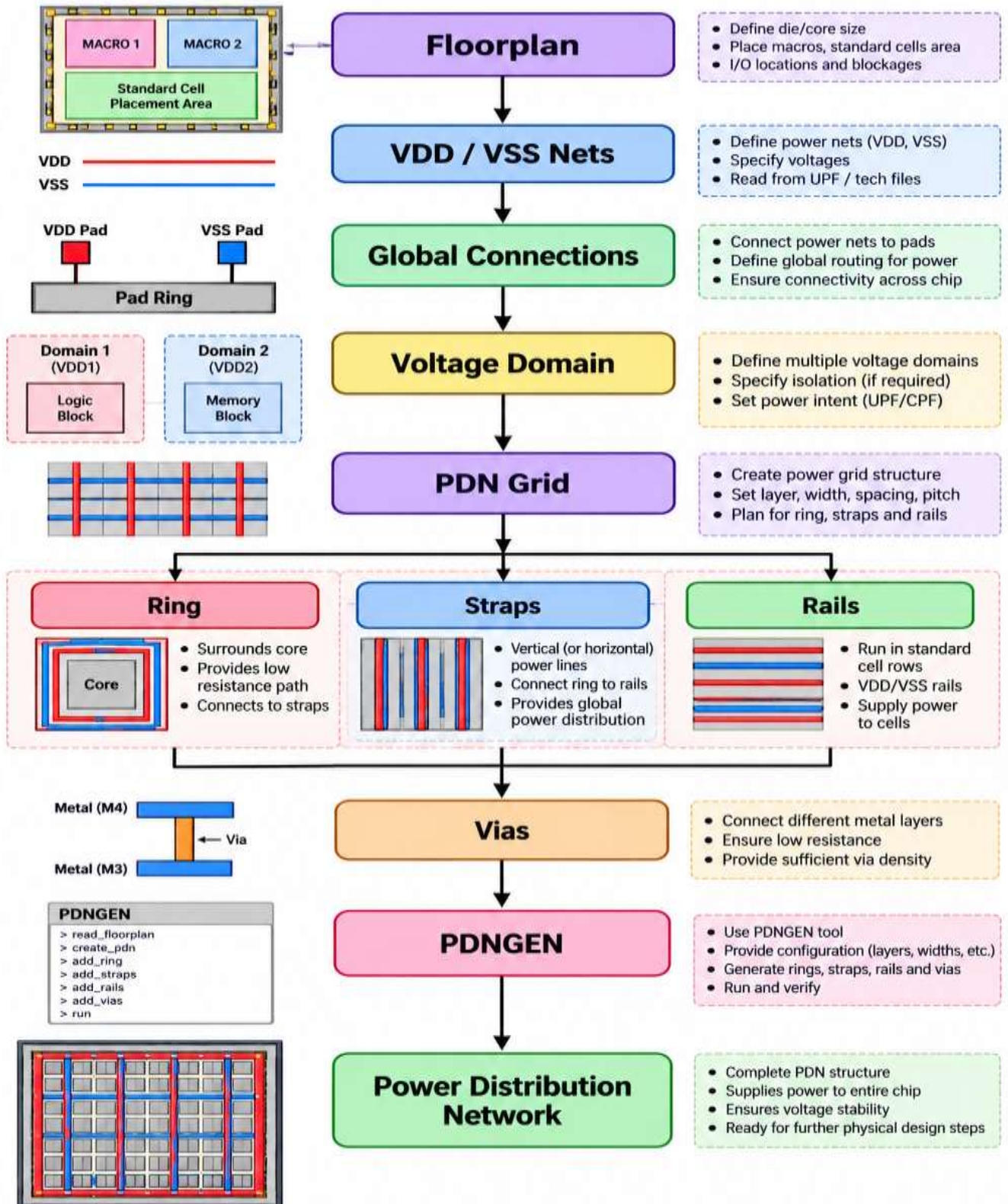
add\_pdn\_connect \  
-grid Core \  
-layers {M4 M5}

pdngen

- write\_db pdn.odb
- write\_def pdn.def

**Important:** The metal-layer names and numerical values above are examples. Use the values defined for your selected technology/PDK.

## 28. PDN GENERATION FLOW



## 29. OBSERVATION TABLE

| Parameter               | Observation |
|-------------------------|-------------|
| OpenROAD Version        |             |
| Design Name             |             |
| Technology              |             |
| Power Net               |             |
| Ground Net              |             |
| Voltage Domain          |             |
| PDN Grid Name           |             |
| Ring Layers             |             |
| Ring Width              |             |
| Ring Spacing            |             |
| Strap Layers            |             |
| Strap Width             |             |
| Strap Pitch             |             |
| Follow-Pin Layer        |             |
| Number of Power Straps  |             |
| Number of Ground Straps |             |
| PDN DEF                 |             |
| PDN ODB                 |             |

## 30. ANALYSIS

### **33. EXPECTED OUTPUT**

### **34. RESULT**

### **37. CONCLUSION**

### EXPERIMENT-6

#### Title

**Learn the Pin Placement Using ioPlacer**

**Contact Hours: 2 Hours**

#### 1. AIM

To perform I/O pin placement using the ioPlacer (PPL) module in OpenROAD and understand how input and output pins are positioned on the boundary of the chip according to specified placement constraints.

#### 2. OBJECTIVES

**After completing this experiment, the student will be able to:**

1. Understand the concept of I/O pin placement.
2. Understand the role of ioPlacer/PPL in physical design.
3. Identify input and output ports in a design.
4. Define pin-placement constraints.
5. Specify horizontal and vertical routing layers for pins.
6. Place I/O pins along the chip boundary.
7. Analyze pin spacing and routing accessibility.
8. Save the pin-placement information for further physical-design stages.

#### 3. SOFTWARE / TOOLS REQUIRED

| S.No. | Software / Tool | Purpose                         |
|-------|-----------------|---------------------------------|
| 1     | Ubuntu/Linux    | Operating environment           |
| 2     | OpenROAD        | Physical design                 |
| 3     | ioPlacer / PPL  | I/O pin placement               |
| 4     | OpenDB          | Design database                 |
| 5     | LEF             | Technology/physical information |
| 6     | Liberty         | Timing information              |
| 7     | DEF             | Physical design information     |
| 8     | Verilog         | Gate-level netlist              |
| 9     | GVim            | Tcl/script editing              |

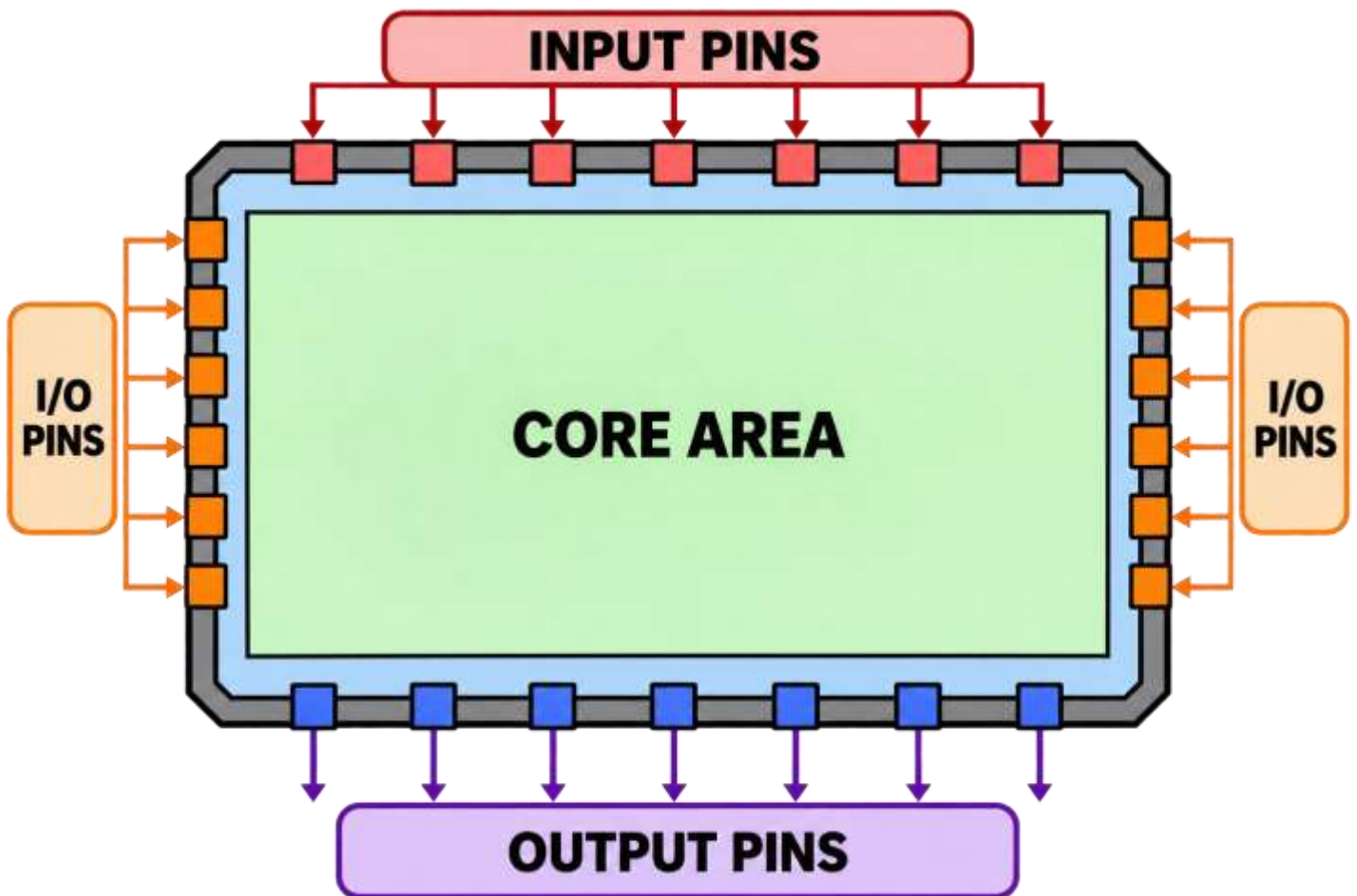
## 4. THEORY

### 4.1 Introduction to I/O Pin Placement

**I/O pin placement** is the process of assigning physical locations to the input and output pins of a chip.

I/O pins are generally placed around the **boundary of the core or die** so that signals can enter and leave the chip.

A simplified representation is:



Good I/O pin placement is important because it affects:

- Routing congestion
- Wirelength
- Timing
- Signal integrity
- Routing accessibility
- Chip-level connectivity

## 5. WHAT IS ioPlacer?

**ioPlacer**, commonly associated with OpenROAD's **PPL (Pin Placer)** module, is used to automatically place I/O pins along the boundary of the design.

It can consider:

- Pin direction
- Pin location
- Pin layer
- Pin spacing
- Pin order
- Region constraints
- Routing tracks

The pin placer can optimize pin locations to reduce estimated wirelength while respecting the specified constraints.

## 6. I/O PIN TYPES

The main types of design I/O pins are:

### 6.1 Input Pins

Input pins receive signals from outside the design.

Example:

- A ———▶
- B ———▶
- CLK ———▶
- RST ———▶

### 6.2 Output Pins

Output pins send signals outside the design.

- ———▶ Y
- ———▶ OUT

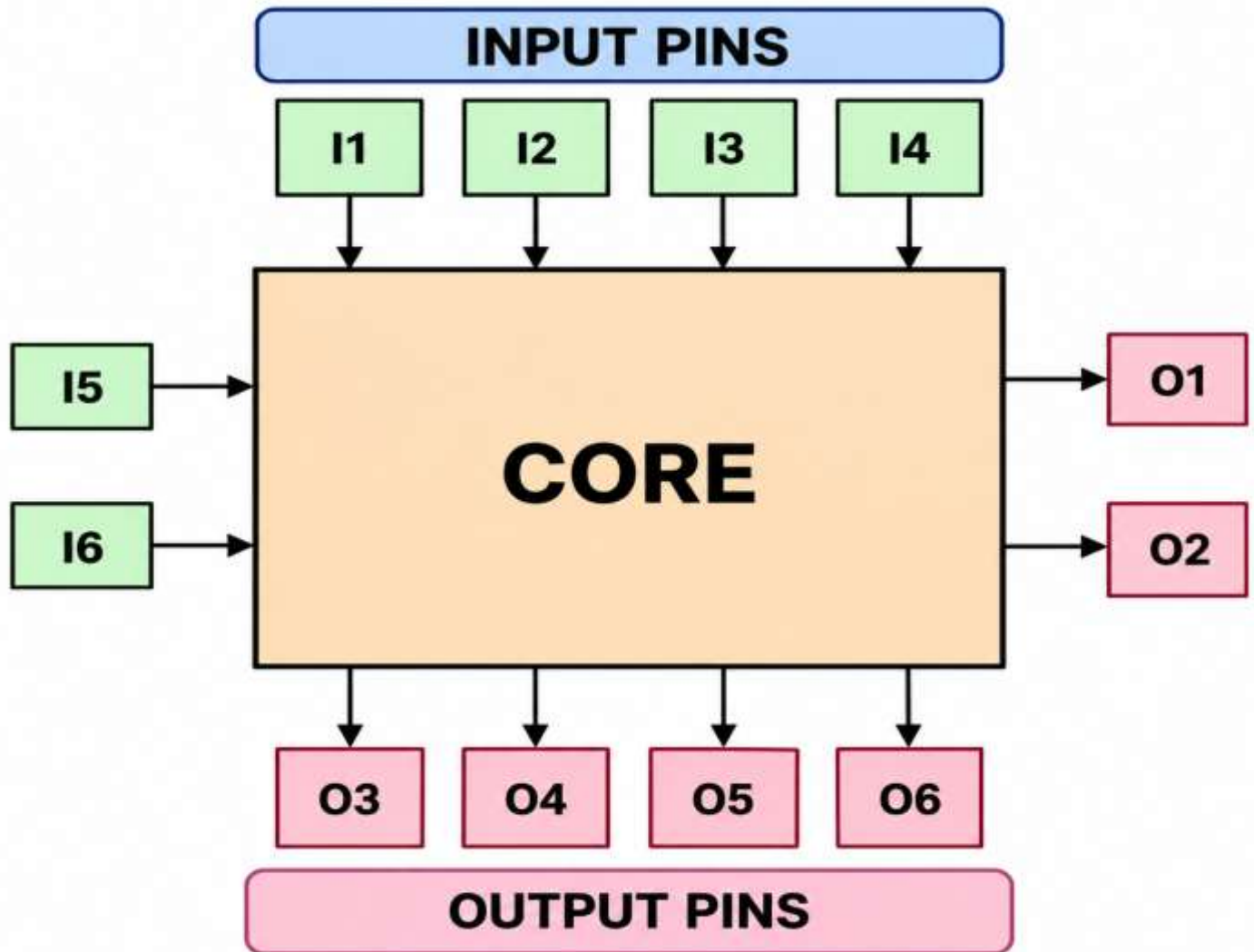
### 6.3 Bidirectional Pins

Bidirectional pins can act as both inputs and outputs.

- ◀——▶ DATA

## 7. PIN PLACEMENT CONCEPT

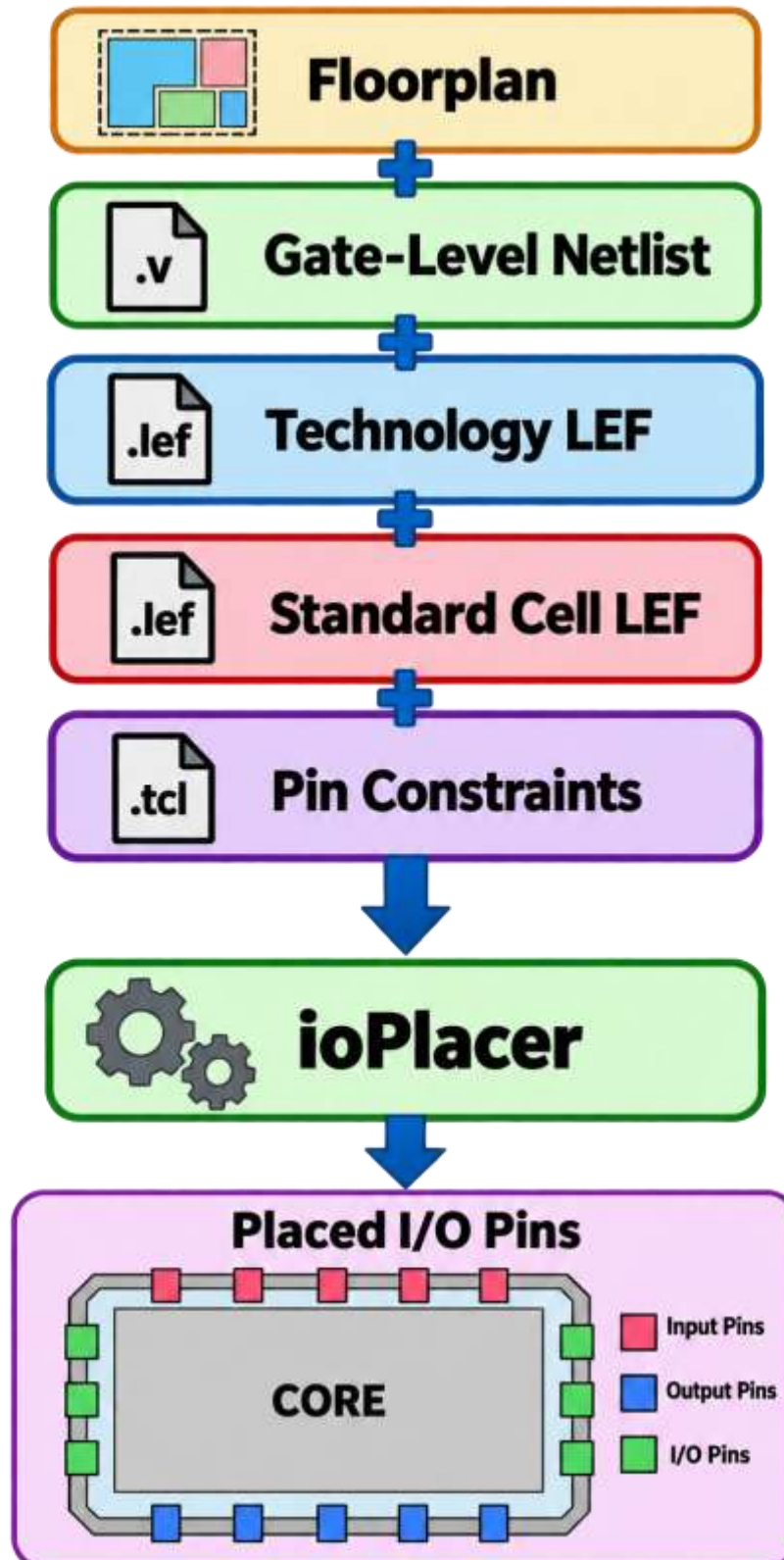
A typical pin placement can be represented as:



The actual pin locations depend on the constraints and design connectivity.

## 8. INPUTS REQUIRED

The experiment requires:



## 9. PROCEDURE

### Step 1: Create Working Directory

Open the Linux terminal:

- `mkdir -p ~/openroad_exp6`

Move into the directory:

- `cd ~/openroad_exp6`

Check the current directory:

- `pwd`

## 10. STEP 2: START OPENROAD

Run:

- `openroad`

Check the version:

- `version`

Check the pin-placement commands:

- `help place_pins`

You can also check:

- `help set_io_pin_constraint`

## 11. STEP 3: LOAD THE FLOORPLAN DATABASE

Load the database generated from the previous experiment:

- `read_db pdn.oddb`

If the PDN database is not available, load the required design files:

- `read_lef <technology.lef>`
- `read_lef <standard_cell.lef>`
- `read_liberty <standard_cell.lib>`
- `read_verilog <synthesized_netlist.v>`
- `link_design <TOP_MODULE>`

Then initialize/read the floorplan as required.

## 12. STEP 4: CHECK THE DESIGN

Run:

- `report_design_area`

Check the design placement:

- `check_placement`

Make sure the floorplan is valid before placing I/O pins.

### 13. STEP 5: IDENTIFY DESIGN PORTS

The design may contain ports such as:

Inputs:

- clk
- rst
- a
- b

Outputs:

- y

The actual ports depend on the selected RTL/netlist.

You can inspect the design through the OpenROAD GUI or use the available database/reporting commands.

### 14. STEP 6: DEFINE I/O PIN CONSTRAINTS

I/O pin constraints determine where pins should be placed.

For example,

input pins can be constrained to the left side:

**tcl:**

```
set_io_pin_constraint \
-direction input \
-region left:*
```

Output pins can be constrained to the right side:

**Tcl:**

```
set_io_pin_constraint \
-direction output \
-region right:*
```

This gives a conceptual arrangement:

#### INPUTS

#### OUTPUTS

|      |      |      |
|------|------|------|
| I1 → |      | ← O1 |
| I2 → | CORE | ← O2 |
| I3 → |      | ← O3 |
| I4 → |      | ← O4 |

## 15. STEP 7: CONSTRAIN SPECIFIC PINS

Individual pins can also be constrained.

For **example:Tcl:**

```
set_io_pin_constraint \  
-pin_names {clk rst} \  
-region top:*
```

This places the specified pins in the top region according to the pin-placement constraints.

Another **example:Tcl:**

```
set_io_pin_constraint \  
-pin_names {y} \  
-region bottom:*
```

This constrains output y to the bottom region.

**Note:** Exact constraint syntax and supported options depend on the OpenROAD version being used. Use help set\_io\_pin\_constraint to verify the syntax in your installation.

## 16. STEP 8: SPECIFY PIN LAYERS

The pin placer can be instructed to use particular horizontal and vertical metal layers.

**Example:**

```
place_pins \  
-hor_layers M3 \  
-ver_layers M2
```

Conceptually:

- Horizontal pin access → M3
- Vertical pin access → M2

The actual metal layers must be selected according to the technology/PDK.

## 17. STEP 9: PERFORM PIN PLACEMENT

Run: Tcl

```
place_pins \  
-hor_layers M3 \  
-ver_layers M2
```

The pin placer now determines legal locations for the design I/O pins.

The placement considers the defined constraints and available routing tracks.

### 18. STEP 10: INSPECT PIN PLACEMENT

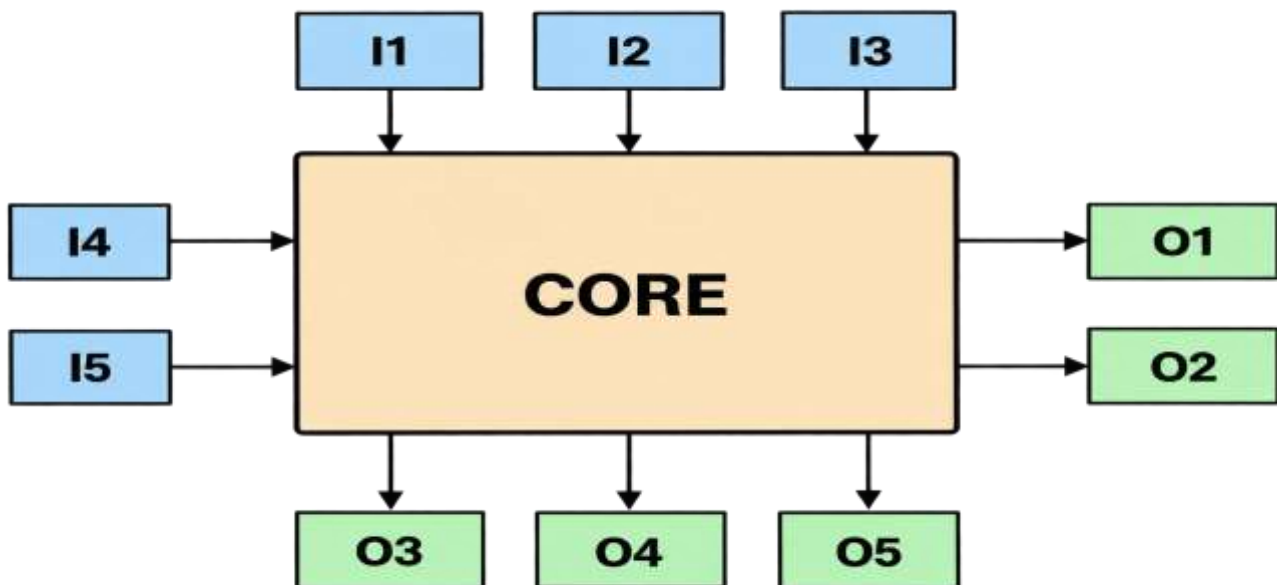
Open the OpenROAD GUI:

- `openroad -gui`

Inspect:

- Input pins
- Output pins
- Pin layers
- Pin spacing
- Pin order
- Core boundary
- Routing tracks

Better distributed placement:



### 19. STEP 11: SAVE PIN PLACEMENT

Pin placement can be saved for reuse.

Example:

- `write_pin_placement pin_placement.tcl`

Check the generated file:

- `ls -lh`

Expected:

- pin\_placement.tcl

## **20. STEP 12: WRITE THE UPDATED DESIGN**

Save the updated physical design:

- write\_def io\_placed.def

Save the OpenDB database:

- write\_db io\_placed.odb

Check:

- ls -lh

Expected files:

- io\_placed.def
- io\_placed.odb
- pin\_placement.tcl

## **21. COMPLETE COMMAND SEQUENCE**

A representative command sequence is:

read\_db pdn.odb

report\_design\_area

check\_placement

set\_io\_pin\_constraint \  
-direction input \  
-region left:\*

set\_io\_pin\_constraint \  
-direction output \  
-region right:\*

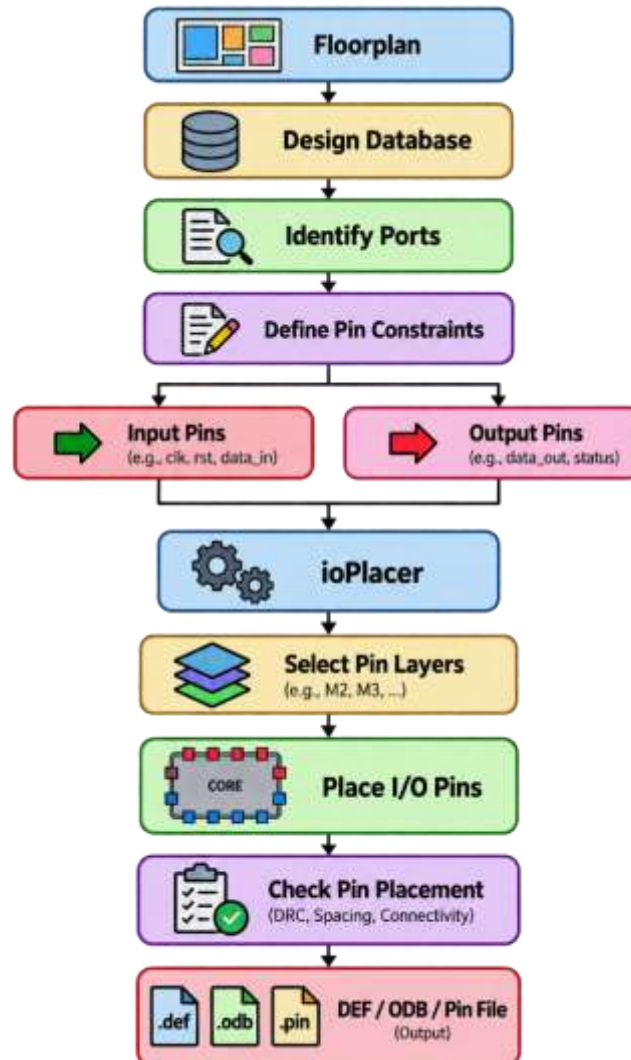
place\_pins \  
-hor\_layers M3 \  
-ver\_layers M2

write\_pin\_placement pin\_placement.tcl

write\_def io\_placed.def

write\_db io\_placed.od

## 22. I/O PIN PLACEMENT FLOW



## 23. OBSERVATION TABLE

| Parameter             | Observation |
|-----------------------|-------------|
| OpenROAD Version      |             |
| Design Name           |             |
| Top Module            |             |
| Number of Input Pins  |             |
| Number of Output Pins |             |

| Parameter                    | Observation |
|------------------------------|-------------|
| Number of Bidirectional Pins |             |
| Input Pin Region             |             |
| Output Pin Region            |             |
| Horizontal Pin Layer         |             |
| Vertical Pin Layer           |             |
| Pin Spacing                  |             |
| Pin Placement Status         |             |
| Pin Placement File           |             |
| DEF File                     |             |
| ODB File                     |             |

## 24. ANALYSIS

## 25. EXPECTED OUTPUT

## 28. RESULT

## 31. CONCLUSION

### EXPERIMENT-7

#### Title

Place an I/O Ring Around the Boundary of a Chip and Connect with Either Wirebond Pads or a Bump Array

**Contact Hours: 2 Hours**

#### 1. AIM

To understand and perform I/O ring placement around the chip boundary and study the connection of I/O terminals using wirebond pads or a bump array in the OpenROAD physical-design flow.

#### 2. OBJECTIVES

After completing this experiment, the student will be able to:

1. Understand the concept of an I/O ring.
2. Understand the role of I/O pads in chip-level physical design.
3. Understand wirebond and flip-chip packaging.
4. Place I/O pads around the chip boundary.
5. Understand the purpose of power and ground pads.
6. Understand bump-array placement.
7. Connect I/O terminals to the corresponding pads or bumps.
8. Inspect the completed I/O structure using the OpenROAD environment.
9. Save the final I/O-ring design in DEF/OpenDB format.

#### 3. SOFTWARE / TOOLS REQUIRED

| S.No. | Software / Tool                | Purpose                      |
|-------|--------------------------------|------------------------------|
| 1     | Ubuntu/Linux                   | Operating environment        |
| 2     | OpenROAD                       | Physical design              |
| 3     | OpenDB                         | Design database              |
| 4     | ICeWall / Chip-level I/O tools | I/O pad-ring implementation  |
| 5     | LEF                            | Physical library information |
| 6     | Liberty                        | Timing/library information   |
| 7     | DEF                            | Physical design information  |
| 8     | Verilog                        | Gate-level netlist           |
| 9     | GVim                           | Tcl/configuration editing    |

## 4. THEORY

### 4.1 Introduction to I/O Ring

An I/O ring, also called a pad ring, is a collection of I/O cells or pads placed around the outer boundary of a chip.

It provides the interface between:

External Package



I/O Pads / Bumps

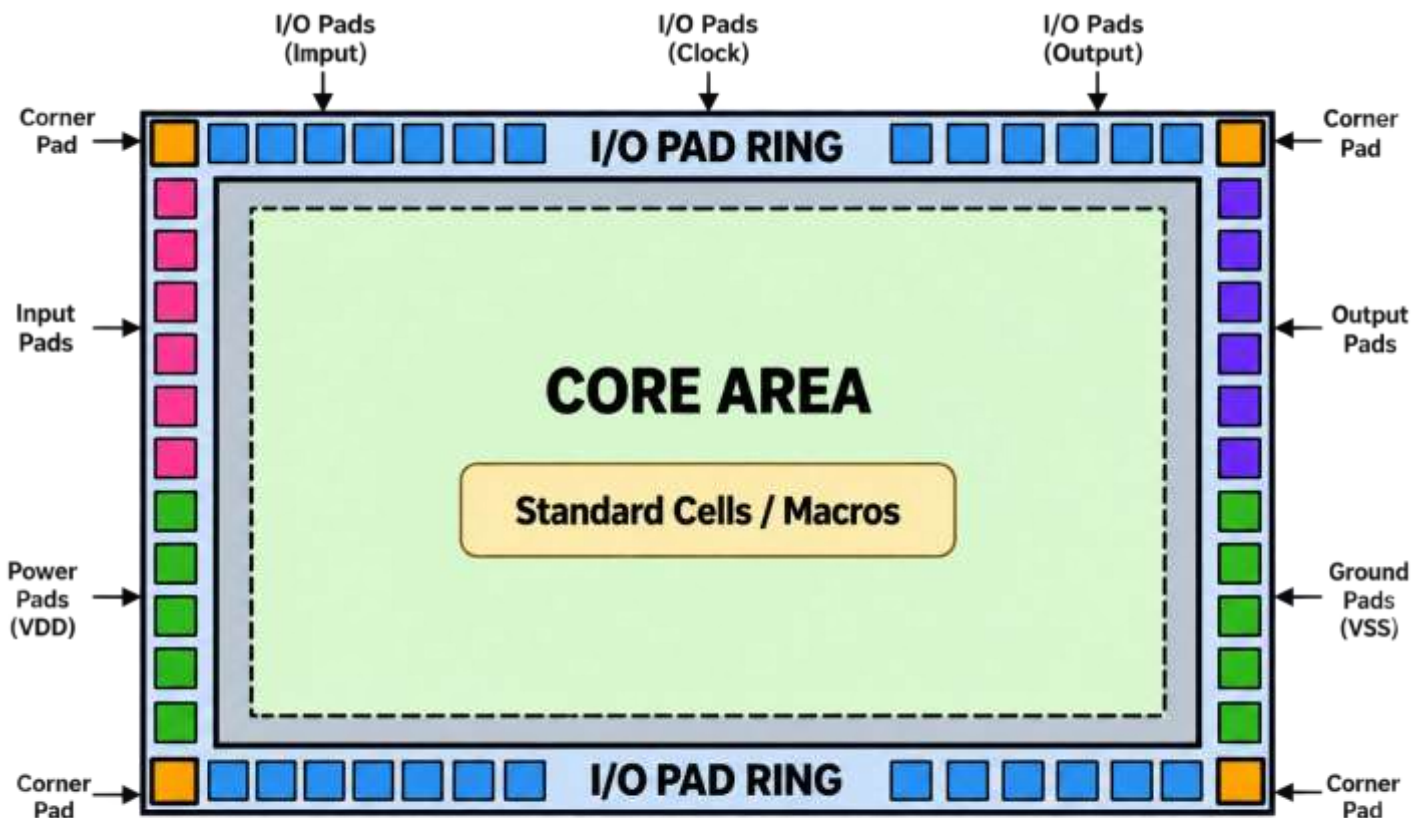


Core Interconnect



Internal Logic

A simplified chip-level structure is:



## 5. PURPOSE OF AN I/O RING

The I/O ring provides:

- External signal connections
- Power connections
- Ground connections
- ESD protection
- Physical separation between core and package
- Connection to package pins or bumps

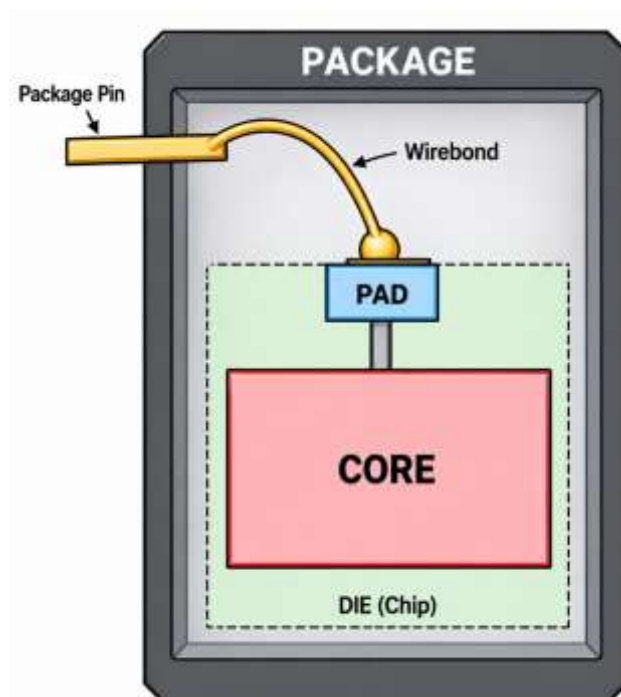
Typical I/O cells include:

- Input Pad
- Output Pad
- Bidirectional Pad
- Power Pad
- Ground Pad
- Corner Cell
- ESD Cell

## 6. WIREBOND PACKAGING

In **wirebond packaging**, the chip pads are connected to package leads using thin bonding wires.

Conceptually:

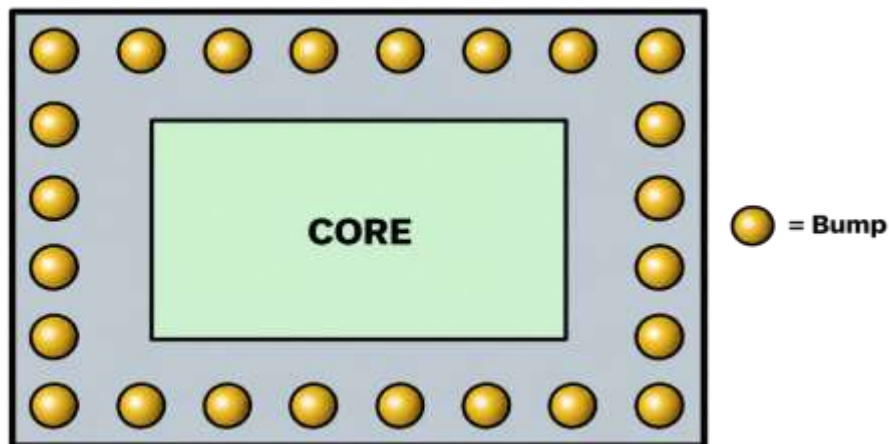


## 7. BUMP ARRAY

A **bump array** is commonly used in flip-chip packaging.

Instead of connecting only around the perimeter, solder bumps can be distributed over a large portion of the chip/package interface.

Conceptually:



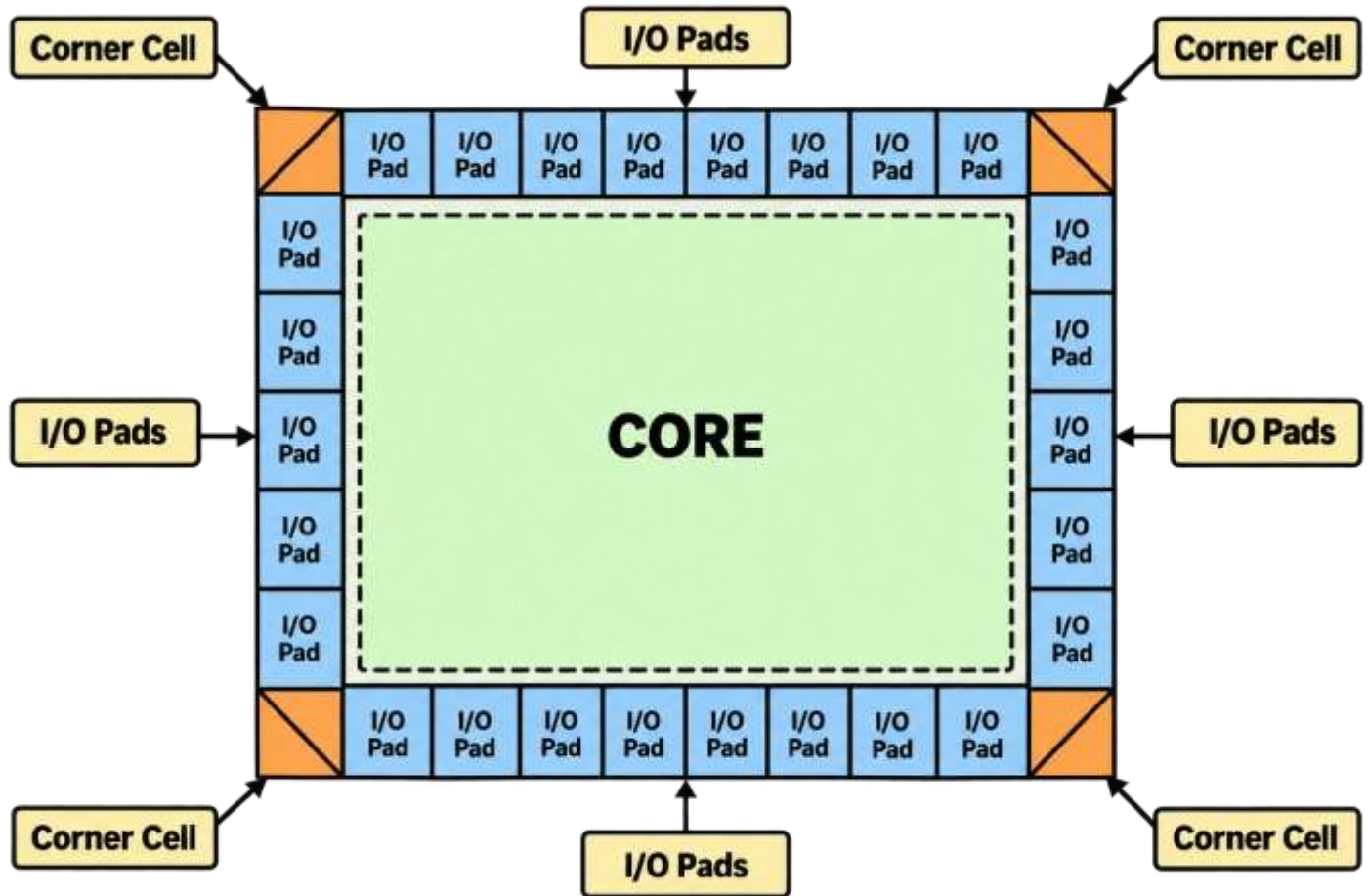
A bump array can provide many signal, power and ground connections.

## 8. WIREBOND VS BUMP ARRAY

| Parameter          | Wirebond                 | Bump Array                        |
|--------------------|--------------------------|-----------------------------------|
| Connection         | Bonding wire             | Solder bump                       |
| Location           | Usually chip perimeter   | Can cover larger area             |
| Connection density | Lower                    | Higher                            |
| Package type       | Wirebond package         | Flip-chip/package                 |
| Signal path        | Longer                   | Generally shorter                 |
| Power distribution | More perimeter dependent | Can be distributed across chip    |
| Typical use        | Conventional packaging   | High-performance/high-I/O designs |

## 9. I/O RING STRUCTURE

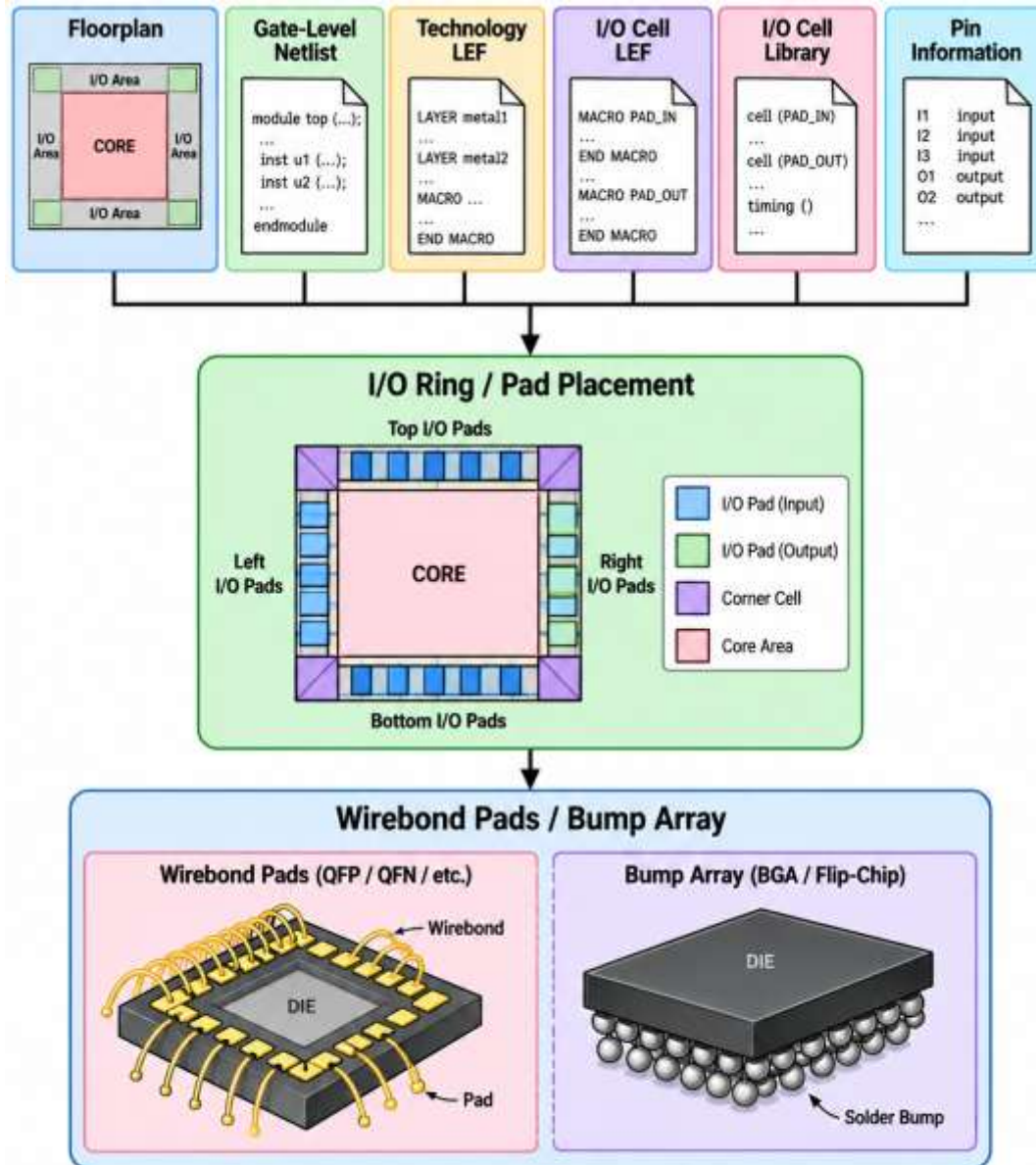
A typical I/O ring contains:



Power and ground pads are distributed around the ring to provide adequate supply connectivity.

## 10. INPUTS REQUIRED

The experiment requires:



The design from **Experiment-6** can be used as the starting point.

## 11. PROCEDURE

### Step 1: Create Working Directory

Open the Linux terminal:

- `mkdir -p ~/openroad_exp7`

Move into the directory:

- `cd ~/openroad_exp7`

Check the directory:

- pwd

## **12. STEP 2: START OPENROAD**

Run:

- openroad

Check the version:

version

Check available I/O-related commands:

help

## **13. STEP 3: LOAD THE PREVIOUS DESIGN**

Load the database generated after I/O pin placement:

- read\_db io\_placed.odb

If the database is unavailable, load the required design files:

- read\_lef <technology.lef>
- read\_lef <standard\_cell.lef>
- read\_lef <io\_cell.lef>
  
- read\_liberty <standard\_cell.lib>
- read\_verilog <synthesized\_netlist.v>
  
- link\_design <TOP\_MODULE>

## **14. STEP 4: CHECK THE FLOORPLAN**

Run:

- report\_design\_area

Check placement:

- check\_placement

The die and core boundaries should already be available from the previous floorplanning stage.

### 15. STEP 5: LOAD I/O PAD INFORMATION

I/O pad cells must be available in the selected technology library.

Typical I/O cells may include:

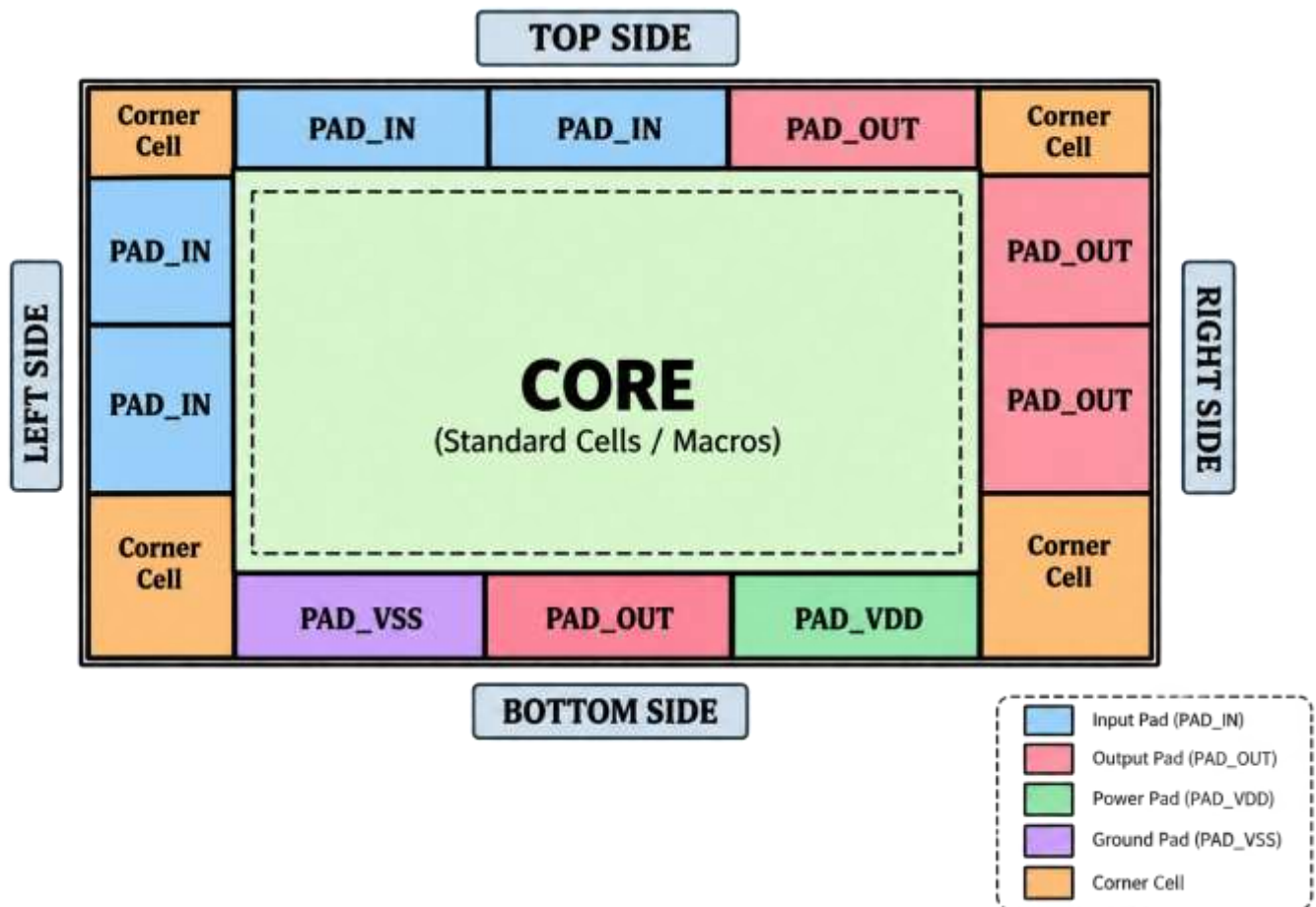
- PAD\_IN
- PAD\_OUT
- PAD\_BIDIR
- PAD\_VDD
- PAD\_VSS
- PAD\_CORNER

The actual cell names depend on the selected PDK.

Check the available library cells using the library/technology information supplied with the PDK.

### 16. STEP 6: UNDERSTAND PAD ASSIGNMENT

Create a pad assignment conceptually as:



The actual assignment is technology and package dependent.

### 17. STEP 7: PLACE I/O TERMINALS

OpenROAD provides chip-level I/O functionality through its pad/chip-level flow.

A representative operation for placing terminals is:

- `place_io_terminals <IO_INSTANCES>`

For example:

- `place_io_terminals u_in*`

or:

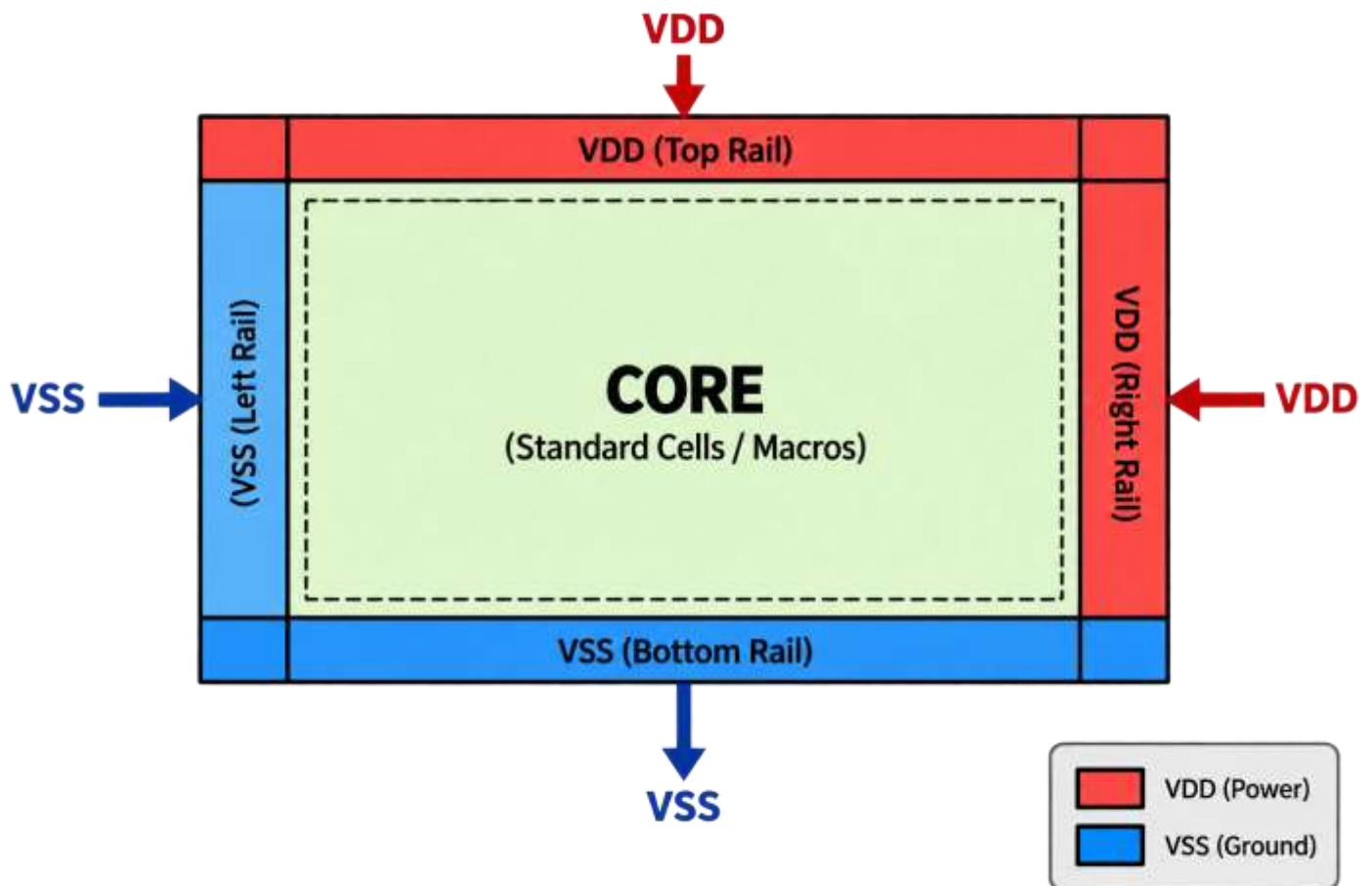
- `place_io_terminals u_out*`

The exact instance patterns depend on the design.

### 18. STEP 8: PLACE POWER AND GROUND PADS

Power and ground pads must be distributed appropriately.

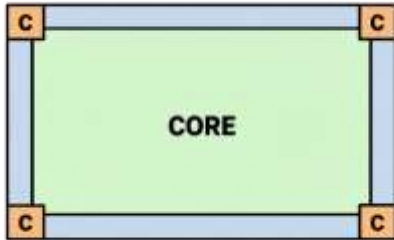
Conceptually:



Power and ground pads should be connected to the corresponding power network.

## 19. STEP 9: CORNER CELLS

Corner cells are used at the four corners of the I/O ring.



Where:

C = Corner Cell

Corner cells help complete the physical continuity of the pad ring and may also provide required ESD/powering structures depending on the technology.

## 20. STEP 10: CREATE A BUMP ARRAY

For a flip-chip style implementation, a bump array can be generated using the chip-level I/O flow.

A representative command is: **Tcl:**

```

make_io_bump_array \
-bump <BUMP_MASTER> \
-origin {200 200} \
-rows 14 \
-columns 14 \
-pitch {200 200}
  
```

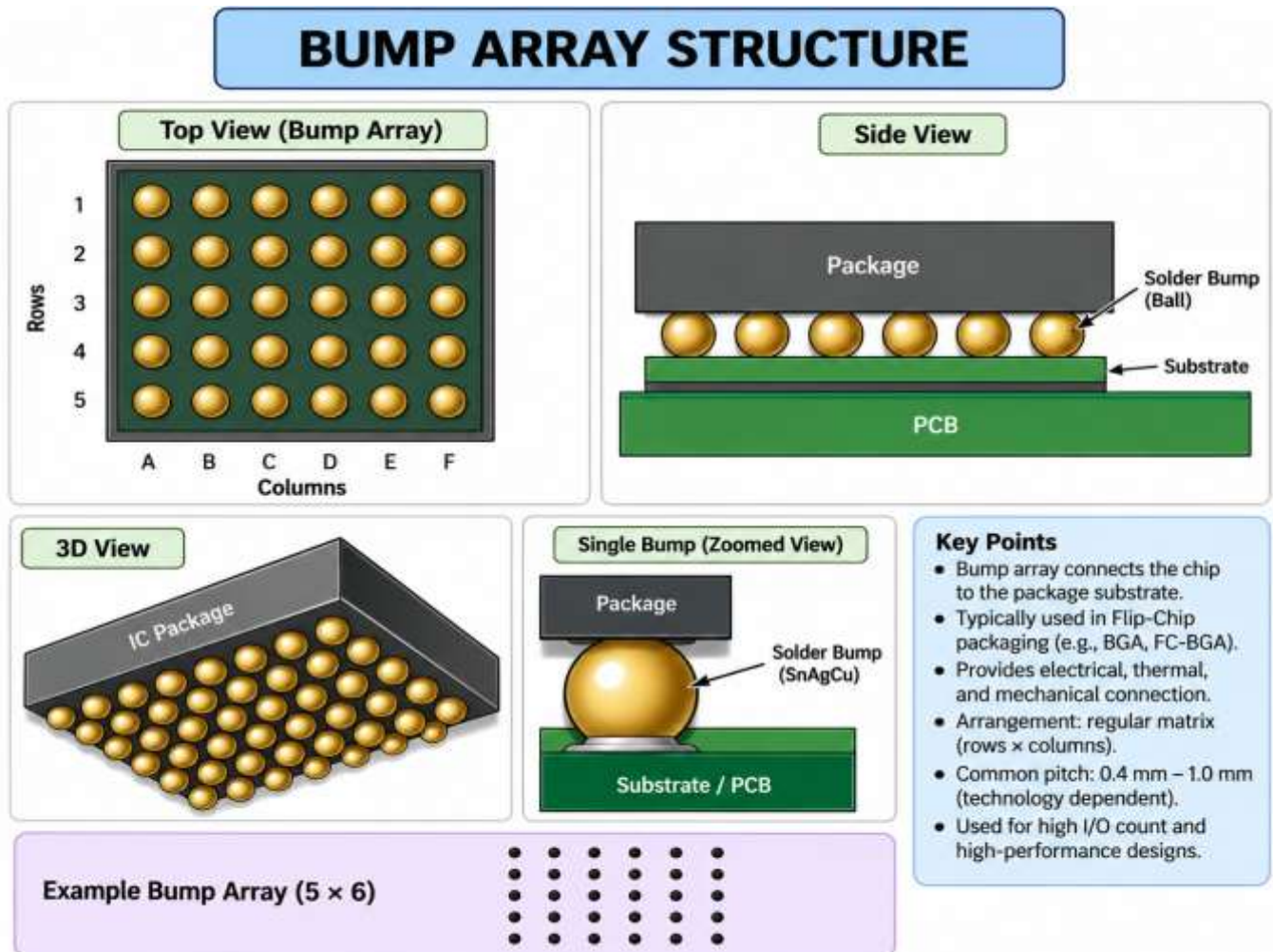
Here:

| Parameter | Meaning                |
|-----------|------------------------|
| -bump     | Bump master cell       |
| -origin   | Starting coordinate    |
| -rows     | Number of bump rows    |
| -columns  | Number of bump columns |
| -pitch    | Distance between bumps |

**Important:** The bump master, coordinates, rows, columns and pitch are only examples. They must be replaced with values supported by your PDK/package technology.

## 21. BUMP ARRAY STRUCTURE

Example:



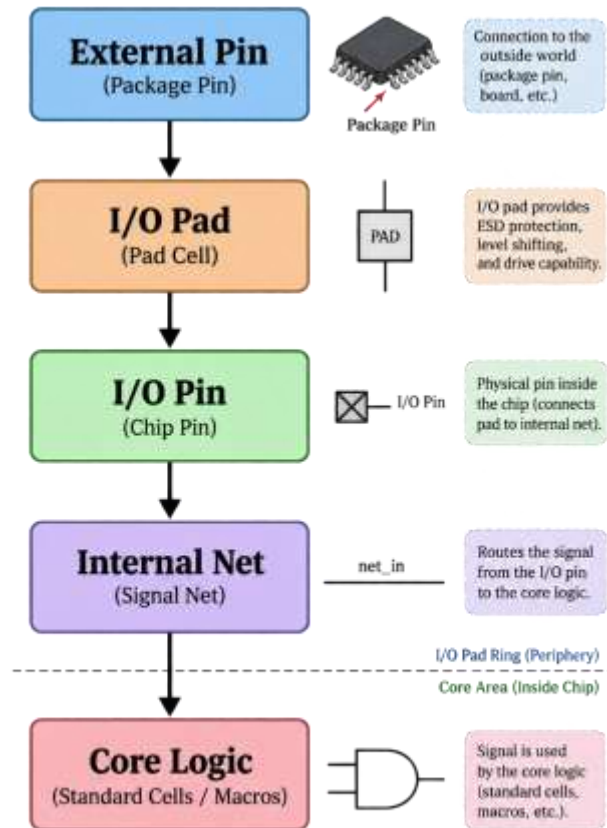
The bumps can be assigned for:

- Signal
- VDD
- VSS
- Clock
- Reset
- Other power domains

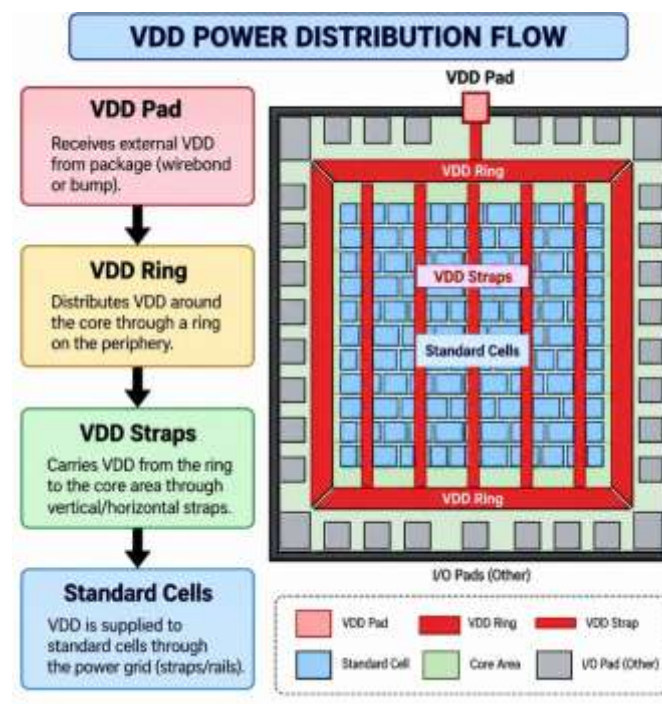
## 22. STEP 11: CONNECT I/O PADS/BUMPS

The pads or bumps must be electrically connected to the corresponding design nets.

Conceptually:



For power:



### **23. STEP 12: INSPECT USING OPENROAD GUI**

Launch the GUI:

- `openroad -gui`

Inspect:

- I/O pads
- Corner cells
- Core boundary
- Die boundary
- Power pads
- Ground pads
- Signal pads
- Bump array
- Connections

### **24. STEP 13: CHECK I/O PLACEMENT**

Check that:

1. Pads do not overlap.
2. Corner cells are correctly positioned.
3. Signal pads are connected to the correct ports.
4. Power pads connect to VDD.
5. Ground pads connect to VSS.
6. The I/O ring surrounds the required core boundary.
7. Bumps are placed at legal locations.
8. Sufficient spacing is maintained between pads/bumps.

### **25. STEP 14: SAVE THE I/O-RING DESIGN**

Save the updated database:

- `write_db io_ring.odb`

Write the DEF:

- `write_def io_ring.def`

Check the files:

- `ls -lh`

Expected files:

- `io_ring.odb`
- `io_ring.def`

## 26. COMPLETE REPRESENTATIVE COMMAND SEQUENCE

```
read_db io_placed.odb
```

```
report_design_area
```

```
check_placement
```

```
place_io_terminals u_*
```

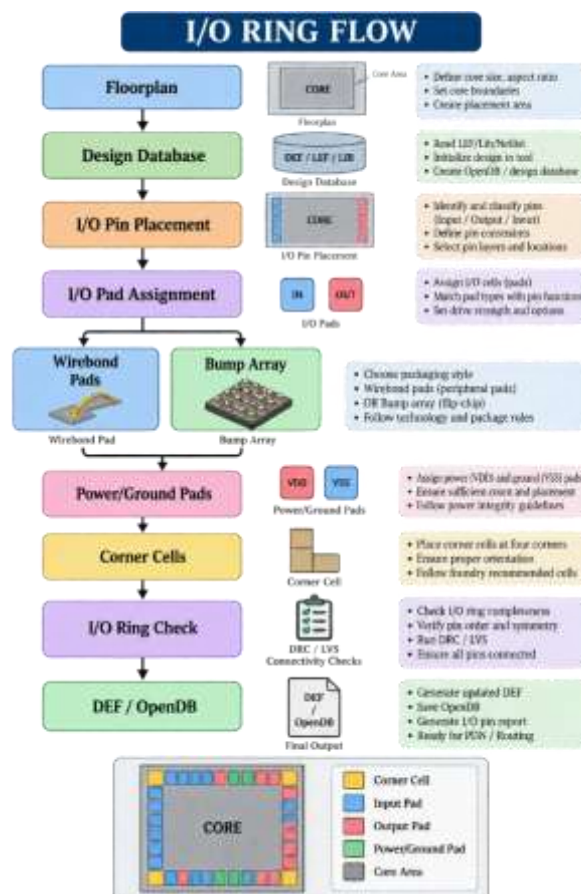
```
make_io_bump_array \
-bump <BUMP_MASTER> \
-origin {200 200} \
-rows 14 \
-columns 14 \
-pitch {200 200}
```

```
write_db io_ring.odb
```

```
write_def io_ring.def
```

The exact I/O-ring and pad/bump commands depend on the OpenROAD version and the PDK's I/O-cell definitions. Always verify the available syntax using help in your installed OpenROAD build.

## 27. I/O RING FLOW



## 28. OBSERVATION TABLE

| Parameter                    | Observation |
|------------------------------|-------------|
| OpenROAD Version             |             |
| Design Name                  |             |
| Top Module                   |             |
| Number of Input Pads         |             |
| Number of Output Pads        |             |
| Number of Bidirectional Pads |             |
| Number of Power Pads         |             |
| Number of Ground Pads        |             |
| Number of Corner Cells       |             |
| I/O Ring Type                |             |
| Bump Master                  |             |
| Number of Bump Rows          |             |
| Number of Bump Columns       |             |
| Bump Pitch                   |             |
| Pad Placement Status         |             |
| Bump Placement Status        |             |
| Final DEF                    |             |
| Final ODB                    |             |

## 29. ANALYSIS

### **30. EXPECTED OUTPUT**

### **31. RESULT**

### **34. CONCLUSION**

### EXPERIMENT-8

Explore OpenSTA as a Gate-Level Static Timing Verifier and as a Stand-Alone Executable to Verify the Timing of a Design Using Standard File Formats

**Contact Hours:** 2 Hours

#### 1. AIM

To explore **OpenSTA (Open Static Timing Analyzer)** as a gate-level static timing analysis tool and as a stand-alone executable, and to verify the timing characteristics of a synthesized design using standard timing-related file formats such as **Verilog, Liberty, SDC, SPEF, and SDF**.

#### 2. OBJECTIVES

- After completing this experiment, the student should be able to:
- Understand the concept of **Static Timing Analysis (STA)**.
- Understand the role of **OpenSTA** in the ASIC physical design flow.
- Run OpenSTA as a **stand-alone executable**.
- Load a gate-level Verilog netlist into OpenSTA.
- Read standard-cell timing information from a **Liberty (.lib) file**.
- Read timing constraints from an **SDC (.sdc) file**.
- Create and analyze timing paths.
- Understand:
  - Setup timing
  - Hold timing
  - Slack
  - Arrival time
  - Required time
  - Clock period
  - Clock skew
- Generate timing reports using OpenSTA.
- Understand how parasitic information such as **SPEF** affects timing analysis.

#### 3. SOFTWARE / TOOLS REQUIRED

| S.No. | Software / File    | Purpose                          |
|-------|--------------------|----------------------------------|
| 1     | Linux / Ubuntu     | Operating environment            |
| 2     | OpenSTA            | Static timing analysis           |
| 3     | OpenROAD           | Physical design flow integration |
| 4     | Yosys              | RTL synthesis                    |
| 5     | GVim / Vim         | Editing Verilog and SDC files    |
| 6     | Gate-level Verilog | Design netlist                   |

| S.No. | Software / File | Purpose                           |
|-------|-----------------|-----------------------------------|
| 7     | Liberty .lib    | Cell delay and timing information |
| 8     | SDC .sdc        | Timing constraints                |
| 9     | SPEF .spef      | Parasitic information             |
| 10    | SDF .sdf        | Delay annotation information      |

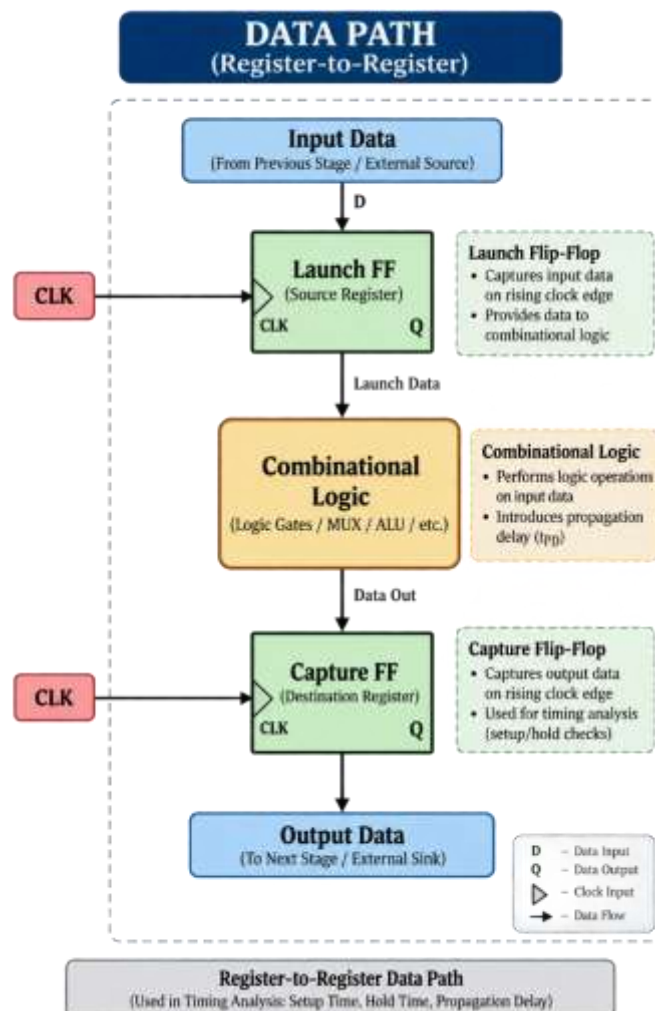
## 4. THEORY

### 4.1 What is Static Timing Analysis?

**Static Timing Analysis (STA)** is a method used to verify whether a digital design can operate correctly at the required clock frequency without applying simulation vectors.

Instead of simulating every possible input combination, STA mathematically analyzes timing paths between sequential and combinational elements.

A simplified timing path is:



## 5. WHAT IS OpenSTA?

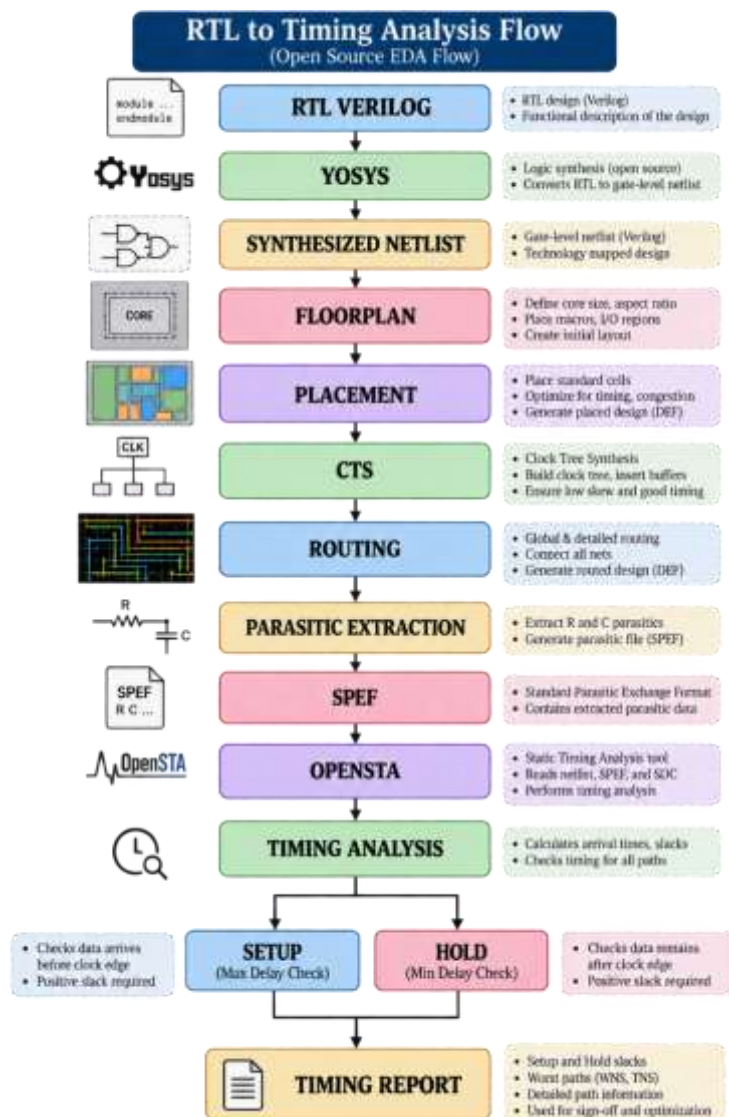
**OpenSTA** is an open-source **static timing analysis engine** used to verify the timing of gate-level digital designs. It can analyze timing using standard industry file formats including:

- **Verilog** – gate-level connectivity
- **Liberty (.lib)** – standard-cell timing and power models
- **SDC (.sdc)** – timing constraints
- **SPEF (.spef)** – extracted parasitic resistance and capacitance
- **SDF (.sdf)** – delay annotation
- **VCD / SAIF** – switching activity information for power-related analysis

OpenSTA can be used independently or as part of the OpenROAD physical-design flow.

## 6. ROLE OF OPENSTA IN ASIC FLOW

A simplified ASIC flow is:



OpenSTA can therefore be used at different stages to determine whether timing requirements are satisfied.

## 7. IMPORTANT STA CONCEPTS

### 7.1 Clock

A clock controls the operation of sequential circuits.

For example:

create\_clock -name clk -period 10 [get\_ports clk]

Here:

Clock period = 10 ns

Frequency =  $1 / 10 \text{ ns}$

= 100 MHz

### 7.2 Clock Frequency

The relationship between clock period and frequency is:

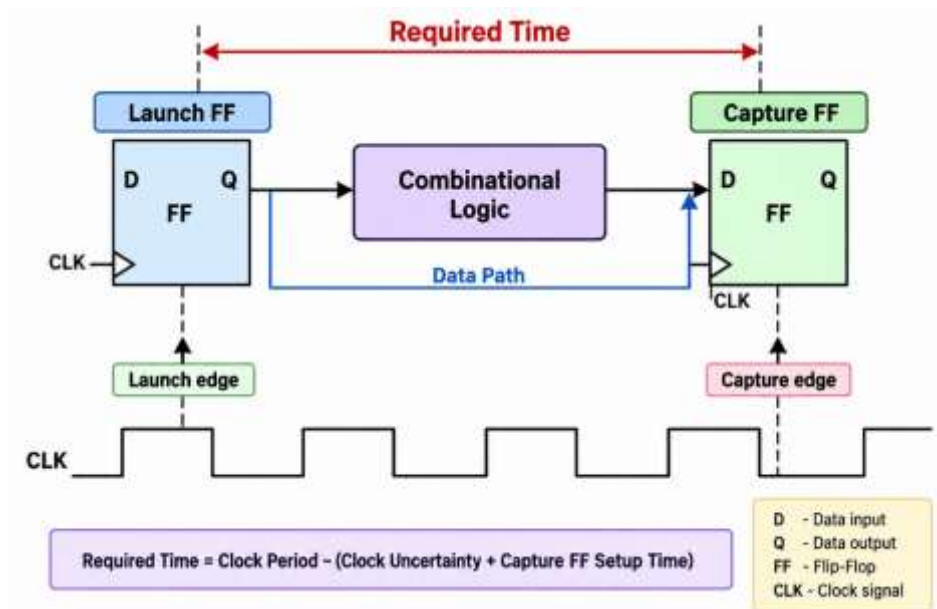
$$F = \frac{1}{T}$$

For a 10 ns clock:

$$F = \frac{1}{10\text{ns}} = 100\text{MHz}$$

## 8. SETUP TIMING

Setup timing verifies that data reaches the capture flip-flop sufficiently **before the active clock edge**.  
Simplified representation:



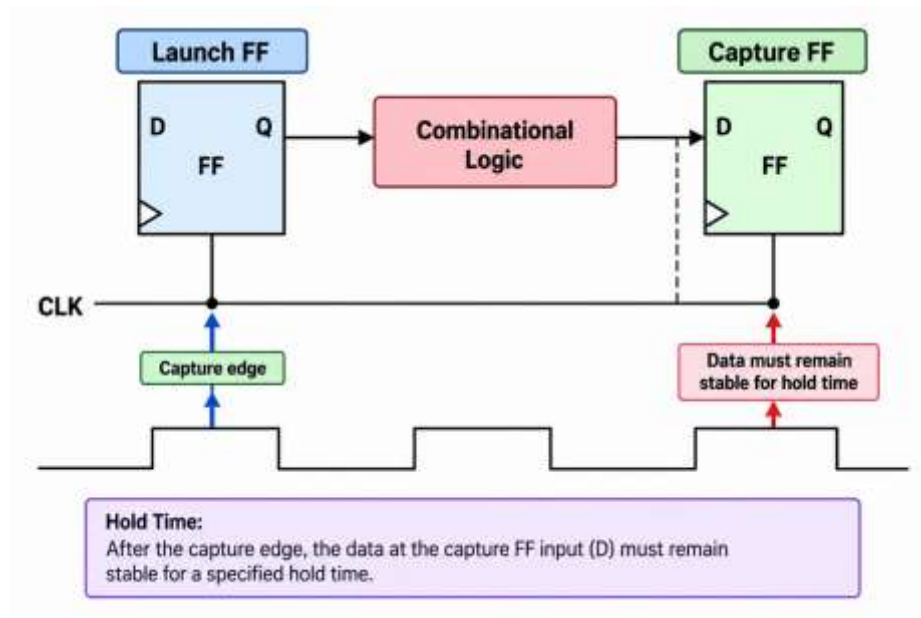
A simplified setup condition is:

$$T_{clk} \geq T_{cq} + T_{logic} + T_{setup}$$

If the data arrives too late, a **setup violation** occurs.

## 9. HOLD TIMING

Hold timing verifies that data does not change too quickly after the capture clock edge.



A hold violation occurs when the new data reaches the capture element too early.

## 10. SLACK

Slack indicates whether a timing path satisfies its timing requirement.

### Setup Slack

$$\text{Slack} = \text{Required Time} - \text{Arrival Time}$$

Example:

Required time = 10 ns

Arrival time = 8 ns

Slack = 10 - 8

= +2 ns

Therefore, the path passes setup timing.

### Negative Slack

Required time = 10 ns

Arrival time = 11 ns

Slack = 10 - 11

= -1 ns

The path has a **1 ns setup violation**.

## 11. WNS AND TNS

### WNS – Worst Negative Slack

WNS represents the worst timing violation in the design.

Example:

Path 1 = +0.50 ns

Path 2 = -0.20 ns

Path 3 = -0.80 ns

WNS = -0.80 ns

### **TNS – Total Negative Slack**

TNS represents the sum of negative slack values for violating paths.

For example:

Slack 1 = -0.2 ns

Slack 2 = -0.5 ns

Slack 3 = -0.3 ns

TNS = -1.0 ns

For a timing-clean design:

WNS  $\geq 0$

TNS = 0

## **12. INPUT FILES FOR OPENSTA**

### **12.1 Gate-Level Verilog**

The Verilog netlist describes the connectivity of standard cells.

Example:

```
module top (  
    input clk,  
    input a,  
    input b,  
    output y  
);
```

```
wire n1;
```

```
AND2_X1 U1 (  
    .A(a),  
    .B(b),  
    .Y(n1)  
);
```

```
DFF_X1 U2 (  
    .D(n1),  
    .CK(clk),  
    .Q(y)  
);
```

```
endmodule
```

## 12.2 Liberty File

The Liberty file contains information such as:

- Cell area
- Input capacitance
- Output capacitance
- Cell delay
- Transition
- Timing arcs
- Setup time
- Hold time
- Leakage power
- Dynamic power

Example:

```
cell (AND2_X1) {  
    area : 1.0;  
  
    pin(A) {  
        direction : input;  
    }  
  
    pin(Y) {  
        direction : output;  
    }  
}
```

Actual Liberty files are much more detailed and are supplied by the technology library/PDK.

## 13. SDC FILE

**SDC = Synopsys Design Constraints**

SDC describes the timing requirements of the design.

Typical constraints include:

- create\_clock
- set\_input\_delay
- set\_output\_delay
- set\_clock\_uncertainty
- set\_clock\_latency
- set\_false\_path
- set\_multicycle\_path
- set\_max\_delay
- set\_min\_delay

#### Example:

- `create_clock -name clk -period 10 [get_ports clk]`

Input delay:

- `set_input_delay 2.0 -clock clk [get_ports a]`

Output delay:

- `set_output_delay 2.0 -clock clk [get_ports y]`

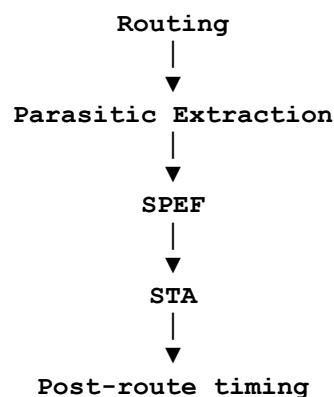
#### 14. SPEF FILE

**SPEF = Standard Parasitic Exchange Format**

SPEF contains extracted parasitic information such as:

- Resistance
- Capacitance
- Coupling capacitance

After routing:



Post-route timing is generally more realistic because routing parasitics affect the delay.

#### 15. PROCEDURE

##### Step 1 – Check OpenSTA Installation

Open a terminal:

- `opensta -version`

If the executable is available, the installed version is displayed.

Depending on the installation, the executable may also be invoked as:

- `sta`

Check:

- `which opensta`
- `which sta`

##### 16. Step 2 – Create an Experiment Directory

- `mkdir -p ~/opensta_exp8`
- `cd ~/opensta_exp8`

**17. Step 3 – Prepare the Design Files**

The basic STA setup requires:

opensta\_exp8/

```
|  
├─ synthesized_netlist.v  
├─ constraints.sdc  
└─ standard_cell.lib
```

For post-route analysis:

opensta\_exp8/

```
|  
├─ synthesized_netlist.v  
├─ constraints.sdc  
├─ standard_cell.lib  
└─ design.spef
```

**18. Step 4 – Create an SDC Constraint File**

Open the file:

gvim constraints.sdc

Example:

```
create_clock -name clk -period 10 [get_ports clk]
```

```
set_clock_uncertainty 0.2 [get_clocks clk]
```

```
set_input_delay 1.0 \  
-clock clk \  
[get_ports a]
```

```
set_input_delay 1.0 \  
-clock clk \  
[get_ports b]
```

```
set_output_delay 1.0 \  
-clock clk \  
[get_ports y]
```

Save the file.

**20. Step 6 – Read the Liberty File**

Inside OpenSTA:

- read\_liberty <standard\_cell.lib>

Example:

- read\_liberty NangateOpenCellLibrary\_typical.lib

The Liberty file provides the timing models for the standard cells.

### **21. Step 7 – Read the Gate-Level Netlist**

- `read_verilog synthesized_netlist.v`

### **22. Step 8 – Link the Design**

- `link_design <TOP_MODULE>`

For example:

- `link_design gcd`

The top-level module must correspond to the synthesized design.

### **23. Step 9 – Read the Timing Constraints**

- `read_sdc constraints.sdc`

Check the clocks:

- `report_clocks`

Example output concept:

Clock    Period

-----

clk      10.000

### **24. Step 10 – Check the Design**

Useful commands include:

- `check_setup`

and:

- `report_units`

The setup check helps identify missing or inconsistent timing information.

### **25. Step 11 – Generate Timing Report**

For maximum-delay/setup analysis:

- `report_checks -path_delay max`

For minimum-delay/hold analysis:

- `report_checks -path_delay min`

A more detailed report can be generated using:

- `report_checks -path_delay max -fields {slew cap input_pins nets fanout} -digits 4`

### **26. STEP 12 – Report Specific Timing Paths**

To report a limited number of worst paths:

- `report_checks -path_delay max -group_count 5`

For hold:

- `report_checks -path_delay min -group_count 5`

## 27. STEP 13 – Report Timing Paths

Another useful command is:

- `report_timing`

Depending on the OpenSTA version and configuration, detailed path reports can be generated using the available timing-report commands.

## 28. STEP 14 – Analyze Clock Information

Use:

- `report_clocks`

This helps verify:

- Clock name
- Clock period
- Clock waveform
- Clock source
- Clock uncertainty

## 29. STEP 15 – Analyze Timing Violations

A typical timing report contains information similar to:

- Startpoint: U1/Q
- Endpoint: U5/D
- Path Group: clk
- Path Type: max
  
- Arrival Time
- Required Time
- Slack

Interpretation:

- Positive Slack → Timing met
- Zero Slack → Timing exactly met
- Negative Slack → Timing violation

## 30. STEP 16 – Read SPEF for Post-Route STA

After parasitic extraction:

- `read_spef design.spef`

Then rerun:

- `report_checks -path_delay max`

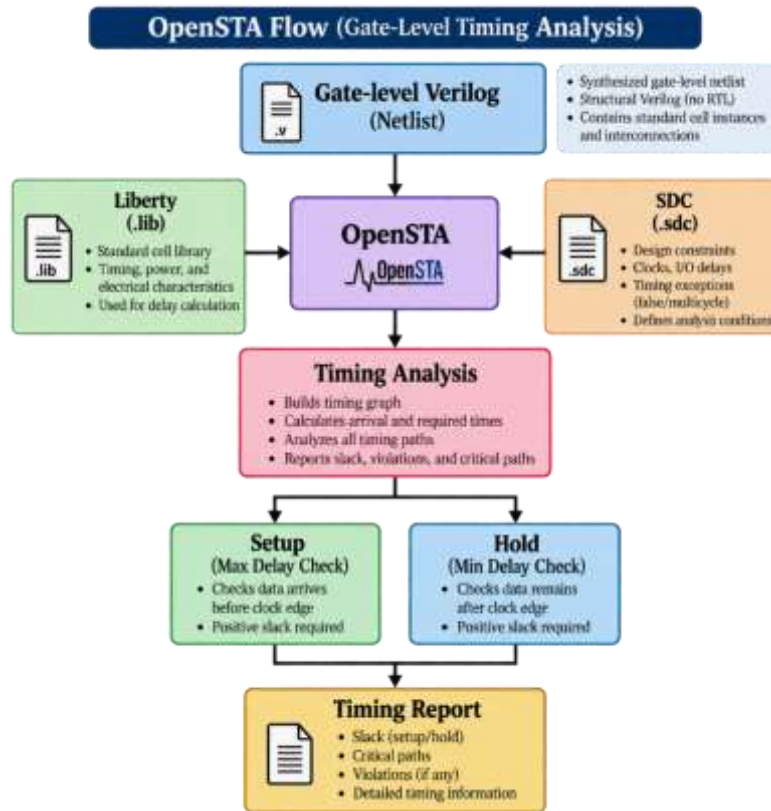
and:

- `report_checks -path_delay min`

Compare the timing before and after SPEF.

### 31. STAND-ALONE OPENSTA FLOW

OpenSTA can be used independently from OpenROAD



For post-route timing:



### 32. RUNNING OPENSTA WITH A TCL SCRIPT

Instead of entering commands one by one, create a script.

- `gvim run_sta.tcl`

Example:

- `read_liberty <standard_cell.lib>`
- `read_verilog synthesized_netlist.v`
- `link_design <TOP_MODULE>`
- `read_sdc constraints.sdc`
- `report_clocks`
- `check_setup`
- `report_checks -path_delay max`
- `report_checks -path_delay min`

Run:

- `opensta run_sta.tcl`

or:

- `sta run_sta.tcl`

depending on the installed executable.

### 33. SAVE THE TIMING REPORT

Timing output can be redirected from the Linux shell:

- `opensta run_sta.tcl > timing_report.txt`

Then inspect:

- `gvim timing_report.txt`

or:

- `cat timing_report.txt`

### 34. EXAMPLE TIMING ANALYSIS

Consider:

Clock Period = 10.0 ns

Clock Uncertainty = 0.2 ns

Data Arrival = 8.1 ns

Required Time = 9.8 ns

Therefore:

$$\begin{aligned} \text{Slack} &= 9.8 - 8.1 \\ \text{Slack} &= +1.7\text{ns} \end{aligned}$$

Hence:

Setup Timing → PASS

If:

Arrival Time = 10.3 ns

Required Time = 9.8 ns

then:

$$\begin{aligned} \text{Slack} &= 9.8 - 10.3 \\ \text{Slack} &= -0.5\text{ns} \end{aligned}$$

Therefore:

Setup Timing → FAIL

### 35. SETUP AND HOLD ANALYSIS

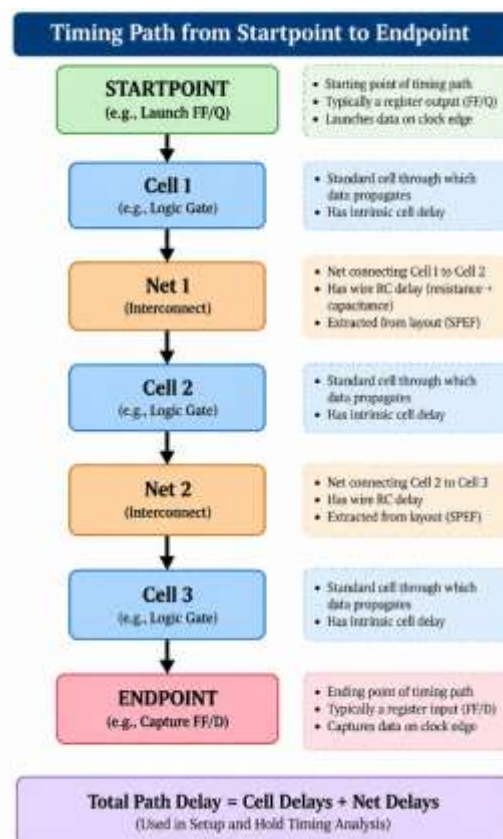
| Analysis        | OpenSTA Command               | Purpose                         |
|-----------------|-------------------------------|---------------------------------|
| Setup / Max     | report_checks -path_delay max | Checks maximum data delay       |
| Hold / Min      | report_checks -path_delay min | Checks minimum data delay       |
| Clock           | report_clocks                 | Reports clock information       |
| Setup checks    | check_setup                   | Checks timing setup             |
| Detailed timing | report_timing                 | Reports timing path information |

### 36. IMPORTANT TIMING TERMS

| Term              | Meaning                                                |
|-------------------|--------------------------------------------------------|
| Startpoint        | Beginning of a timing path                             |
| Endpoint          | End of a timing path                                   |
| Arrival Time      | Time at which data reaches endpoint                    |
| Required Time     | Latest/earliest allowed arrival depending on analysis  |
| Slack             | Difference between required and arrival time           |
| WNS               | Worst Negative Slack                                   |
| TNS               | Total Negative Slack                                   |
| Slew              | Transition time of a signal                            |
| Fanout            | Number of loads driven by a pin                        |
| Capacitance       | Load seen by a driver                                  |
| Clock Skew        | Difference between clock arrival times                 |
| Clock Uncertainty | Timing margin representing clock variation/uncertainty |

### 37. TIMING PATH ANALYSIS

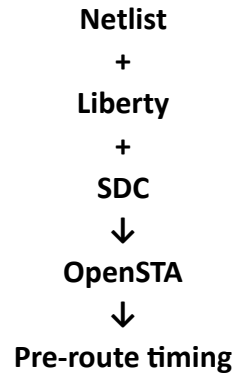
A typical path can be represented as:



### 38. PRE-ROUTE AND POST-ROUTE STA

#### Pre-route STA

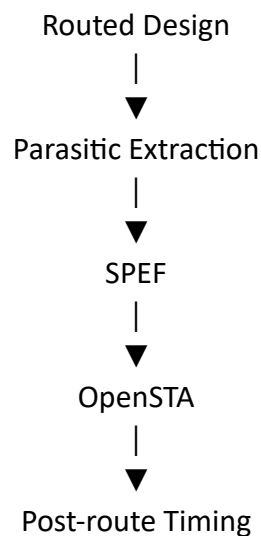
Before routing:



Wire parasitics may be estimated or represented differently depending on the flow.

#### Post-route STA

After routing:



Post-route STA is important for final timing verification because actual routing parasitics can significantly affect delay.

### 40. ANALYSIS

### 39. OBSERVATION TABLE

| S.No. | Parameter           | Observation |
|-------|---------------------|-------------|
| 1     | OpenSTA Version     |             |
| 2     | Top Module          |             |
| 3     | Liberty File        |             |
| 4     | Netlist             |             |
| 5     | SDC File            |             |
| 6     | Clock Name          |             |
| 7     | Clock Period        |             |
| 8     | Setup WNS           |             |
| 9     | Setup TNS           |             |
| 10    | Hold WNS            |             |
| 11    | Hold TNS            |             |
| 12    | Critical Startpoint |             |
| 13    | Critical Endpoint   |             |
| 14    | Timing Status       | PASS / FAIL |

### 41. EXPECTED OUTPUT

### 42. RESULT

#### **45. CONCLUSION**

## EXPERIMENT-9

Perform the Detailed Placement Analysis Using OpenDP

**Contact Hours:** 2 Hours

### 1. AIM

To study and perform **detailed placement** of standard cells using **OpenDP**, understand the detailed placement optimization process, and analyze the placement quality with respect to **legality, cell overlap, displacement, congestion, and timing**.

**Note:** In current OpenROAD releases, the detailed placement functionality is provided by the **dpl module**. It is based on the **OpenDP** detailed-placement engine. Therefore, the syllabus term **OpenDP** corresponds to the current OpenROAD detailed-placement flow using commands such as `detailed_placement`.

### 2. OBJECTIVES

After completing this experiment, the student should be able to:

1. Understand the concept of **placement** in ASIC physical design.
2. Differentiate between **global placement** and **detailed placement**.
3. Understand the purpose of OpenDP.
4. Perform detailed placement using OpenROAD.
5. Detect and resolve:
  - a. Cell overlaps
  - b. Illegal cell locations
  - c. Row misalignment
  - d. Placement violations
6. Analyze cell displacement after legalization.
7. Verify placement using OpenROAD checks.
8. Understand the effect of detailed placement on:
  - a. Timing
  - b. Wirelength
  - c. Congestion
  - d. Placement legality
9. Generate a legalized placement database and DEF file.

### 3. SOFTWARE / TOOLS REQUIRED

| S.No. | Software / File | Purpose                        |
|-------|-----------------|--------------------------------|
| 1     | Linux / Ubuntu  | Operating environment          |
| 2     | OpenROAD        | Physical design implementation |
| 3     | OpenDP          | Detailed placement engine      |
| 4     | OpenDB          | Physical design database       |

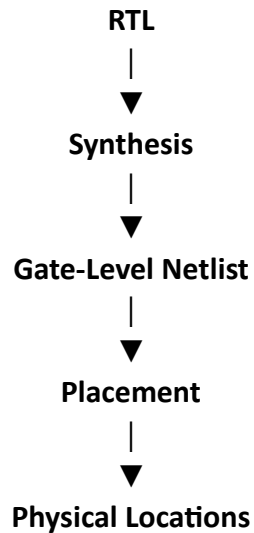
| S.No. | Software / File | Purpose                        |
|-------|-----------------|--------------------------------|
| 5     | Yosys           | RTL synthesis                  |
| 6     | LEF             | Physical library information   |
| 7     | Liberty .lib    | Timing information             |
| 8     | Verilog netlist | Gate-level connectivity        |
| 9     | DEF             | Physical placement information |
| 10    | ODB             | OpenROAD database              |

#### 4. THEORY

##### 4.1 What is Placement?

Placement is the process of determining the physical locations of standard cells and macros inside the chip's core area.

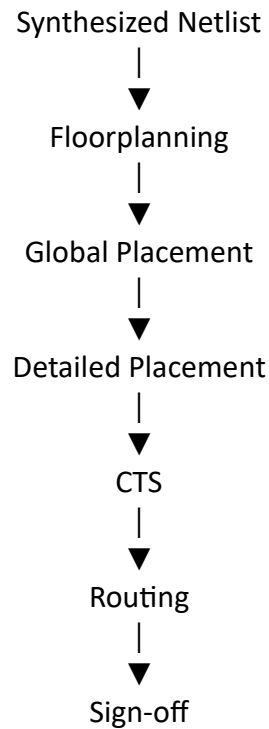
After synthesis, the design consists of logical cells:



The placement stage determines where each cell will physically reside.

## 5. PLACEMENT IN ASIC PHYSICAL DESIGN

A simplified physical-design flow is:



Detailed placement occurs after global placement and before clock-tree synthesis.

## 6. GLOBAL PLACEMENT VS DETAILED PLACEMENT

### Global Placement

Global placement determines approximate locations for cells.

It focuses primarily on:

- Wirelength
- Density
- Timing
- Congestion
- Placement distribution

Cells may temporarily overlap during this stage.

### Detailed Placement

Detailed placement takes the globally placed cells and moves them into **legal locations**.

It attempts to ensure:

- No cell overlap
- Cells are aligned to legal rows
- Cells satisfy placement-site requirements
- Cells remain inside legal regions
- Cell displacement is minimized

## 7. WHAT IS OpenDP?

**OpenDP** is a detailed-placement engine used to legalize and optimize the placement of standard cells.

In current OpenROAD, the detailed-placement functionality is exposed through the **DPL (dpl) module**.

Its major responsibilities include:

- Placement legalization
- Cell overlap removal
- Row alignment
- Cell displacement minimization
- Placement optimization
- Placement legality verification

## 8.WHY DETAILED PLACEMENT IS REQUIRED

- Global placement can produce a solution that is approximately optimal but physically illegal.
- Detailed placement converts this into:

## 9. PLACEMENT LEGALITY

A standard cell placement is considered legal when cells satisfy the physical placement rules of the technology.

Important conditions include:

### 1. No overlap

Two standard cells should not occupy the same physical area.

### 2. Row alignment

Cells must be aligned with legal standard-cell rows.

### 3. Site alignment

Cell locations must correspond to legal placement sites.

### 4. Core boundary

Cells must remain inside the legal placement area.

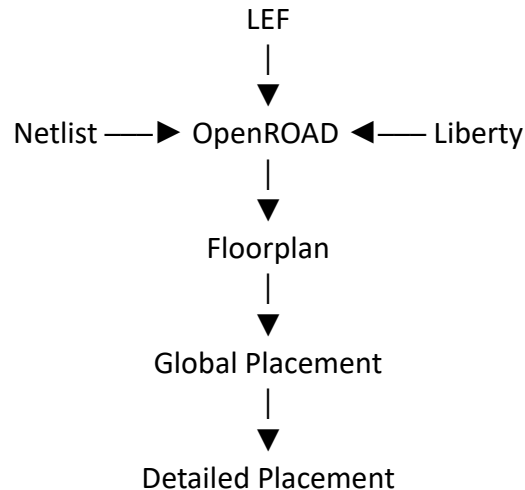
### 5. Placement blockages

Cells must not be placed in prohibited regions.

## 10. INPUTS FOR DETAILED PLACEMENT

Detailed placement generally requires a previously initialized and globally placed design.

Typical inputs include:



**Typical files:**

| File | Information                                     |
|------|-------------------------------------------------|
| .lef | Physical cell/macros and technology information |
| .lib | Cell timing and electrical characteristics      |
| .v   | Gate-level connectivity                         |
| .def | Physical design information                     |
| .odb | OpenROAD database                               |

## 11. PLACEMENT QUALITY PARAMETERS

Detailed placement can be analyzed using several parameters.

### 11.1 Cell Displacement

Cell displacement is the distance a cell moves from its original global-placement location.

Global position      Legal position

The objective is generally to legalize the design with minimal unnecessary displacement.

### 11.2 Wirelength

Placement affects the distance between connected cells.

Cell A ————— Cell B

Wirelength

Excessive wirelength can increase:

- Delay
- Power
- Routing congestion

### **11.3 Congestion**

Poor placement can cause routing congestion.

Low congestion:

High congestion:

### **11.4 Timing**

Cell locations influence:

- Net delay
- Slew
- Setup timing
- Hold timing

Therefore, detailed placement may be followed by timing analysis.

## **12. PROCEDURE**

### **Step 1 – Check OpenROAD Installation**

Run:

- `openroad -version`

You can also start the OpenROAD shell:

- `openroad`

Then:

- `version`

### **13. Step 2 – Prepare the Experiment Directory**

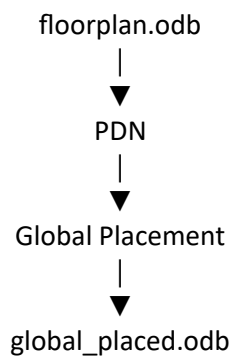
- `mkdir -p ~/openroad_exp9`
- `cd ~/openroad_exp9`

#### 14. Step 3 – Obtain the Placed Design

For this experiment, start with a design that has already undergone:

- Floorplan initialization
- Power planning
- Global placement

For example:



If using the previous experiment flow, use the corresponding placed database generated by your OpenROAD flow.

#### 15. Step 4 – Read the Design Database

Start OpenROAD:

- openroad

Read the placed database:

- read\_db global\_placed.odb

Check the design:

- report\_design\_area

#### 16. Step 5 – Check the Current Placement

Before detailed placement, run:

- check\_placement

This helps identify placement legality problems.

Typical issues may include:

- Cell overlap
- Out-of-bound cells
- Illegal locations
- Row/site alignment issues

### 17. Step 6 – Inspect the Placement

Launch the OpenROAD GUI:

- `gui::show`

Depending on the OpenROAD version, you can also use:

- `gui`

Inspect:

- Core area
- Standard-cell rows
- Cell distribution
- Macros
- Placement density
- Overlapping cells
- Blockages

### 18. Step 7 – Perform Detailed Placement

Run:

- `detailed_placement`

This invokes the current OpenROAD detailed-placement functionality based on OpenDP.

The engine attempts to legalize the placement by moving cells to valid locations.

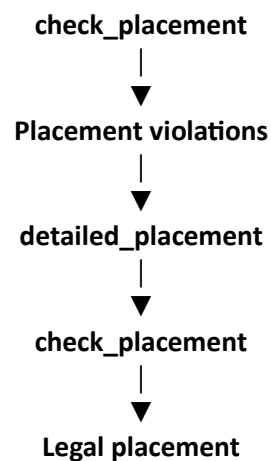
### 19. STEP 8 – Check Placement Again

After detailed placement:

- `check_placement`

Compare the results before and after detailed placement.

Before:



## **20. STEP 9 – Optimize Detailed Placement**

OpenROAD provides placement optimization capabilities that can be used to improve timing and/or placement quality after legalization.

A representative command is:

- `improve_placement`

Depending on the design and OpenROAD version, optimization options may be available for timing and placement objectives.

For example, inspect available options with:

- `help improve_placement`

## **21. STEP 10 – Analyze Timing After Placement**

If timing constraints are loaded, run:

- `report_checks -path_delay max`

For hold:

- `report_checks -path_delay min`

Record:

- Setup WNS
- Setup TNS
- Hold WNS
- Hold TNS

Compare timing before and after detailed placement.

## **22. STEP 11 – Analyze Placement Area**

Run:

- `report_design_area`

Record:

- Core area
- Die area
- Standard-cell area
- Utilization

## **23. STEP 12 – Save the Detailed Placement**

Save the legalized database:

- `write_db detailed_placement.odb`

Save the DEF:

- `write_def detailed_placement.def`

These files can be used in subsequent stages.

## 24. COMPLETE REPRESENTATIVE OPENROAD SCRIPT

Create a Tcl script:

- `gvim exp9_detailed_placement.tcl`

Example:

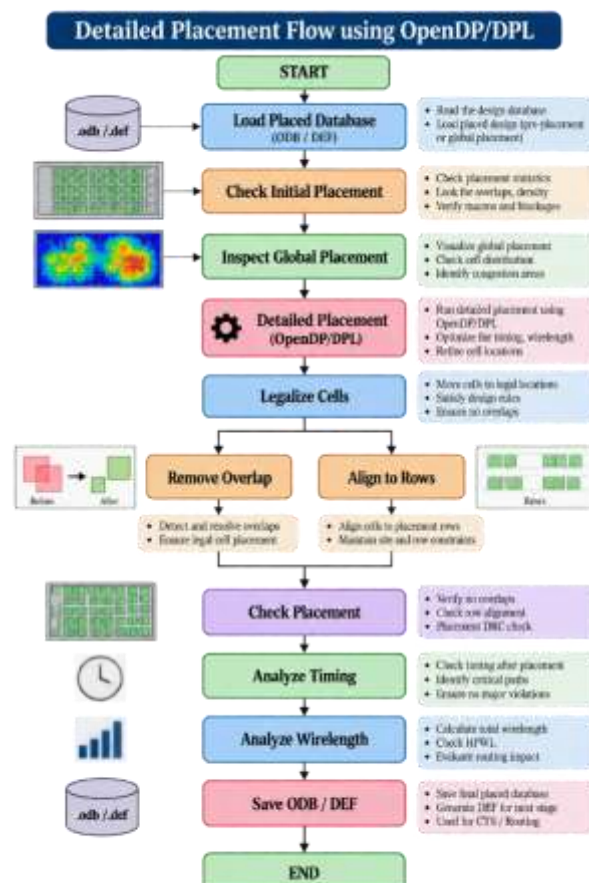
- `read_db global_placed.odb`
- `report_design_area`
- `check_placement`
- `detailed_placement`
- `check_placement`
- `report_design_area`
- `report_checks -path_delay max`
- `report_checks -path_delay min`
- `write_db detailed_placement.odb`
- `write_def detailed_placement.def`

Run:

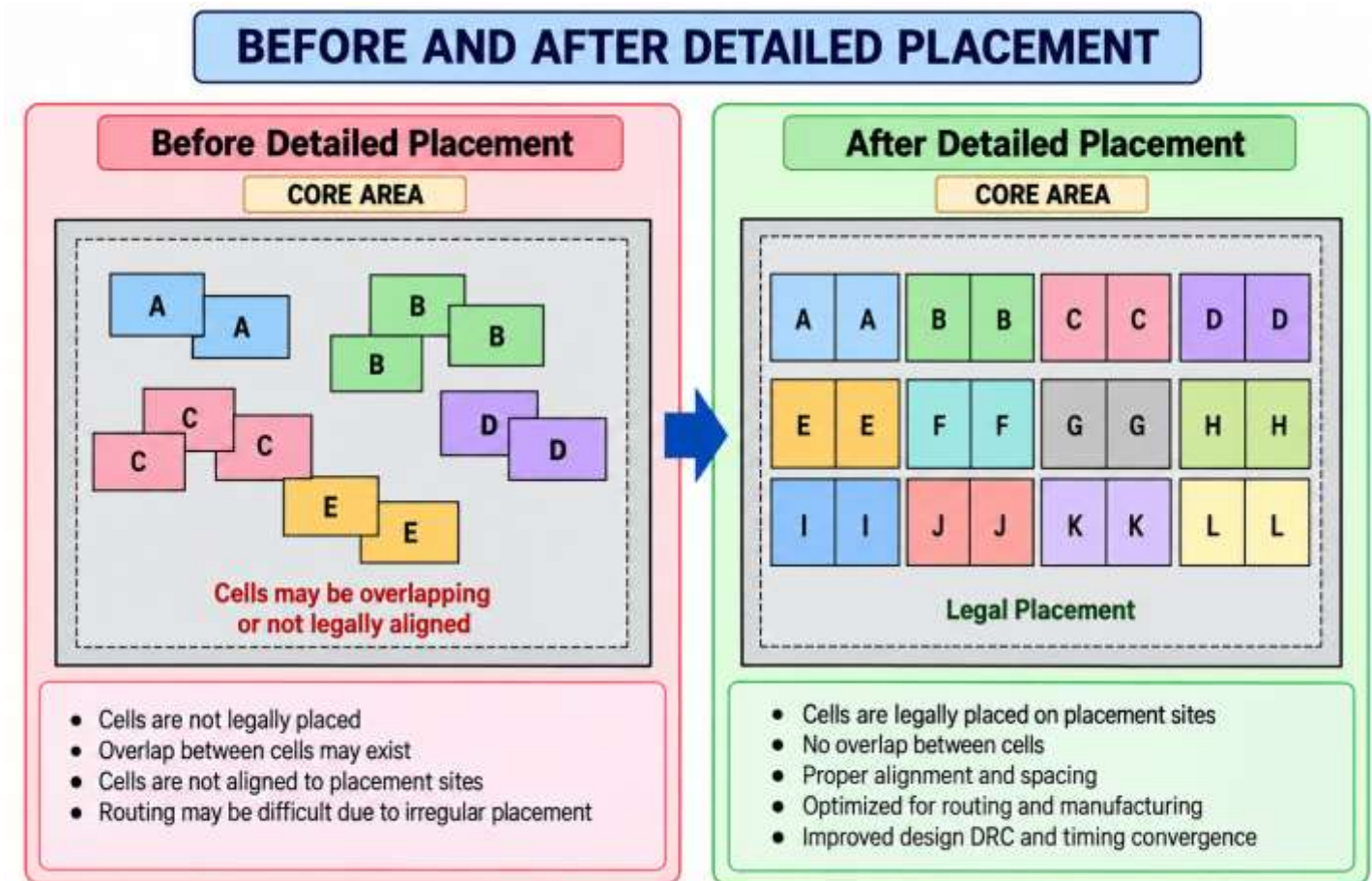
- `openroad exp9_detailed_placement.tcl`

The exact input database name, timing setup, and optimization commands depend on the design and OpenROAD flow being used.

## 25. DETAILED PLACEMENT FLOW



## 26. BEFORE AND AFTER DETAILED PLACEMENT



## 27. OBSERVATION TABLE

| S.No. | Parameter        | Before Detailed Placement | After Detailed Placement |
|-------|------------------|---------------------------|--------------------------|
| 1     | Design Name      |                           |                          |
| 2     | Core Area        |                           |                          |
| 3     | Die Area         |                           |                          |
| 4     | Utilization      |                           |                          |
| 5     | Placement Status |                           |                          |
| 6     | Cell Overlap     |                           |                          |
| 7     | Row Alignment    |                           |                          |
| 8     | Setup WNS        |                           |                          |
| 9     | Setup TNS        |                           |                          |
| 10    | Hold WNS         |                           |                          |

| S.No. | Parameter         | Before Detailed Placement | After Detailed Placement |
|-------|-------------------|---------------------------|--------------------------|
| 11    | Hold TNS          |                           |                          |
| 12    | Placement Quality |                           |                          |

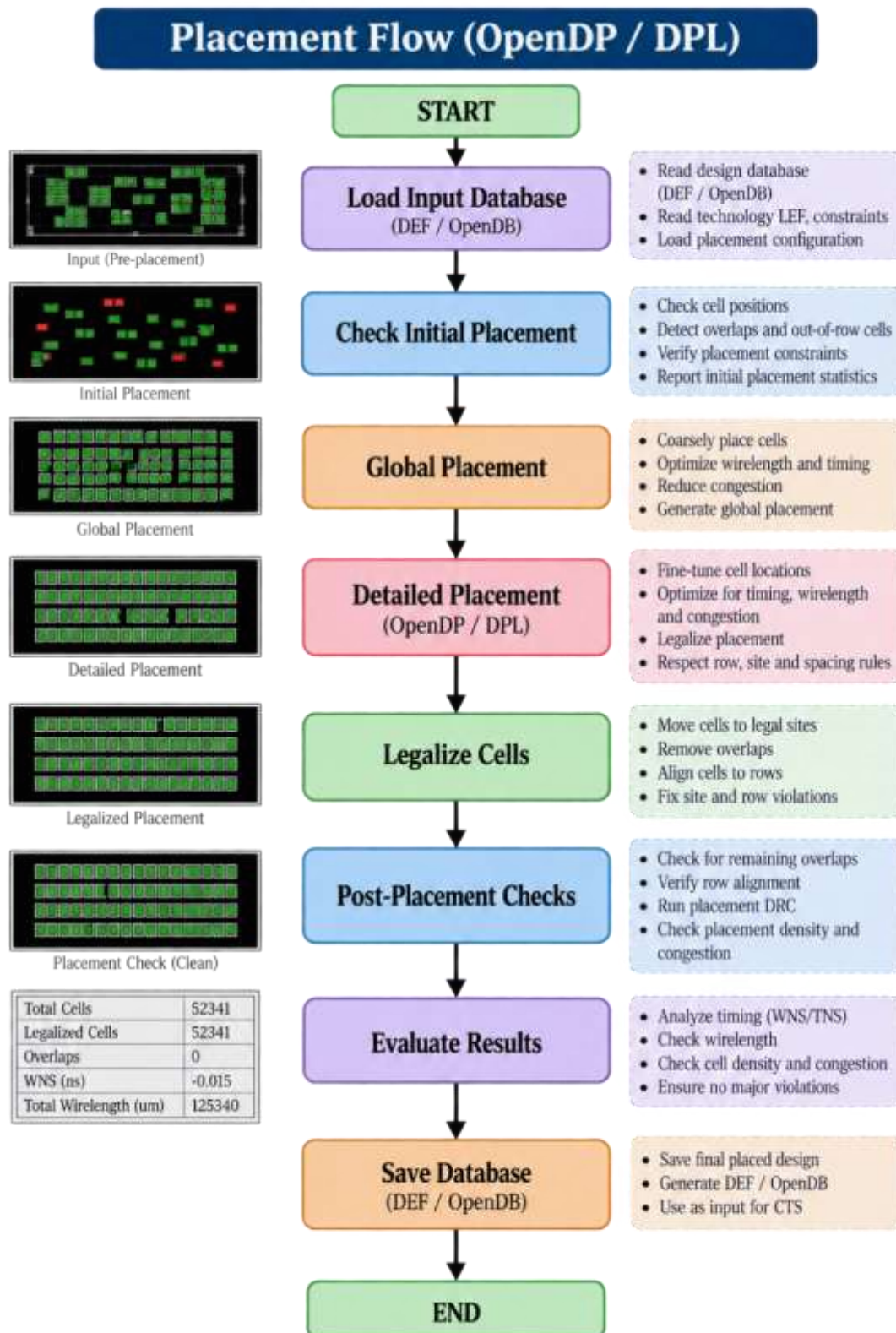
## 28. ANALYSIS

## 29. EXPECTED OUTPUT

## 30. IMPORTANT OPENROAD COMMANDS

| Command                 | Purpose                                 |
|-------------------------|-----------------------------------------|
| read_db                 | Read OpenROAD database                  |
| report_design_area      | Report design area and utilization      |
| check_placement         | Check placement legality                |
| detailed_placement      | Perform detailed placement/legalization |
| improve_placement       | Improve placement quality               |
| report_checks           | Analyze timing                          |
| write_db                | Save OpenROAD database                  |
| write_def               | Write DEF                               |
| gui::show               | Display GUI                             |
| help detailed_placement | Display command help                    |

### 31. ROLE OF DETAILED PLACEMENT IN THE ASIC FLOW



Detailed placement is therefore the bridge between **global placement** and **clock-tree synthesis**.

### 32. EFFECT OF DETAILED PLACEMENT

| Parameter          | Effect of Detailed Placement                |
|--------------------|---------------------------------------------|
| Cell overlap       | Reduced/removed                             |
| Placement legality | Improved                                    |
| Row alignment      | Corrected                                   |
| Cell displacement  | Minimized where possible                    |
| Wirelength         | Can change                                  |
| Congestion         | Can improve or worsen depending on movement |
| Timing             | Can improve or degrade                      |
| Routing readiness  | Improved                                    |
| CTS readiness      | Improved                                    |

### 35. RESULT

### 36. CONCLUSION

### EXPERIMENT-10

#### Analyze Clock Tree Synthesis Using TritonCTS 2.0

**Contact Hours:** 2 Hours

#### 1. AIM

To study and perform **Clock Tree Synthesis (CTS)** using **TritonCTS 2.0** in OpenROAD, and to analyze clock-tree characteristics such as **clock skew, insertion delay, clock latency, buffer insertion, clock-tree structure, and timing impact**.

#### 2. OBJECTIVES

After completing this experiment, the student should be able to:

1. Understand the purpose of Clock Tree Synthesis.
2. Understand the problems associated with clock distribution.
3. Study the architecture of a clock tree.
4. Understand **TritonCTS 2.0**.
5. Perform CTS using OpenROAD.
6. Analyze:
  - a. Clock insertion delay
  - b. Clock skew
  - c. Clock latency
  - d. Clock-tree buffering
  - e. Clock fanout
7. Generate clock-tree reports.
8. Analyze timing before and after CTS.
9. Understand the importance of CTS before routing.
10. Save the CTS database for subsequent routing stages.

#### 3. SOFTWARE / TOOLS REQUIRED

| S.No. | Software / File | Purpose                        |
|-------|-----------------|--------------------------------|
| 1     | Linux / Ubuntu  | Operating environment          |
| 2     | OpenROAD        | Physical design implementation |
| 3     | TritonCTS 2.0   | Clock Tree Synthesis           |
| 4     | OpenDB          | Physical design database       |
| 5     | OpenSTA         | Static timing analysis         |
| 6     | LEF             | Physical cell information      |
| 7     | Liberty .lib    | Timing and cell information    |
| 8     | Verilog         | Gate-level netlist             |
| 9     | SDC             | Clock and timing constraints   |

| S.No. | Software / File | Purpose                  |
|-------|-----------------|--------------------------|
| 10    | DEF / ODB       | Physical design database |

## 4. THEORY

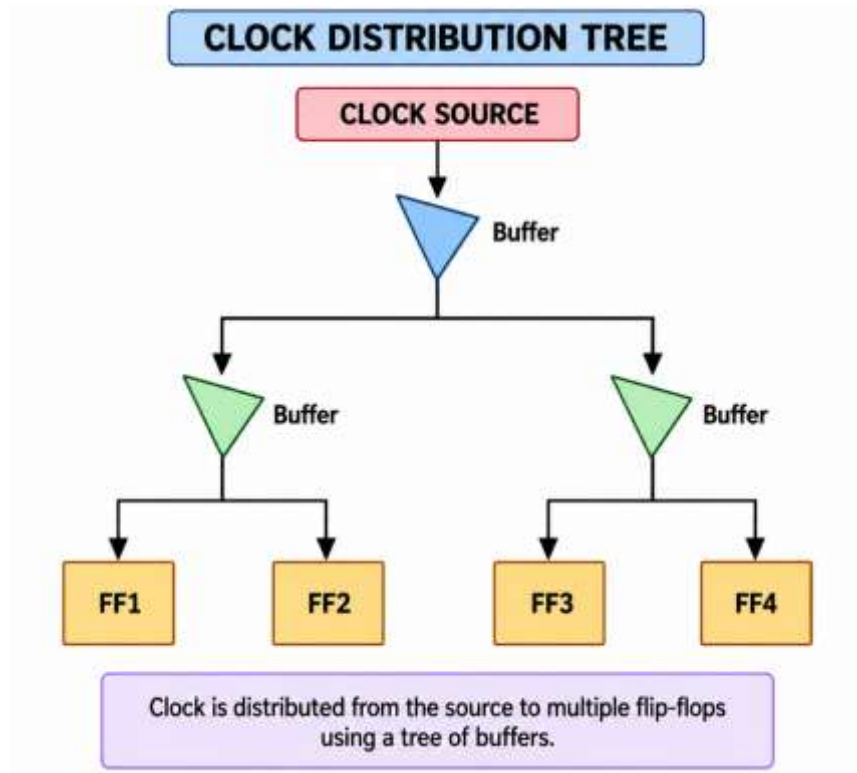
### 4.1 What is Clock Tree Synthesis?

**Clock Tree Synthesis (CTS)** is the process of creating a physical clock-distribution network that delivers the clock signal from its source to all required sequential elements.

A clock network typically contains:

- Clock source
- Clock buffers
- Clock inverters
- Clock branches
- Flip-flop clock pins

Simplified structure:



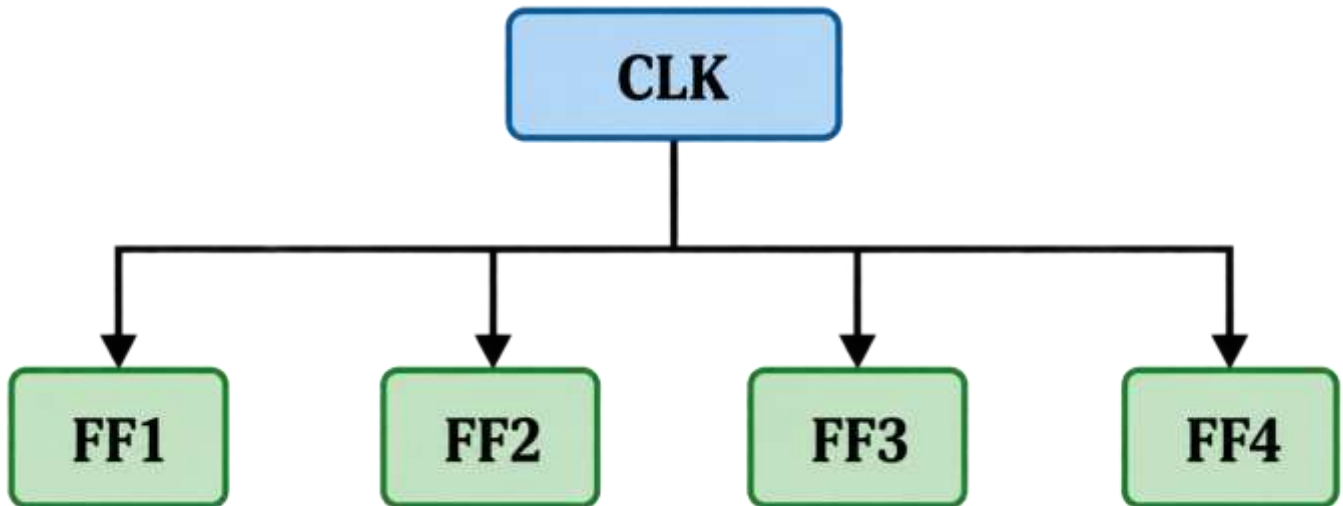
The objective is to distribute the clock with acceptable:

- Skew
- Insertion delay
- Slew
- Fanout
- Power

## 5. WHY IS CTS REQUIRED?

Before CTS, a synthesized design may contain a clock net connected directly to many flip-flops.

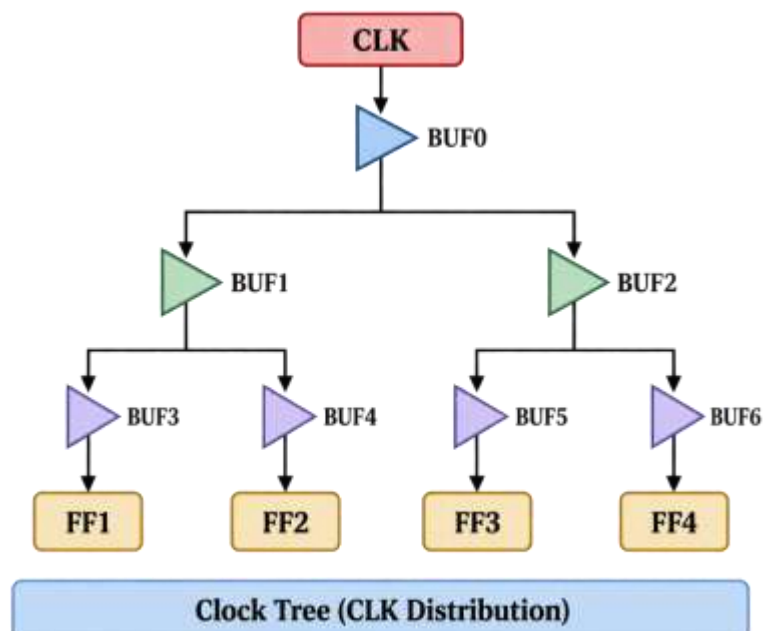
For example:



This is not an ideal physical clock network because:

- Fanout can be very high.
- Wire lengths differ.
- Clock arrival times differ.
- Signal transition can become slow.
- Timing violations can occur.

CTS converts the clock network into a buffered tree:



## 6. CLOCK TREE OBJECTIVES

The major objectives of CTS are:

### 1. Minimize Clock Skew

Clock skew should be kept as small as possible.

### 2. Control Clock Insertion Delay

The time taken for the clock to travel from source to sink should be controlled.

### 3. Control Slew

The clock transition should remain within acceptable limits.

### 4. Manage Fanout

A single clock buffer should not drive an excessive number of loads.

### 5. Maintain Timing

CTS should help the design satisfy setup and hold timing requirements.

### 6. Control Power

Clock networks can consume significant dynamic power, so excessive buffering should be avoided

## 7. CLOCK SKEW

**Clock skew** is the difference between clock arrival times at two sequential elements.

For example:

Clock arrival at FF1 = 1.20 ns

Clock arrival at FF2 = 1.35 ns

Therefore:

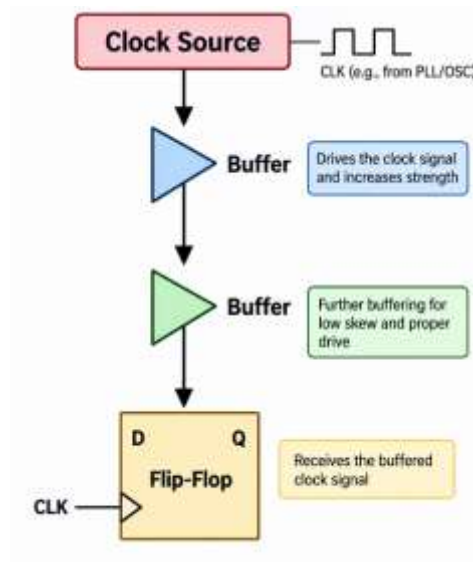
$$Skew = 1.35 - 1.20$$

$$Skew = 0.15ns$$

The objective of CTS is generally to minimize undesirable skew.

## 8. CLOCK INSERTION DELAY

Clock insertion delay, also called clock latency, represents the delay from the clock source to a clock sink.



If:

Buffer delay = 0.2 ns

Buffer delay = 0.3 ns

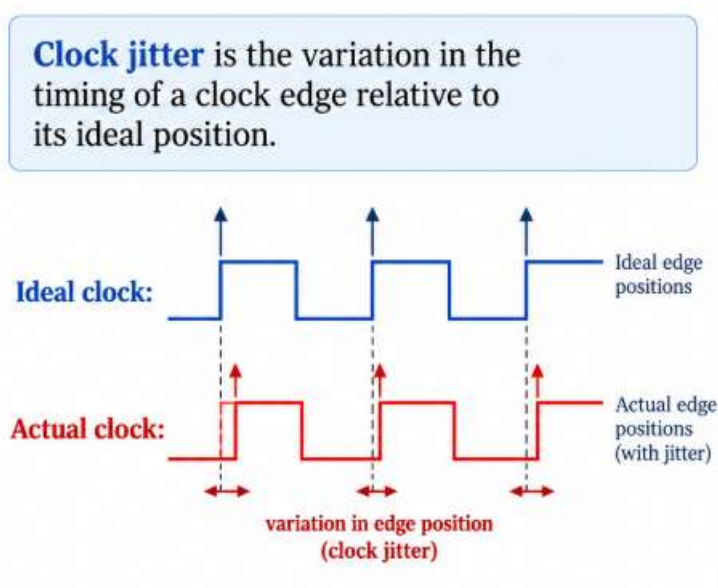
Wire delay = 0.2 ns

Then approximately:

$$\begin{aligned}
 \text{Insertion Delay} &= 0.2 + 0.3 + 0.2 \\
 &= 0.7\text{ns}
 \end{aligned}$$

## 9. CLOCK JITTER

**Clock jitter** is the variation in the timing of a clock edge relative to its ideal position.



Clock uncertainty in timing constraints can account for clock-related variations.

Example:

set\_clock\_uncertainty 0.2 [get\_clocks clk]

## 10. CLOCK LATENCY

Clock latency can be divided conceptually into:

### Source Latency

Delay from the external clock source to the clock definition point.

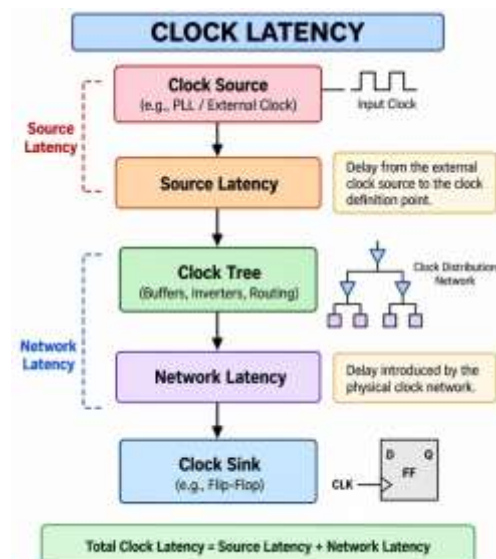
### Network Latency

Delay introduced by the physical clock network.

## 11. CLOCK TREE STRUCTURES

Common clock-tree structures include:

- **Balanced Tree**
- **H-Tree**
- **Clock Mesh**



A mesh uses multiple interconnected clock wires and can provide low skew but generally has greater routing and power cost.

TritonCTS primarily constructs a buffered clock tree rather than a conventional full clock mesh.

## 12. WHAT IS TRITONCTS 2.0?

**TritonCTS 2.0** is the clock-tree synthesis engine used in the OpenROAD project.

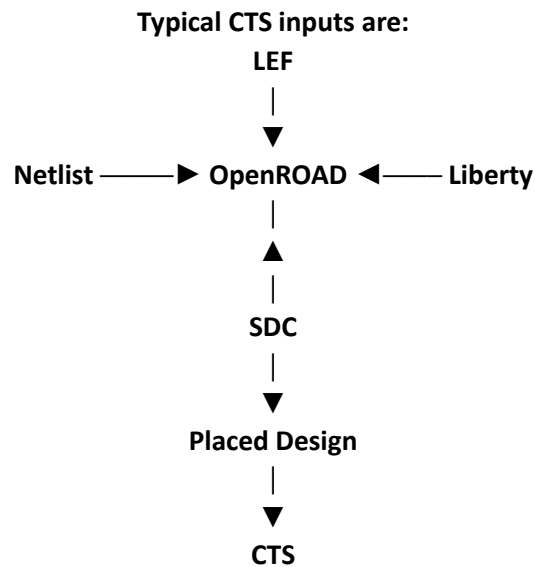
It performs clock-tree construction by inserting clock buffers and creating a clock distribution network between clock sources and sinks.

In current OpenROAD, CTS is invoked using:

- `clock_tree_synthesis`

The CTS module is based on the TritonCTS 2.0 technology.

## 13. INPUTS REQUIRED FOR CTS



Important information includes:

- Placed standard cells
- Clock definitions
- Clock source
- Clock sinks
- Available clock buffers
- Liberty timing information
- Technology LEF
- Routing information

#### 14. PROCEDURE

##### Step 1 – Check OpenROAD Version

Run:

- openroad -version

Start OpenROAD:

- openroad

Check:

- version

##### 15. STEP 2 – Check CTS Command

Inside OpenROAD:

- help clock\_tree\_synthesis

This displays the CTS command and the options supported by the installed OpenROAD version.

##### 16. STEP 3 – Create Experiment Directory

- mkdir -p ~/openroad\_exp10
- cd ~/openroad\_exp10

##### 17. STEP 4 – Load the Detailed-Placed Database

Use the database generated after Experiment-9:

- read\_db detailed\_placement.odb

Check the design:

- report\_design\_area

##### 18. STEP 5 – Check Placement

Before CTS, ensure the design has a legal placement:

- check\_placement

The design should not have major placement violations.

##### 19. STEP 6 – Check Clock Definitions

If the SDC has already been loaded into the database, inspect the clocks:

- report\_clocks

You should identify:

- Clock name
- Clock period
- Clock source
- Clock waveform

For example:

Clock name : clk

Clock period : 10 ns

## 20. STEP 7 – Load Timing Information if Required

If starting from a database that does not contain all required timing information, load the relevant Liberty and constraints:

- `read_liberty <standard_cell.lib>`

Then:

- `read_sdc <constraints.sdc>`

The exact setup depends on how the input ODB was generated.

## 21. STEP 8 – Configure CTS Buffer Cells

TritonCTS requires appropriate clock-buffer cells.

A representative command is:

```
set_clock_tree_options \  
-buffer_list {CLKBUF_X1 CLKBUF_X2 CLKBUF_X4}
```

The actual clock-buffer cell names must exist in your technology library.

For example, a PDK may provide cells such as:

- CLKBUF\_X1
- CLKBUF\_X2
- CLKBUF\_X4
- CLKINV\_X1
- CLKINV\_X2

Always check the actual Liberty/LEF library before using these names.

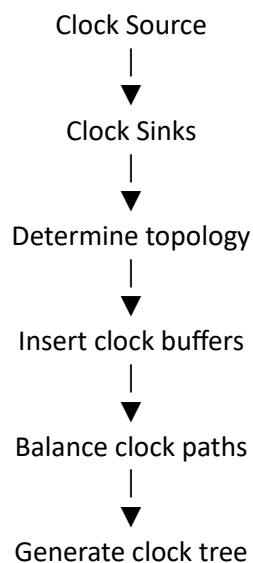
## 22. STEP 9 – Perform Clock Tree Synthesis

Run:

- `clock_tree_synthesis`

TritonCTS analyzes the clock network and inserts the required clock-tree buffers.

The conceptual operation is:



### **23. STEP 10 – Useful CTS Options**

The exact options available depend on the installed OpenROAD version.

Check them using:

- `help clock_tree_synthesis`

Common CTS concepts/options include:

- `clock_tree_synthesis`
- `-buffer_list`
- `-root_buf`
- `-wire_unit`
- `-obstruction_aware`
- `-delay_buffer_derate`

Do not copy technology-specific values blindly; use the command help and the PDK configuration.

### **24. STEP 11 – Analyze the Clock Tree**

After CTS, inspect the clock network.

Run:

- `report_clock_skew`

This report is useful for analyzing clock arrival differences.

You can also use:

- `report_cts`

if supported by your OpenROAD version.

Check available reporting commands:

- `help report_cts`
- `help report_clock_skew`

### **25. STEP 12 – Analyze Timing After CTS**

Run maximum-delay timing analysis:

- `report_checks -path_delay max`

For minimum-delay analysis:

- `report_checks -path_delay min`

Record:

- Setup WNS
- Setup TNS
- Hold WNS
- Hold TNS

**26. STEP 13 – Analyze Clock Paths**

A timing path after CTS contains clock-tree delays.

Conceptually:

Different branches may have different delays.

CTS attempts to balance these paths.

**27. STEP 14 – Inspect CTS in GUI**

Launch the GUI:

- `gui::show`

Inspect:

- Clock source
- Clock buffers
- Clock branches
- Flip-flop clock pins
- Clock-tree topology
- Cell locations

The clock tree should appear as a branching network connecting the clock source to sequential sinks.

**28. STEP 15 – Check Placement After CTS**

Run:

- `check_placement`

CTS inserts new cells, so it is useful to verify that the resulting placement remains legal.

**29. STEP 16 – Save the CTS Database**

Save the OpenROAD database:

- `write_db cts.odt`

Save the DEF:

- `write_def cts.def`

These files are used by later routing stages.

**30. COMPLETE REPRESENTATIVE CTS SCRIPT**

Create:

- `gvim exp10_cts.tcl`

Use a technology-specific configuration similar to:

```
read_db detailed_placement.odt
```

```
report_design_area
```

```
check_placement
```

report\_clocks

```
set_clock_tree_options \
-buffer_list {CLKBUF_X1 CLKBUF_X2 CLKBUF_X4}
```

clock\_tree\_synthesis

check\_placement

report\_clock\_skew

report\_checks -path\_delay max

report\_checks -path\_delay min

write\_db cts.oddb

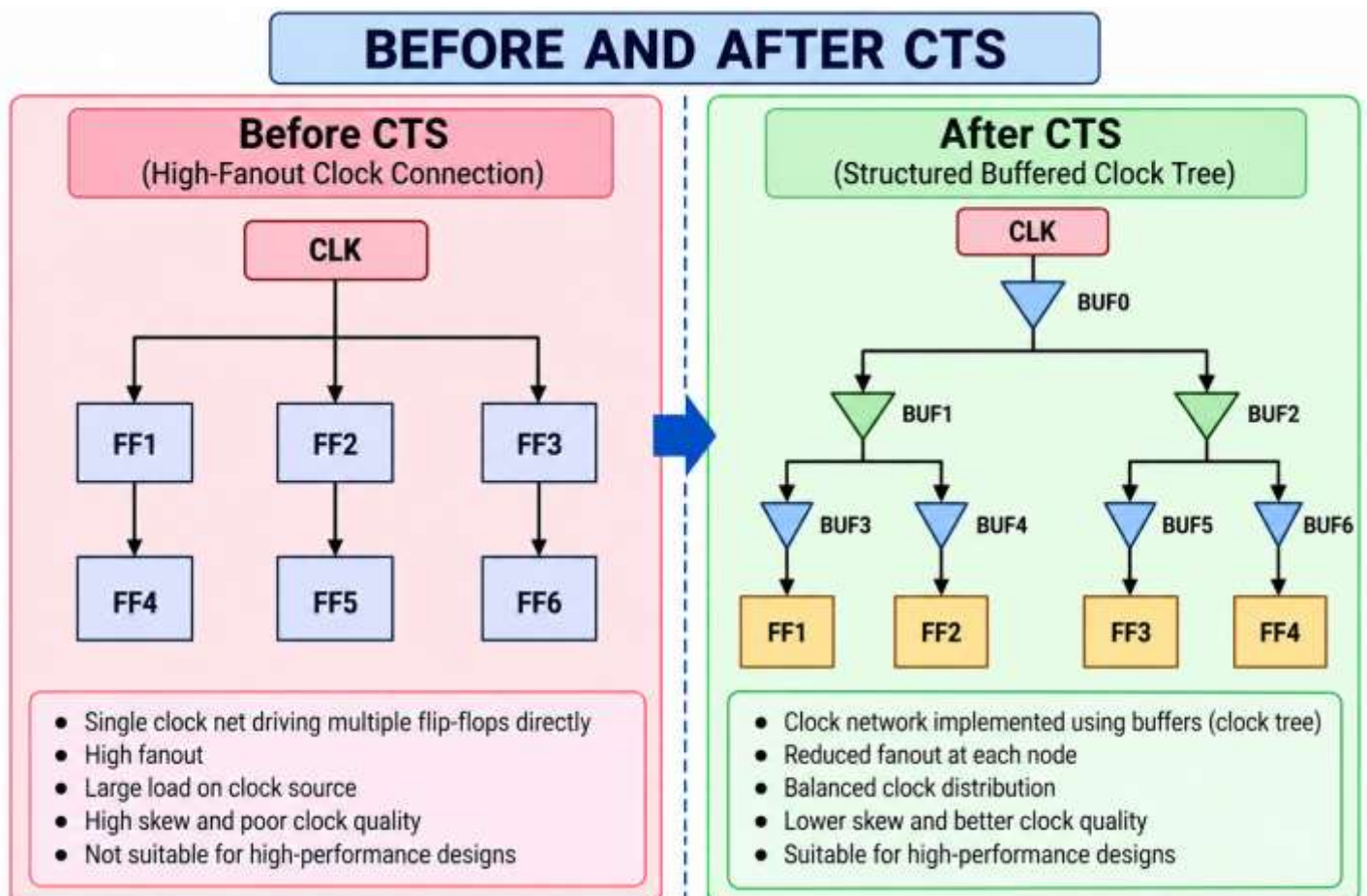
write\_def cts.def

Run:

- openroad exp10\_cts.tcl

**Important:** CLKBUF\_X1, CLKBUF\_X2, and CLKBUF\_X4 are example names. Replace them with clock-buffer cells available in your technology library.

### 31. BEFORE CTS and AFTER CTS



### 34. CLOCK SKEW ANALYSIS

Suppose:

Clock arrival at FF1 = 1.25 ns

Clock arrival at FF2 = 1.30 ns

Clock arrival at FF3 = 1.28 ns

Clock arrival at FF4 = 1.27 ns

Maximum arrival time:

1.30ns

Minimum arrival time:

1.25ns

Therefore:

$$\text{Clock Skew} = 1.30 - 1.25$$

$$\text{Clock Skew} = 0.05\text{ns}$$

Thus the clock skew is:

0.05 ns = 50 ps

### 32. CLOCK TREE QUALITY PARAMETERS

| Parameter       | Description                       | Desired Behavior |
|-----------------|-----------------------------------|------------------|
| Clock Skew      | Difference in clock arrival times | Low              |
| Insertion Delay | Source-to-sink clock delay        | Controlled       |
| Clock Slew      | Clock transition time             | Within limits    |
| Fanout          | Number of loads driven            | Controlled       |
| Buffer Count    | Number of inserted clock cells    | Optimized        |
| Clock Power     | Power consumed by clock network   | Low              |
| Timing Slack    | Timing margin                     | Positive         |
| Clock Balance   | Similarity of branch delays       | High             |

### 32. OBSERVATION TABLE

| S.No. | Parameter    | Before CTS | After CTS |
|-------|--------------|------------|-----------|
| 1     | Design Name  |            |           |
| 2     | Clock Name   |            |           |
| 3     | Clock Period |            |           |

| S.No. | Parameter             | Before CTS | After CTS |
|-------|-----------------------|------------|-----------|
| 4     | Number of Clock Sinks |            |           |
| 5     | Clock Buffers         |            |           |
| 6     | Clock Insertion Delay |            |           |
| 7     | Clock Skew            |            |           |
| 8     | Maximum Clock Slew    |            |           |
| 9     | Setup WNS             |            |           |
| 10    | Setup TNS             |            |           |
| 11    | Hold WNS              |            |           |
| 12    | Hold TNS              |            |           |
| 13    | Placement Status      |            |           |

### 33. ANALYSIS

### 34. EXPECTED OUTPUT

### 37. RESULT

### EXPERIMENT–11

Perform Global Routing Using TritonRoute and Do Analysis of Design

**Contact Hours: 2 Hours**

#### 1. AIM

To study the routing process in ASIC physical design, perform global routing using OpenROAD, understand the role of TritonRoute in detailed routing, and analyze routing quality using parameters such as wirelength, congestion, routing guides, design-rule violations, and timing.

#### 2. OBJECTIVES

After completing this experiment, the student should be able to:

1. Understand the concept of routing in ASIC physical design.
2. Understand the difference between:
  - a. Global routing
  - b. Detailed routing
3. Understand the role of FastRoute in OpenROAD global routing.
4. Understand the role of TritonRoute in detailed routing.
5. Perform global routing using global\_route.
6. Generate routing guides.
7. Perform detailed routing using detailed\_route.
8. Analyze:
  - a. Routing congestion
  - b. Wirelength
  - c. Routing resources
  - d. Design-rule violations
  - e. Antenna-related issues
  - f. Timing after routing
9. Save the routed design in ODB and DEF formats.
10. Understand how routing affects post-route timing.

#### 3. SOFTWARE / TOOLS REQUIRED

| S.No. | Software / File | Purpose                                  |
|-------|-----------------|------------------------------------------|
| 1     | Linux / Ubuntu  | Operating environment                    |
| 2     | OpenROAD        | Physical implementation                  |
| 3     | FastRoute       | Global routing engine                    |
| 4     | TritonRoute     | Detailed routing engine                  |
| 5     | OpenDB          | Design database                          |
| 6     | OpenSTA         | Timing analysis                          |
| 7     | LEF             | Technology and physical-cell information |
| 8     | Liberty .lib    | Cell timing information                  |
| 9     | Verilog         | Gate-level netlist                       |

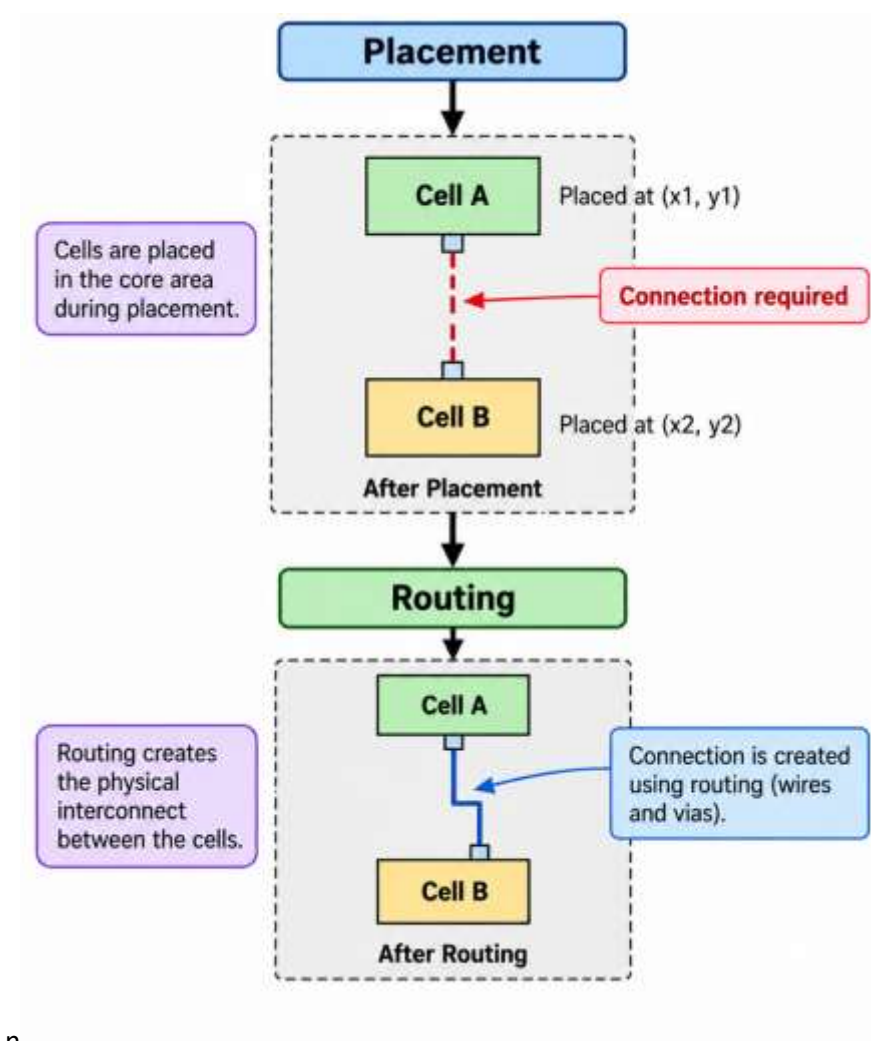
| S.No. | Software / File | Purpose                        |
|-------|-----------------|--------------------------------|
| 10    | SDC             | Timing constraints             |
| 11    | DEF             | Physical design representation |
| 12    | ODB             | OpenROAD database              |

## 4. THEORY

### 4.1 What is Routing?

**Routing** is the process of creating physical metal connections between the pins of cells according to the logical connectivity specified by the netlist.

After placement and CTS, the design contains physically positioned cells but the signal nets still need to be connected.

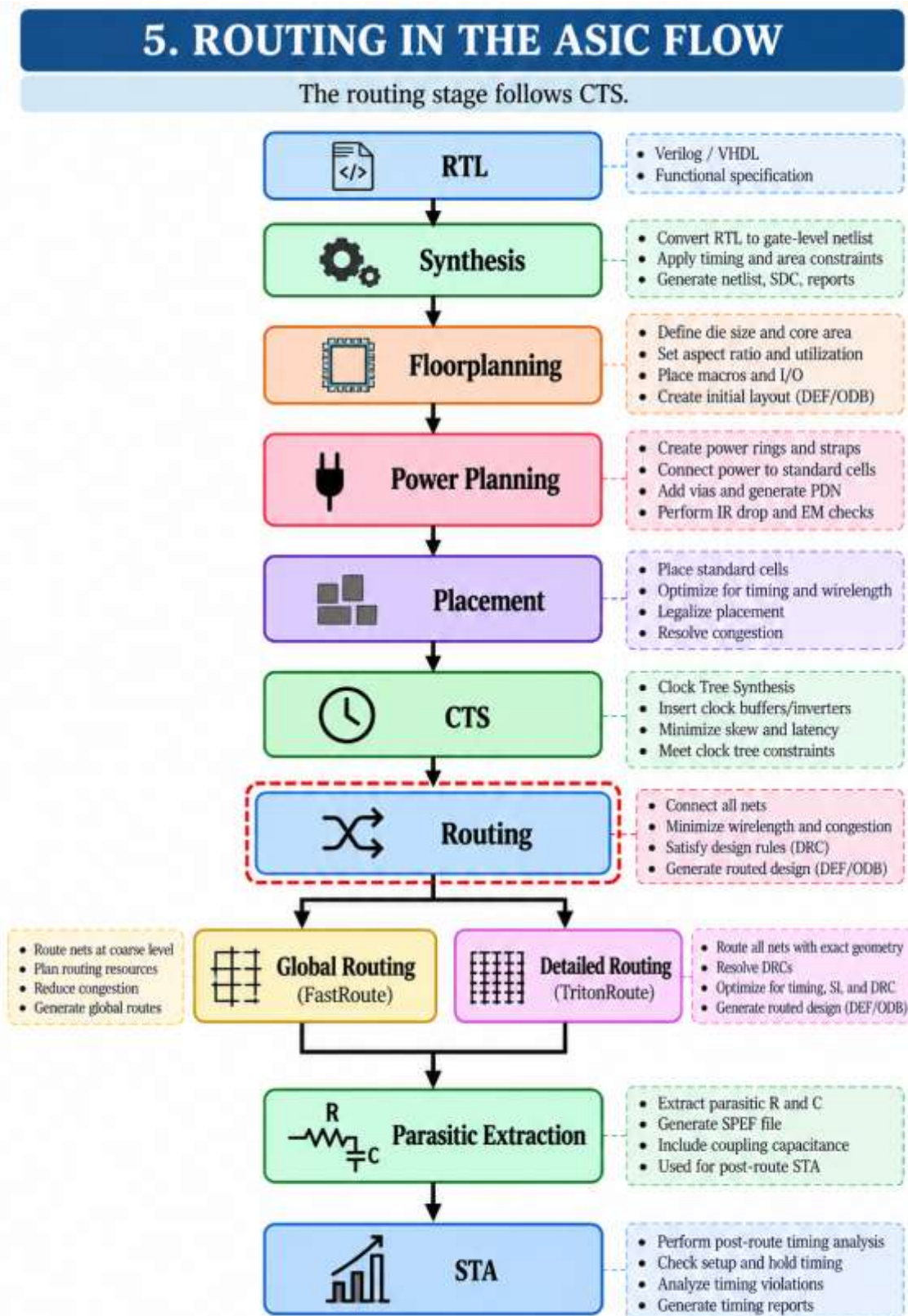


n

Routing creates the actual physical metal paths.

## 5. ROUTING IN THE ASIC FLOW

The routing stage follows CTS.



## 6. GLOBAL ROUTING

- Global routing determines which routing regions and metal layers should be used to connect each net.
- It does not necessarily generate the final geometrically exact wires.
- The chip is divided conceptually into routing regions called global-routing cells or GCells.
- The router determines a path through these routing regions.

### Chip divided into GCells

|    |     |     |     |
|----|-----|-----|-----|
| G1 | G2  | G3  | G4  |
| G5 | G6  | G7  | G8  |
| G9 | G10 | G11 | G12 |

#### Legend

- GCell (routing region)
- Global route path (for a net)
- Source / Sink

### Example Global Routing Path

|    |     |     |     |
|----|-----|-----|-----|
| G1 | G2  | G3  | G4  |
| G5 | G6  | G7  | G8  |
| G9 | G10 | G11 | G12 |

The global router determines a path through GCells but does not create the exact wires.

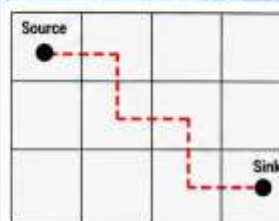
## 7. DETAILED ROUTING

- Detailed routing converts the global-routing solution into actual physical wires while satisfying technology-specific design rules.

- It determines:
  - Exact wire coordinates
  - Metal layer
  - Wire width
  - Wire spacing
  - Via locations
  - Design-rule constraints

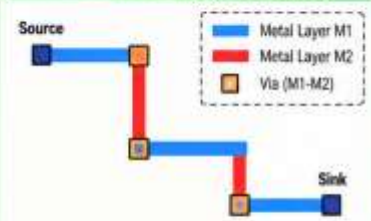


### Global Routing (Conceptual)



- Shows routing regions and preferred metal layers.
- Provides routing guides (coarse paths).
- Does not define exact geometry.

### Detailed Routing (Actual Wires)



- Converts guides into exact wires.
- Determines precise coordinates, metal layer, width, spacing and via locations.
- Satisfies all technology design rules.
- Generates final routable geometry.

## 6. GLOBAL ROUTING

Global routing determines **which routing regions and metal layers should be used** to connect each net.

It does not necessarily generate the final geometrically exact wires.

The chip is divided conceptually into routing regions called **global-routing cells** or **GCells**.

The router determines a path through these routing regions.

## 7. DETAILED ROUTING

Detailed routing converts the global-routing solution into actual physical wires while satisfying technology-specific design rules.

It determines:

- Exact wire coordinates
- Metal layer
- Wire width
- Wire spacing
- Via locations
- Design-rule constraints

## 8. FASTROUTE AND TRITONROUTE

### FastRoute

OpenROAD's current global-routing stage uses the **FastRoute-based grt module**.

Command:

- global\_route

Its purpose is to:

- Estimate routing paths
- Generate routing guides
- Analyze congestion
- Assign routing resources

### TritonRoute

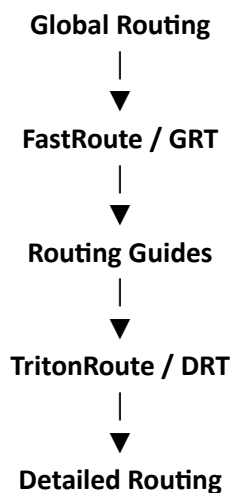
**TritonRoute** is OpenROAD's detailed-routing engine.

The current command is:

detailed\_route

It creates detailed physical routing while considering design rules.

Therefore:



## 9. ROUTING OBJECTIVES

The major objectives of routing are:

### 1. Complete Connectivity

All required nets should be connected.

### 2. Minimize Wirelength

Shorter wires generally reduce:

- Delay
- Capacitance
- Power

### 3. Minimize Congestion

Routing resources should be distributed efficiently.

### 4. Satisfy Design Rules

The routing must satisfy:

- Minimum width
- Minimum spacing
- Via rules
- Metal rules
- End-of-line rules

#### 5. Meet Timing

Routing should preserve or improve timing closure.

#### 6. Minimize Power

Long interconnects and excessive capacitance can increase dynamic power.

### 10. ROUTING LAYERS

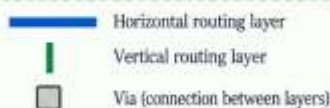
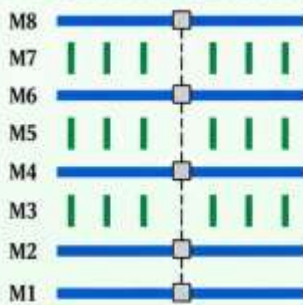
Modern ASICs contain multiple metal layers.

## 10. ROUTING LAYERS

**Modern ASICs** contain multiple metal layers.

- Multiple metal layers are used to route signals and power.
- They help in reducing congestion and enable complex designs.
- Vias are used to make connections between adjacent metal layers.
- Typically, neighboring metal layers use different preferred routing directions.
- The actual directions and number of layers depend on the technology (e.g., 7nm, 14nm, 28nm, etc.).

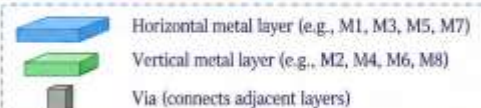
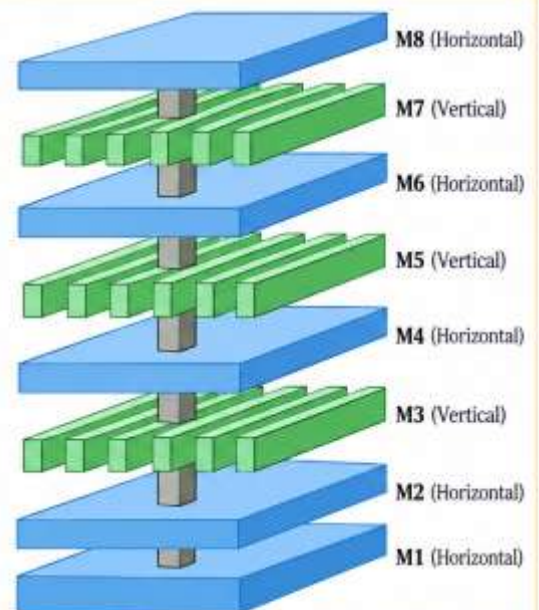
#### Example:



#### Preferred Routing Directions

| Metal Layer | Preferred Direction | Example Representation |
|-------------|---------------------|------------------------|
| M1          | Horizontal          | Blue bar               |
| M2          | Vertical            | Green bar              |
| M3          | Horizontal          | Blue bar               |
| M4          | Vertical            | Green bar              |
| M5          | Horizontal          | Blue bar               |
| M6          | Vertical            | Green bar              |
| M7          | Horizontal          | Blue bar               |
| M8          | Vertical            | Green bar              |

#### 3D View of Routing Layers



## 11. VIAS

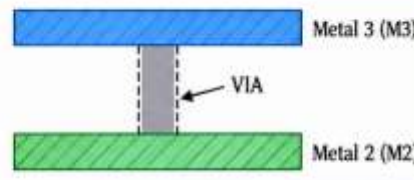
A **via** provides a connection between different metal layers.

11. VIAS

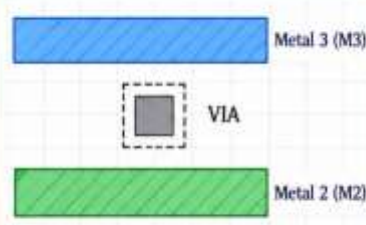
**A via** provides a connection between different metal layers.

- A via is a small vertical interconnect that connects one metal layer to another.
- It is typically filled with a conductive material (e.g., tungsten).
- Vias enable signals and power to move between metal layers.
- Without vias, signals cannot transition between metal layers.

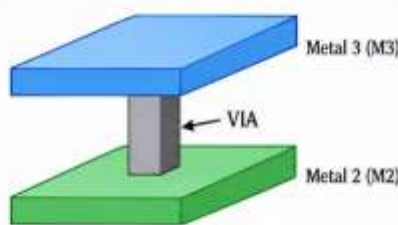
**Example: Via between Metal Layers**



**2D Layout View**



**3D View**



**Key Points**

- Vias provide electrical connection between adjacent metal layers.
- They are placed at intersections of metal layers.
- Vias are filled with a conductive material (usually tungsten).
- Different types of vias exist depending on the technology (e.g., single via, double via, stacked via).
- Without vias, signals cannot transition between metal layers.

Vias enable vertical interconnects in multi-layer metal routing.

## 12. ROUTING CONGESTION

Routing congestion occurs when routing demand exceeds the available routing resources.

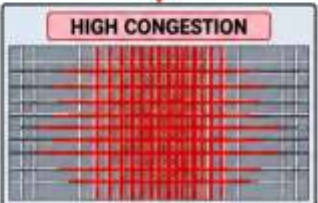
12. ROUTING CONGESTION

Routing congestion occurs when **routing demand exceeds the available routing resources**.

**Routing Demand**  
(Number of nets to be routed)

↓

HIGH CONGESTION



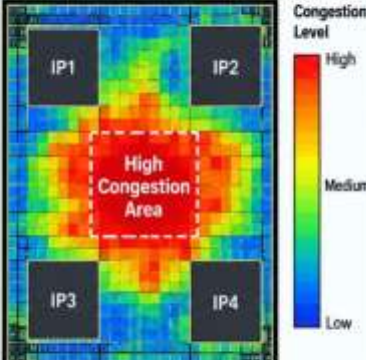
↑

Available Routing Resources

(Routing tracks, metal layers, routing capacity)

If the routing demand is higher than the available routing resources, the region becomes congested.

**Example: Routing Congestion Map**



**Effects of High Congestion**

- ✗
**Routing failures**  
 Some nets may not be routable within design rules.
- ⋈
**Longer wires**  
 Nets may take detours, increasing wire length.
- ⦿
**More vias**  
 Additional layer changes may be required.
- ⌚
**Timing degradation**  
 Longer wires and extra vias increase delay.
- ⚠
**Design-rule violations**  
 Congestion can lead to spacing, width, or via rule violations.

**To reduce congestion:**

- Use multi-layer routing
- Improve placement (spread cells)
- Use clock/route blockages wisely
- Optimize pin locations and floorplan

### 13. ROUTING CAPACITY

Routing capacity represents the available routing resources in a region.

Conceptually:

$$Congestion = \frac{Routing\ Demand}{Routing\ Capacity}$$

If:

$$Demand < Capacity$$

the region is generally routable.

If:

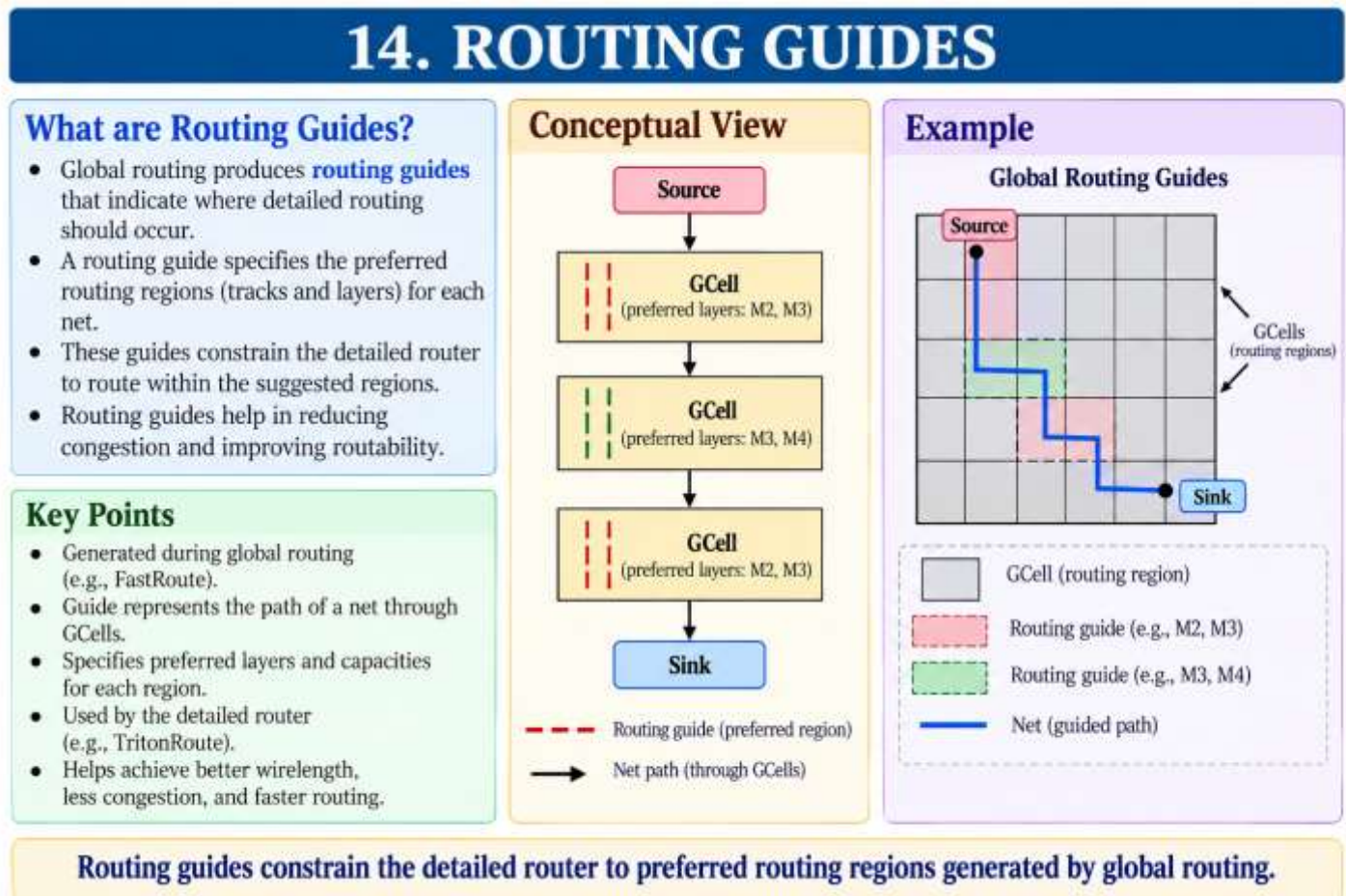
$$Demand > Capacity$$

the region is congested.

### 14. ROUTING GUIDES

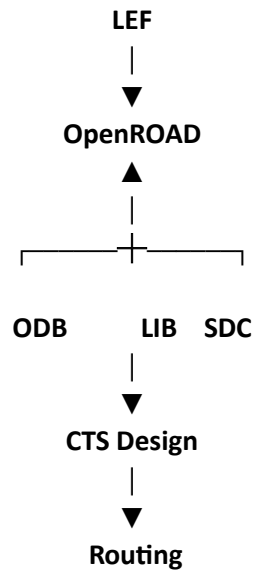
Global routing produces **routing guides** that indicate where detailed routing should occur.

Conceptually:



## 15. INPUTS FOR ROUTING

The routing stage typically requires:



Important information includes:

- Routed/unrouted nets
- Cell locations
- Pin locations
- Routing layers
- Routing blockages
- Technology design rules
- Clock network
- Timing constraints

## 17. STEP 2 – Check Routing Commands

Inside OpenROAD:

- `help global_route`

For detailed routing:

- `help detailed_route`

This is important because routing options can vary between OpenROAD versions.

## 18. STEP 3 – Create Experiment Directory

- `mkdir -p ~/openroad_exp11`
- `cd ~/openroad_exp11`

#### **19. STEP 4 – Load CTS Database**

Use the output generated from Experiment-10:

- read\_db cts.odt

Check placement:

- check\_placement

Report area:

- report\_design\_area

#### **20. STEP 5 – Inspect the Design Before Routing**

Open the GUI:

- gui::show

Inspect:

- Standard cells
- Macros
- Clock-tree buffers
- Signal pins
- Routing blockages
- Power network

At this stage, many signal nets are still unrouted.

#### **21. STEP 6 – Check Clock Information**

Run:

- report\_clocks

Analyze:

- Clock period
- Clock name
- Clock source
- Clock sinks

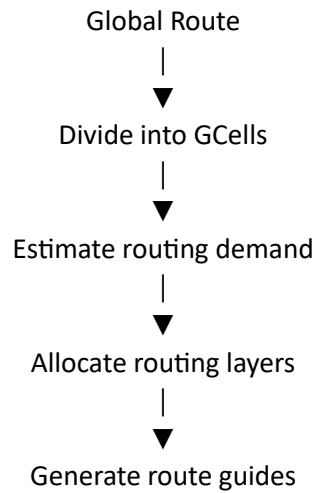
#### **22. STEP 7 – Perform Global Routing**

Run:

- global\_route

The global router determines routing paths through routing resources.

**Conceptually:**



**23. STEP 8 – Generate Routing Guides**

A representative routing flow can write guides using:

- `write_guides route.guide`

The resulting guide file describes the routing regions generated by global routing.

**24. STEP 9 – Analyze Routing Congestion**

After global routing, analyze congestion.

Depending on the OpenROAD version, use:

- `global_route -congestion_report_file congestion.rpt`

A representative command may therefore be:

`global_route \`

`-congestion_report_file congestion.rpt`

Inspect:

- `gvim congestion.rpt`

or:

- `cat congestion.rpt`

The exact options should be verified using:

- `help global_route`

## 25. STEP 10 – Analyze Global Routing

Important parameters include:

- Total wirelength
- Routing overflow
- Horizontal overflow
- Vertical overflow
- Routing demand
- Routing capacity

A desirable result is:

- Routing Overflow → Low / Zero

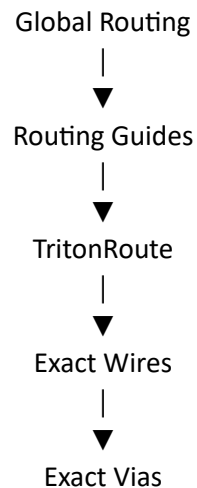
## 26. STEP 11 – Perform Detailed Routing

After global routing, detailed routing can be performed using:

- `detailed_route`

This invokes the current OpenROAD detailed-routing engine based on **TritonRoute**.

**Conceptually:**



## 27. STEP 12 – Check Routing

After detailed routing, inspect the routed design.

Open the GUI:

- `gui::show`

You should see:

Metal wires  
+  
Vias  
+  
Cell pins  
+  
Clock network

### **28. STEP 13 – Analyze Routing DRC**

After detailed routing, use the available routing/design-rule reporting facilities in your OpenROAD version.

For detailed-router options and reports:

- `help detailed_route`

The objective is to determine whether routing-related design-rule violations remain.

### **29. STEP 14 – Check Antenna Violations**

Antenna checking can be performed using:

- `check_antennas`

For more information:

- `help check_antennas`

Antenna violations occur when long metal structures connected to sensitive gate inputs accumulate charge during manufacturing processes.

### **30. STEP 15 – Analyze Post-Route Timing**

After routing, timing should be analyzed again.

For setup:

- `report_checks -path_delay max`

For hold:

- `report_checks -path_delay min`

Compare:

**Before Routing**

**VS**

**After Routing**

Routing introduces real interconnect effects that can change timing.

### 31. STEP 16 – Save the Routed Design

Save the database:

- `write_db routed.odb`

Save the DEF:

- `write_def routed.def`

### 32. COMPLETE REPRESENTATIVE ROUTING SCRIPT

Create:

- `gvim exp11_routing.tcl`

**Representative script:**

`read_db cts.odb`

`check_placement`

`report_design_area`

`report_clocks`

`global_route \`  
`-congestion_report_file congestion.rpt`

`write_guides route.guide`

`detailed_route`

`check_antennas`

`report_checks -path_delay max`

`report_checks -path_delay min`

`write_db routed.odb`

`write_def routed.def`

Run:

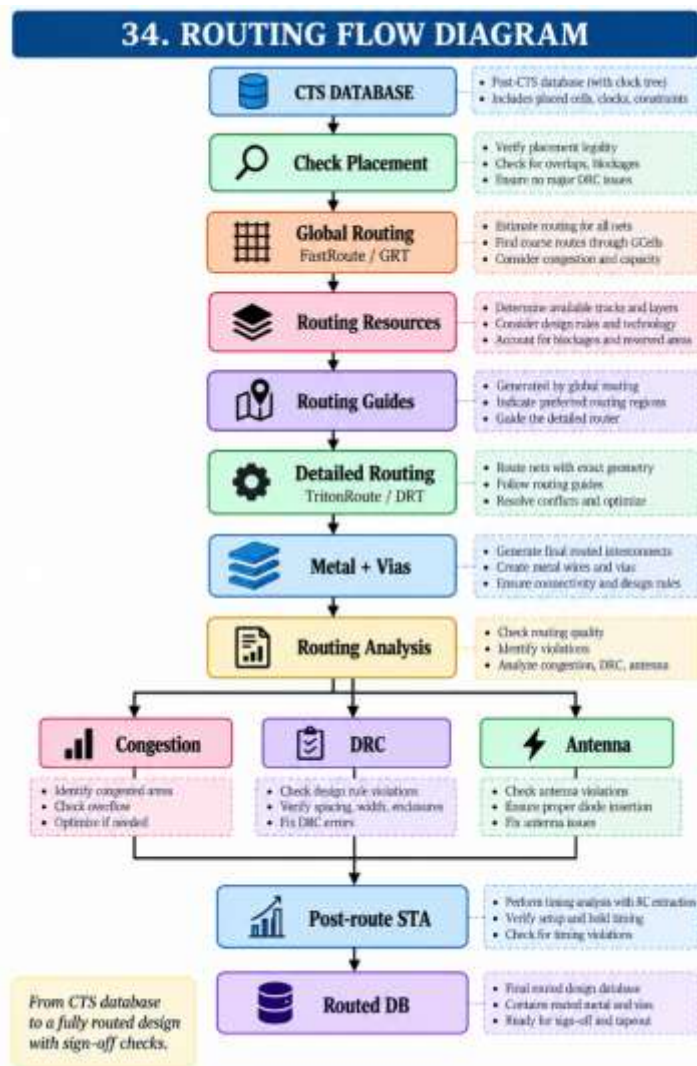
- `openroad exp11_routing.tcl`

**Note:** Routing command options are technology- and version-dependent. Use `help global_route` and `help detailed_route` in your installed OpenROAD version before executing a production flow.

### 33. GLOBAL ROUTING VS DETAILED ROUTING

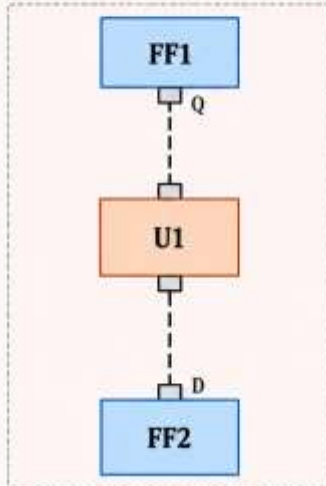
| Feature                 | Global Routing                    | Detailed Routing            |
|-------------------------|-----------------------------------|-----------------------------|
| Main purpose            | Determine routing paths/resources | Create exact physical wires |
| Current OpenROAD module | grt                               | drt                         |
| Current engine          | FastRoute-based                   | TritonRoute-based           |
| Command                 | global_route                      | detailed_route              |
| Exact wire geometry     | No                                | Yes                         |
| Exact vias              | Estimated/abstracted              | Yes                         |
| Routing guides          | Generated                         | Consumed                    |
| Design rules            | Approximate planning              | Detailed enforcement        |
| Output                  | Guides/routes                     | Final physical routing      |

### 34. ROUTING FLOW DIAGRAM



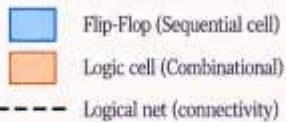
## 35. BEFORE ROUTING

### Logical Connectivity (No Physical Wires)



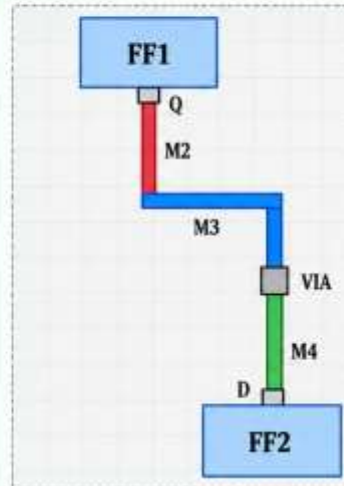
#### Key Points

- Cells (FF1, U1, FF2) are placed in the design.
- Logical connectivity (nets) exists between pins (e.g., FF1 → U1 → FF2).
- Physical signal wires are not finalized.
- Exact routing paths, metal layers, and vias are not determined.
- The design is ready for global routing to find possible paths.



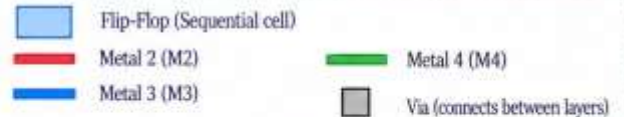
## 36. AFTER ROUTING

### Physical Interconnect (Routed Wires)



#### Key Points

- The logical net is converted into physical interconnect.
- Wires are routed using different metal layers (e.g., M2, M3, M4).
- Vias are inserted to connect between metal layers.
- The exact routing path is determined (geometry, layers, width, spacing, etc.).
- The net now has a complete physical implementation.
- Satisfies design rules (DRC) and electrical constraints.



**The logical net has now been converted into physical interconnect.**

## 37. ROUTING CONGESTION ANALYSIS

Suppose:

Routing demand = 90 tracks

Routing capacity = 100 tracks

Then:

$$\text{Congestion} = \frac{90}{100} = 0.9$$

or:

90%

If:

Demand = 120

Capacity = 100

then:

$$\text{Congestion} = 1.2$$

or:

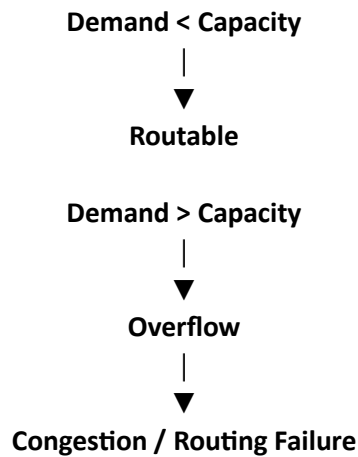
120%

This indicates an overflow condition.

### 38. ROUTING OVERFLOW

Routing overflow represents routing demand that cannot currently be accommodated by available routing capacity.

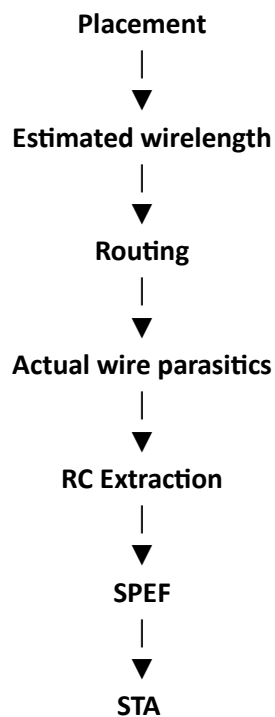
Conceptually:



The goal is generally to achieve **zero overflow** or an acceptable routing solution according to the design requirements.

### 39. ROUTING AND TIMING

Routing directly affects timing.



Longer wires generally increase:

- Resistance
- Capacitance
- Propagation delay

Therefore, routing is a critical part of timing closure.

#### 40. ROUTING AND POWER

Interconnect capacitance contributes to dynamic power:

$$P_{dynamic} = \alpha CV^2 f$$

Routing can increase capacitance due to:

- Long wires
- Large coupling capacitance
- Multiple vias
- Large metal structures

Therefore, routing optimization also contributes to power optimization.

#### 41. OBSERVATION TABLE

| S.No. | Parameter              | Observation |
|-------|------------------------|-------------|
| 1     | Design Name            |             |
| 2     | OpenROAD Version       |             |
| 3     | Number of Nets         |             |
| 4     | Routing Layers         |             |
| 5     | Global Routing Status  |             |
| 6     | Routing Overflow       |             |
| 7     | Horizontal Overflow    |             |
| 8     | Vertical Overflow      |             |
| 9     | Total Wirelength       |             |
| 10    | Number of Vias         |             |
| 11    | Routing DRC Violations |             |
| 12    | Antenna Violations     |             |
| 13    | Setup WNS              |             |
| 14    | Setup TNS              |             |
| 15    | Hold WNS               |             |
| 16    | Hold TNS               |             |
| 17    | Final Routing Status   | PASS / FAIL |

#### 42. ANALYSIS

#### 43. EXPECTED OUTPUT

#### 44. IMPORTANT OPENROAD COMMANDS

| Command             | Purpose                                              |
|---------------------|------------------------------------------------------|
| global_route        | Perform global routing                               |
| write_guides        | Write global-routing guides                          |
| detailed_route      | Perform detailed routing using TritonRoute-based DRT |
| check_antennas      | Check antenna violations                             |
| report_checks       | Perform timing analysis                              |
| check_placement     | Check placement legality                             |
| write_db            | Save ODB database                                    |
| write_def           | Save DEF                                             |
| gui::show           | Open design GUI                                      |
| help global_route   | Show global-routing options                          |
| help detailed_route | Show detailed-routing options                        |

#### **47. RESULT**

#### **48. CONCLUSION**

## EXPERIMENT-12

### Do IR Drop Analysis Using PDNSim

**Contact Hours:** 2 Hours

#### 1. AIM

To study and perform **IR drop analysis** using **PDNSim** in OpenROAD, analyze the quality of the power distribution network, identify voltage-drop hotspots, and understand the effects of power-grid resistance and current demand on chip reliability.

**Important technical note:** In current OpenROAD releases, the PDNSim functionality is provided through the **psm module**. The main analysis command is **analyze\_power\_grid**. The syllabus uses the name **PDNSim**, which is the name of the underlying power-grid analysis capability.

#### 2. OBJECTIVES

After completing this experiment, the student should be able to:

1. Understand the concept of **IR drop**.
2. Understand the purpose of **PDNSim**.
3. Understand the power distribution network (PDN).
4. Analyze:
  - Static IR drop
  - Dynamic IR drop concept
  - Voltage distribution
  - Current density
  - Power-grid resistance
5. Perform power-grid analysis using OpenROAD.
6. Identify maximum voltage-drop locations.
7. Analyze worst-case IR drop.
8. Understand the relationship between:
  - Resistance
  - Current
  - Voltage drop
9. Understand the impact of IR drop on timing and circuit reliability.
10. Generate and interpret an IR-drop report

#### 3. SOFTWARE / TOOLS REQUIRED

| S.No. | Software / File | Purpose                         |
|-------|-----------------|---------------------------------|
| 1     | Linux / Ubuntu  | Operating environment           |
| 2     | OpenROAD        | Physical design implementation  |
| 3     | PDNSim          | Power-grid analysis             |
| 4     | OpenDB          | Design database                 |
| 5     | LEF             | Technology/physical information |

| S.No. | Software / File           | Purpose                     |
|-------|---------------------------|-----------------------------|
| 6     | Liberty .lib              | Cell information            |
| 7     | DEF                       | Physical design information |
| 8     | ODB                       | OpenROAD database           |
| 9     | PDN configuration         | Power-grid definition       |
| 10    | Power/ground connectivity | Supply-network analysis     |

#### 4. THEORY

##### 4.1 What is IR Drop?

**IR drop** is the voltage drop caused by current flowing through the resistance of the power distribution network. It is represented by:

$$V_{drop} = I \times R$$

where:

- $V_{drop}$  = voltage drop
- $I$  = current flowing through the power network
- $R$  = resistance of the power network

For example:

Current = 0.1 A

Resistance = 0.5  $\Omega$

$$\begin{aligned} V_{drop} &= I \times R \\ &= 0.1 \times 0.5 \\ &= 0.05 \text{ V} \end{aligned}$$

Therefore:

IR Drop = 50 mV

#### 5. WHY IR DROP ANALYSIS IS REQUIRED

A standard cell may require a supply voltage such as:

VDD = 1.0 V

Due to resistance in the power network, the cell may receive:

VDD at source = 1.0 V

VDD at cell = 0.92 V

Therefore:

$$\begin{aligned} V_{drop} &= 1.0 - 0.92 \\ V_{drop} &= 0.08 \text{ V} \end{aligned}$$

or:

80 mV

Excessive IR drop can cause:

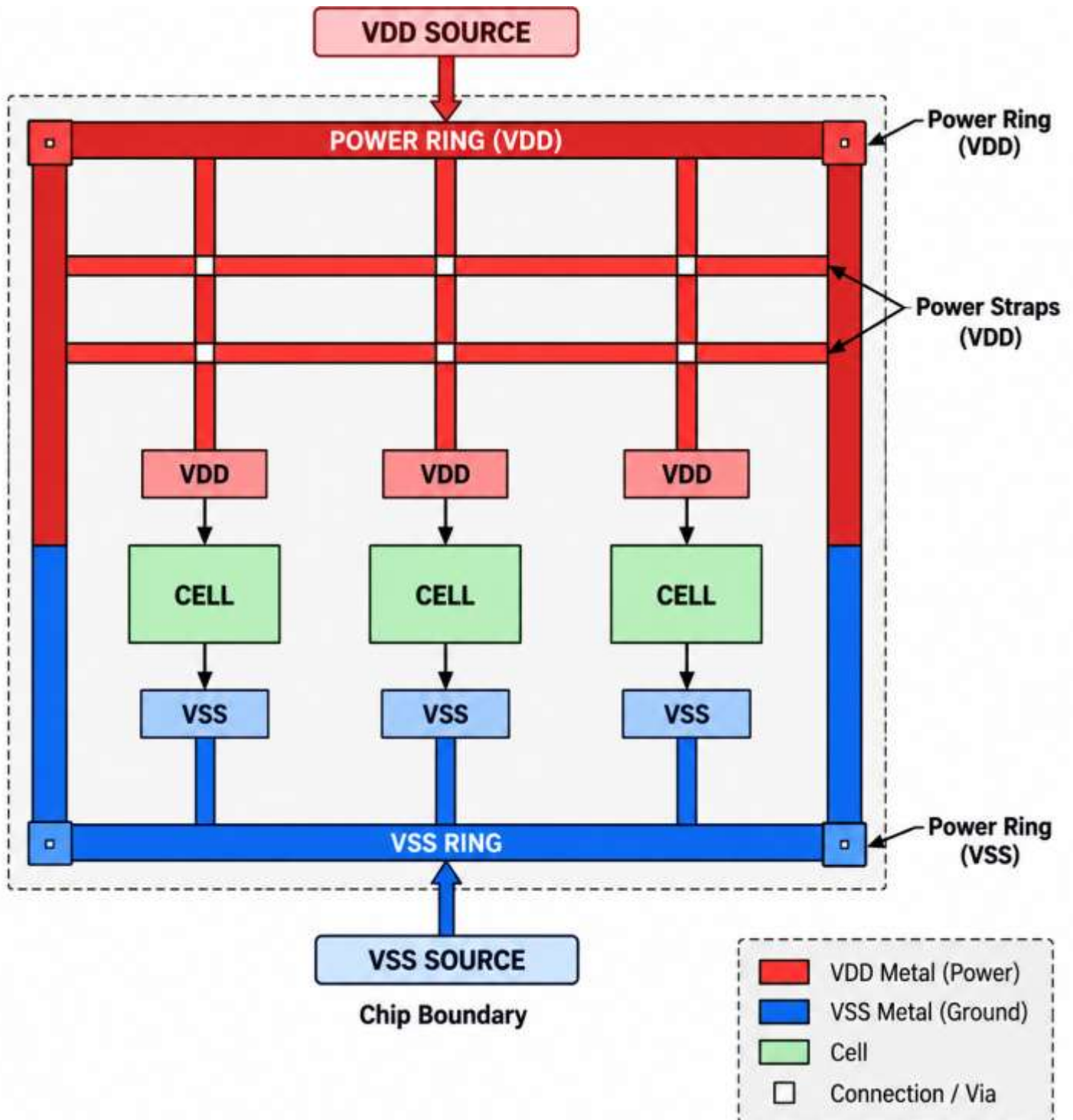
- Increased cell delay
- Setup timing violations

- Incorrect circuit operation
- Reduced noise margin
- Electromigration concerns
- Reliability problems

## 6. POWER DISTRIBUTION NETWORK

The **Power Distribution Network (PDN)** distributes power from the power source to standard cells and macros.

A simplified PDN is:



The PDN typically consists of:

- Power rings
- Power straps
- Standard-cell power rails
- Vias
- Power/ground connections

## 7. VDD AND VSS

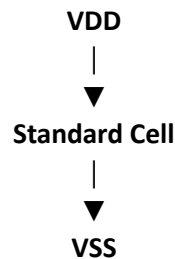
### VDD

VDD is the positive power supply.

### VSS

VSS is the ground/reference supply.

Typical connection:

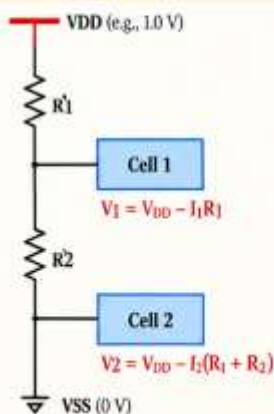


The PDN must provide low-resistance paths for both power and ground.

## 8. STATIC IR DROP

**Static IR drop** is the voltage drop in the power distribution network under relatively steady-state (DC) current assumptions.

### Conceptual View



### Key Points

- Based on relatively steady-state current assumptions.
- Different cells can experience different voltage drops.
- Mainly depends on:
  - Distance from power source
  - Power-grid resistance
  - Local current demand
  - PDN structure (rings, straps, mesh, etc.).
- Analyzed to ensure all cells receive sufficient voltage (e.g., within IR drop limit).

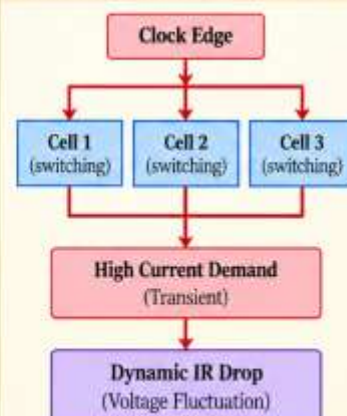
### Voltage Drop Example

- If Cell 1 draws current  $I_1$ , its voltage is:  $V_1 = V_{DD} - I_1 R_1$
- If Cell 2 draws current  $I_2$ , its voltage is:  $V_2 = V_{DD} - I_2 (R_1 + R_2)$
- Cells farther from the source or with higher current demand experience larger IR drop.

## 9. DYNAMIC IR DROP

**Dynamic IR drop** is the voltage drop in the power distribution network due to time-varying (transient) switching activity.

### Simultaneous Switching Example



### Key Points

- Associated with time-varying switching activity.
- Occurs when many cells switch simultaneously (e.g., at clock edges).
- Can cause temporary voltage droop ( $\Delta V$ ) in the power network.
- Mainly affected by:
  - Switching activity
  - Clock activity
  - Simultaneous switching
  - Package effects
  - PDN impedance (R, L, C)
- Can lead to timing failures, functional errors, or increased power noise if not within limits.

### Analysis and Tools

- PDNSim is primarily used for power-grid analysis based on the design's physical power network and current-demand modeling.
- Dynamic IR-drop sign-off may require additional activity-aware analysis tools and methodologies (e.g., using switching activity, vector-based analysis).
- Accurate modeling of PDN, current profiles, and package effects is important.

**IR drop analysis (static and dynamic) ensures reliable power delivery to all cells in the design.**

## 8. STATIC IR DROP

**Static IR drop** is analyzed using relatively steady-state current assumptions.

Conceptually:

Different cells can experience different voltage drops because of:

- Distance from power source
- Power-grid resistance
- Local current demand
- PDN structure

## 9. DYNAMIC IR DROP

**Dynamic IR drop** is associated with time-varying switching activity.

Dynamic IR drop can be affected by:

- Switching activity
- Clock activity
- Simultaneous switching
- Package effects
- PDN impedance

PDNSim is primarily used for power-grid analysis based on the design's physical power network and current-demand modeling; dynamic IR-drop sign-off may require additional activity-aware analysis tools and methodologies.

## 10. EFFECT OF RESISTANCE

From:

$$V = IR$$

If resistance increases:

$$R \uparrow \Rightarrow V_{drop} \uparrow$$

Therefore, a weak power grid can produce significant voltage drop.

Example:

Case 1:

$$I = 0.1 \text{ A}$$

$$R = 0.1 \Omega$$

$$V_{drop} = 10 \text{ mV}$$

Case 2:

$$I = 0.1 \text{ A}$$

$$R = 0.5 \Omega$$

$$V_{drop} = 50 \text{ mV}$$

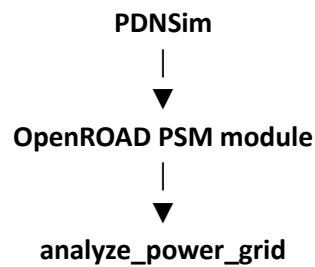
Therefore, reducing PDN resistance helps reduce IR drop.

- High cell density
- High current demand
- Long power paths
- Insufficient straps
- Insufficient vias
- Weak power-ring connections
- Poor PDN topology

### 13. PDNSIM

PDNSim is OpenROAD's power-grid analysis capability used to analyze voltage drops and related power-grid characteristics.

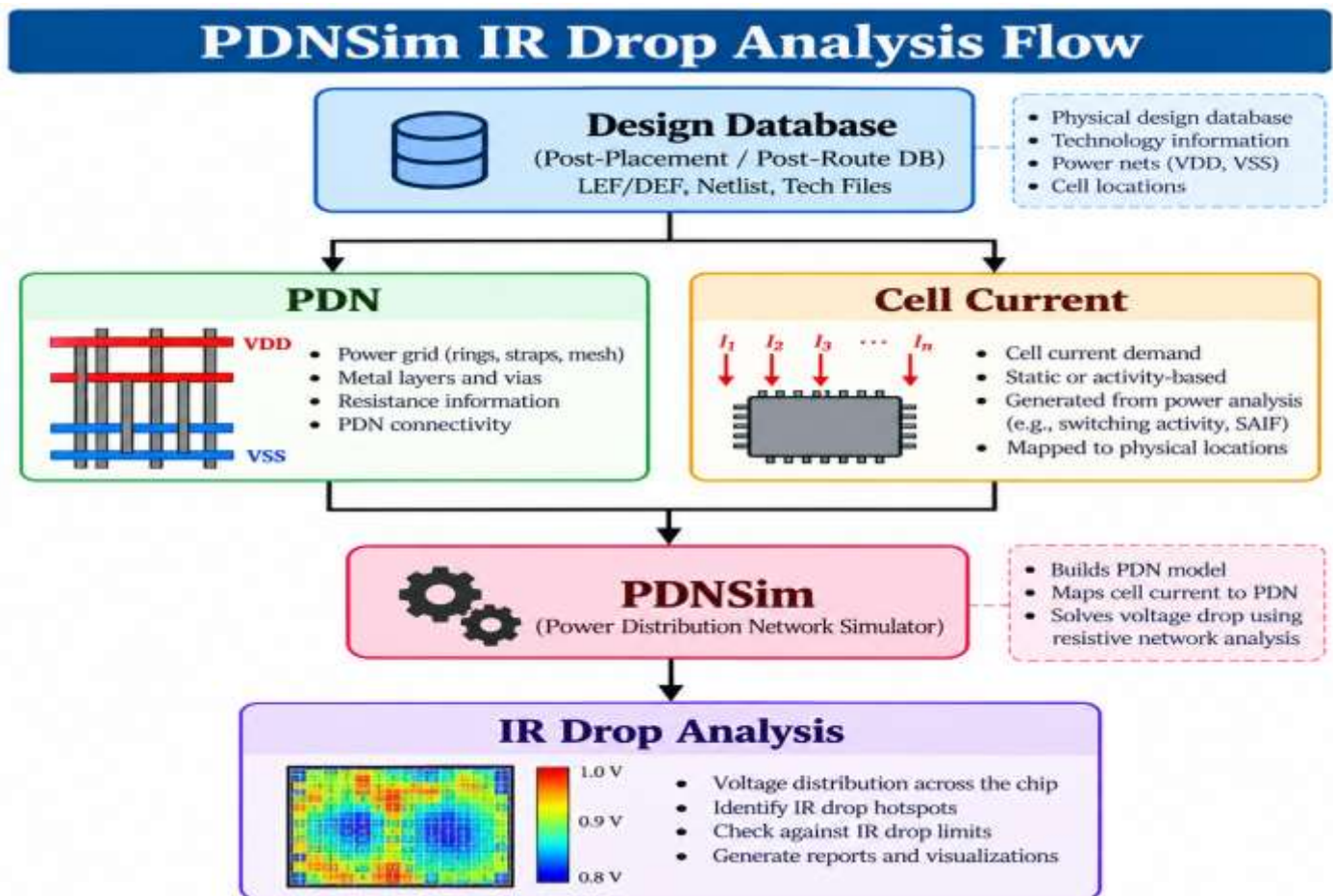
In current OpenROAD:



The analysis uses the physical power network represented in the OpenROAD database.

### 14. PDNSIM INPUTS

A typical PDNSim analysis requires:



The database should contain:

- Power network
- Ground network
- Standard-cell locations
- Power/ground connectivity

- PDN geometry
- Relevant physical information

### 15. ROLE OF PDNGEN AND PDNSIM

PDNGEN and PDNSim have different purposes.

#### PDNGEN

Creates the power distribution network.

PDNGEN  
↓  
Power Rings  
Power Straps  
Power Rails  
Vias

#### PDNSim

Analyzes the generated power network.

PDN  
↓  
PDNSim  
↓  
Voltage Drop  
Current  
Resistance  
IR Hotspots

Therefore:

PDNGEN  
↓  
Generate Power Grid  
↓  
Physical PDN  
↓  
PDNSim  
↓  
IR Drop Analysis

## 16. PROCEDURE

### Step 1 – Check OpenROAD Installation

Run:

- openroad -version

Start OpenROAD:

- openroad

Check the version:

- version

### 17. STEP 2 – Check PDNSim Commands

Inside OpenROAD:

- help analyze\_power\_grid

You can also inspect the module:

- help

The exact options should be checked against the installed OpenROAD version.

### 18. STEP 3 – Create Experiment Directory

- mkdir -p ~/openroad\_exp12
- cd ~/openroad\_exp12

### 19. STEP 4 – Load the Design Database

For a routed design, load:

- read\_db routed.odb

If your experiment is intended to analyze the PDN before routing, use the appropriate PDN database instead:

- read\_db pdn.odb

For a post-routing analysis, the routed database is generally more representative.

### 20. STEP 5 – Check the Design

Run:

- report\_design\_area

Then:

- check\_placement

Make sure the physical design is loaded correctly.

**21. STEP 6 – Inspect POWER/GROUND NETWORK**

The design should contain the appropriate power and ground connections.

Typical power nets are:

- VDD
- VSS

Check the actual power-net names in your design because they may be technology/design dependent.

**22. STEP 7 – Run Basic Power-Grid Analysis**

Run:

- `analyze_power_grid`

PDNSim analyzes the power network and estimates voltage drops based on the modeled power-grid network and cell-current demands.

**23. STEP 8 – Generate a Report**

A representative analysis can specify an output file:

- `analyze_power_grid -outfile ir_drop.rpt`

Check the supported options first:

- `help analyze_power_grid`

Then inspect:

- `gvim ir_drop.rpt`

or:

- `cat ir_drop.rpt`

**24. STEP 9 – Analyze Worst IR Drop**

The report should be examined for quantities such as:

- Worst voltage drop
- Worst IR drop
- Power-grid current
- Voltage at nodes
- Power-grid connectivity

Record the worst-case result.

Example:

Supply Voltage = 1.00 V

Worst Voltage = 0.94 V

Maximum IR Drop = 0.06 V

Therefore:

$$IR\ Drop = 1.00 - 0.94$$

$$IR\ Drop = 0.06V$$

or:

60 mV

## 25. STEP 10 – Inspect IR Drop in the GUI

Launch:

- `gui::show`

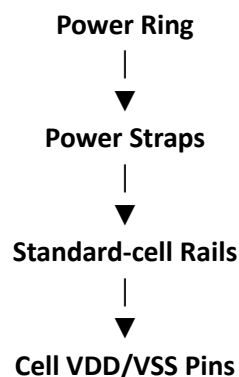
Inspect the physical design for power-grid structures and IR-drop-related information available in the GUI/version being used.

Look for:

- Power rings
- Power straps
- Standard-cell rows
- High-current regions
- IR-drop hotspots

## 26. STEP 11 – Analyze Power-Grid Connectivity

A reliable power grid requires good connectivity between:

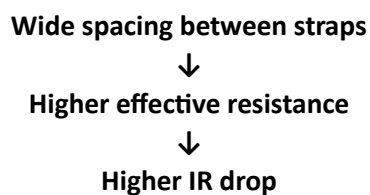


Poor connectivity can increase resistance and produce larger voltage drops.

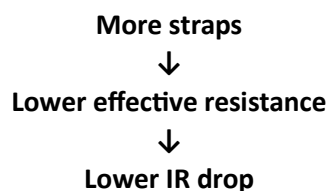
## 27. STEP 12 – Compare Different PDN Configurations

For educational analysis, modify the PDN and compare the results.

### Case 1 – Lower Strap Density



### Case 2 – Higher Strap Density



This demonstrates the relationship between PDN design and voltage stability

**28. STEP 13 – Analyze Power Ring Effect**

A stronger power ring can provide better current distribution.

Actual improvement depends on the complete PDN topology and current distribution.

**29. STEP 14 – Save the Analysis Database**

The IR-drop analysis itself does not necessarily require creating a new physical database, but after completing the experiment you may save the analyzed design state:

- `write_db ir_drop_analysis.odb`

**30. COMPLETE REPRESENTATIVE SCRIPT**

Create:

- `gvim exp12_ir_drop.tcl`

Example:

```
read_db routed.odb
```

```
report_design_area
```

```
check_placement
```

```
analyze_power_grid -outfile ir_drop.rpt
```

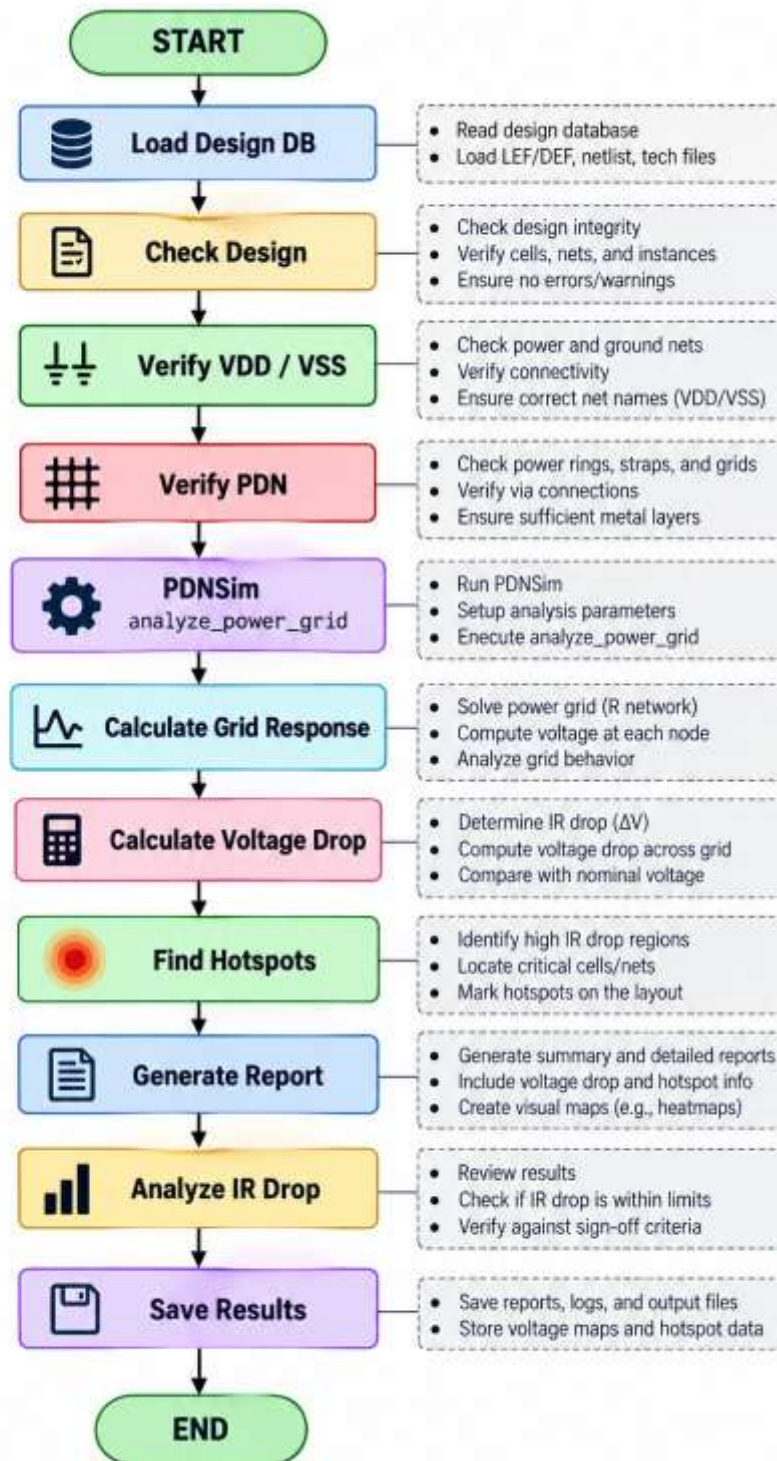
```
write_db ir_drop_analysis.odb
```

Run:

- `openroad exp12_ir_drop.tcl`

**Note:** The exact `analyze_power_grid` options depend on the OpenROAD version and design setup. Always check:  
`help analyze_power_grid`

### 31. IR DROP ANALYSIS FLOW



### 32. IR DROP CALCULATION EXAMPLE

Consider:

Supply Voltage = 1.0 V

Current = 0.2 A

Power-grid resistance = 0.25  $\Omega$

Using:

$$V_{drop} = IR$$
$$V_{drop} = 0.2 \times 0.25$$
$$V_{drop} = 0.05V$$

Therefore:

IR Drop = 50 mV

Voltage delivered to the load:

$$V_{load} = 1.0 - 0.05$$
$$V_{load} = 0.95V$$

### 34. EFFECT OF IR DROP ON CELL DELAY

Suppose a standard cell is characterized for:

- VDD = 1.0 V

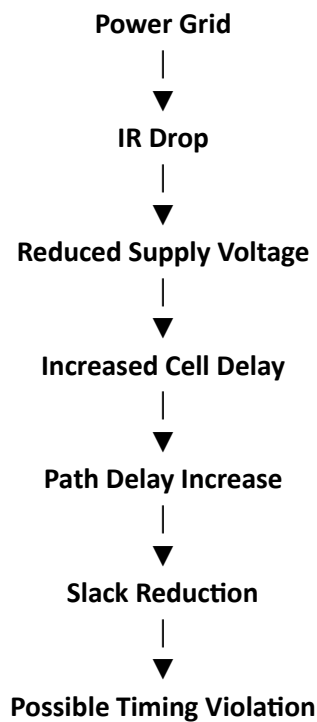
But due to IR drop it receives:

- VDD = 0.90 V

The reduced supply voltage can cause the cell to switch more slowly.

### 35. IR DROP AND TIMING

IR drop can affect timing closure.



Therefore:

Good PDN → Low IR Drop → Better Timing Robustness

### 36. IR DROP AND ELECTROMIGRATION

IR drop and **electromigration (EM)** are related to current flow through the power network, but they are different failure mechanisms.

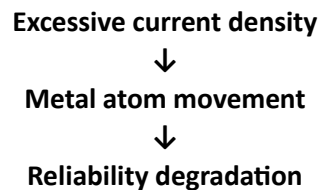
#### IR Drop

Concern:

- Voltage loss

#### Electromigration

Concern:



Therefore, both IR drop and EM should be considered during power-integrity analysis.

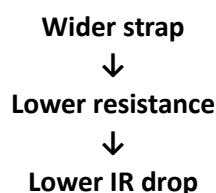
### 37. STATIC VS DYNAMIC IR DROP

| Parameter       | Static IR Drop                | Dynamic IR Drop                |
|-----------------|-------------------------------|--------------------------------|
| Current         | Relatively steady             | Time-varying                   |
| Main cause      | Average/steady current        | Switching activity             |
| Analysis        | DC/resistive network behavior | Activity-dependent behavior    |
| Hotspots        | High-current regions          | Simultaneous switching regions |
| Impact          | Voltage degradation           | Transient voltage droop        |
| Typical concern | Power-grid strength           | Switching-induced supply noise |

### 38. METHODS TO REDUCE IR DROP

If IR drop is too high, possible PDN improvements include:

#### 1. Increase Power-Strap Width



#### 2. Reduce Strap Pitch

More straps can provide additional current paths.

#### 3. Add Additional Metal Layers

Additional high-level metal resources can improve power distribution.

#### 4. Increase Via Density

More parallel vias can reduce resistance at layer transitions.

#### 5. Improve Power-Ring Design

A stronger ring can distribute current more effectively.

#### 6. Reduce Local Current Demand

Power optimization can reduce current demand and therefore reduce IR drop.

### 39. OBSERVATION TABLE

| S.No. | Parameter          | Observation |
|-------|--------------------|-------------|
| 1     | Design Name        |             |
| 2     | OpenROAD Version   |             |
| 3     | Power Net          |             |
| 4     | Ground Net         |             |
| 5     | Supply Voltage     |             |
| 6     | PDN Type           |             |
| 7     | Power Ring         |             |
| 8     | Power Straps       |             |
| 9     | Maximum Current    |             |
| 10    | Worst Voltage      |             |
| 11    | Maximum IR Drop    |             |
| 12    | IR Drop Percentage |             |
| 13    | Hotspot Location   |             |
| 14    | Analysis Status    | PASS / FAIL |

### 40. IR DROP PERCENTAGE

IR drop can be expressed as a percentage of the supply voltage:

$$IR\ Drop\% = \frac{V_{drop}}{V_{DD}} \times 100$$

Example:

VDD = 1.0 V

Vdrop = 0.05 V

Then:

$$IR\ Drop\% = \frac{0.05}{1.0} \times 100$$

$$= 5\%$$

Therefore:

IR Drop = 5%

Whether this is acceptable depends on the technology and design sign-off requirements.

#### 41. ANALYSIS

#### 42. EXPECTED OUTPUT

#### 43. IMPORTANT OPENROAD COMMANDS

| Command                 | Purpose                            |
|-------------------------|------------------------------------|
| read_db                 | Read OpenROAD database             |
| report_design_area      | Report design area                 |
| check_placement         | Check placement                    |
| analyze_power_grid      | Perform PDNSim power-grid analysis |
| write_db                | Save OpenROAD database             |
| gui::show               | Open GUI                           |
| help analyze_power_grid | Display PDNSim command options     |

#### 44. PDNGEN VS PDNSIM

| Feature                     | PDNGEN                  | PDNSim                   |
|-----------------------------|-------------------------|--------------------------|
| Main purpose                | Generate power grid     | Analyze power grid       |
| Operation                   | Physical implementation | Power-integrity analysis |
| Creates rings               | Yes                     | No                       |
| Creates straps              | Yes                     | No                       |
| Adds PDN connections        | Yes                     | Analyzes them            |
| Calculates IR drop          | No                      | Yes                      |
| Identifies voltage hotspots | No                      | Yes                      |
| Current OpenROAD module     | pdn                     | psm                      |

#### 47. RESULT

#### 48. CONCLUSION

### EXPERIMENT–13

#### Perform Parasitic Extraction Analysis Using OpenRCX

**Contact Hours:** 2 Hours

#### 1. AIM

To study and perform **parasitic extraction** using **OpenRCX** in OpenROAD, extract the resistance and capacitance of routed interconnects, generate a **SPEF (Standard Parasitic Exchange Format)** file, and analyze the effect of extracted parasitics on post-route timing.

#### 2. OBJECTIVES

After completing this experiment, the student should be able to:

1. Understand the concept of parasitic extraction.
2. Understand the need for post-route RC extraction.
3. Study the working principle of OpenRCX.
4. Understand:
  - Resistance
  - Capacitance
  - Coupling capacitance
  - RC networks
5. Generate extraction rules for a technology.
6. Load a routed design into OpenROAD.
7. Perform parasitic extraction using OpenRCX.
8. Generate a SPEF file.
9. Read extracted parasitics into OpenSTA/OpenROAD.
10. Compare pre-route and post-route timing.
11. Understand how interconnect parasitics affect:
  - Delay
  - Slew
  - Setup timing
  - Hold timing
  - Power

#### 3. SOFTWARE / TOOLS REQUIRED

| S.No. | Software / File | Purpose                         |
|-------|-----------------|---------------------------------|
| 1     | Linux / Ubuntu  | Operating environment           |
| 2     | OpenROAD        | Physical design environment     |
| 3     | OpenRCX         | Parasitic extraction            |
| 4     | OpenSTA         | Post-route timing analysis      |
| 5     | OpenDB          | Design database                 |
| 6     | LEF             | Physical technology information |
| 7     | Liberty .lib    | Cell timing information         |

| S.No. | Software / File      | Purpose                           |
|-------|----------------------|-----------------------------------|
| 8     | DEF / ODB            | Routed physical design            |
| 9     | RCX extraction rules | Resistance/capacitance extraction |
| 10    | SPEF                 | Extracted parasitic information   |

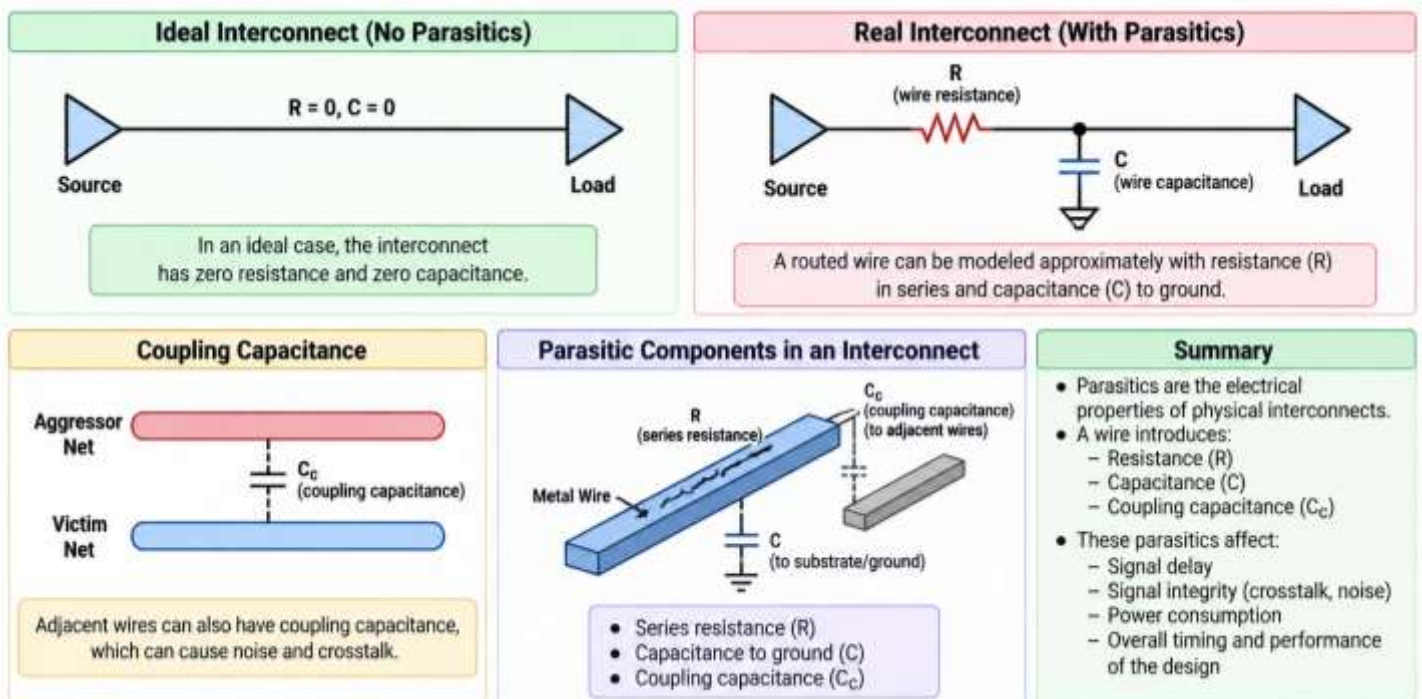
## 4. THEORY

### 4.1 What are Parasitics?

In an ideal digital circuit, a wire would have zero resistance and zero capacitance.

In a real ASIC, every physical interconnect has electrical properties.

A routed wire can be modeled approximately as:



The wire therefore introduces:

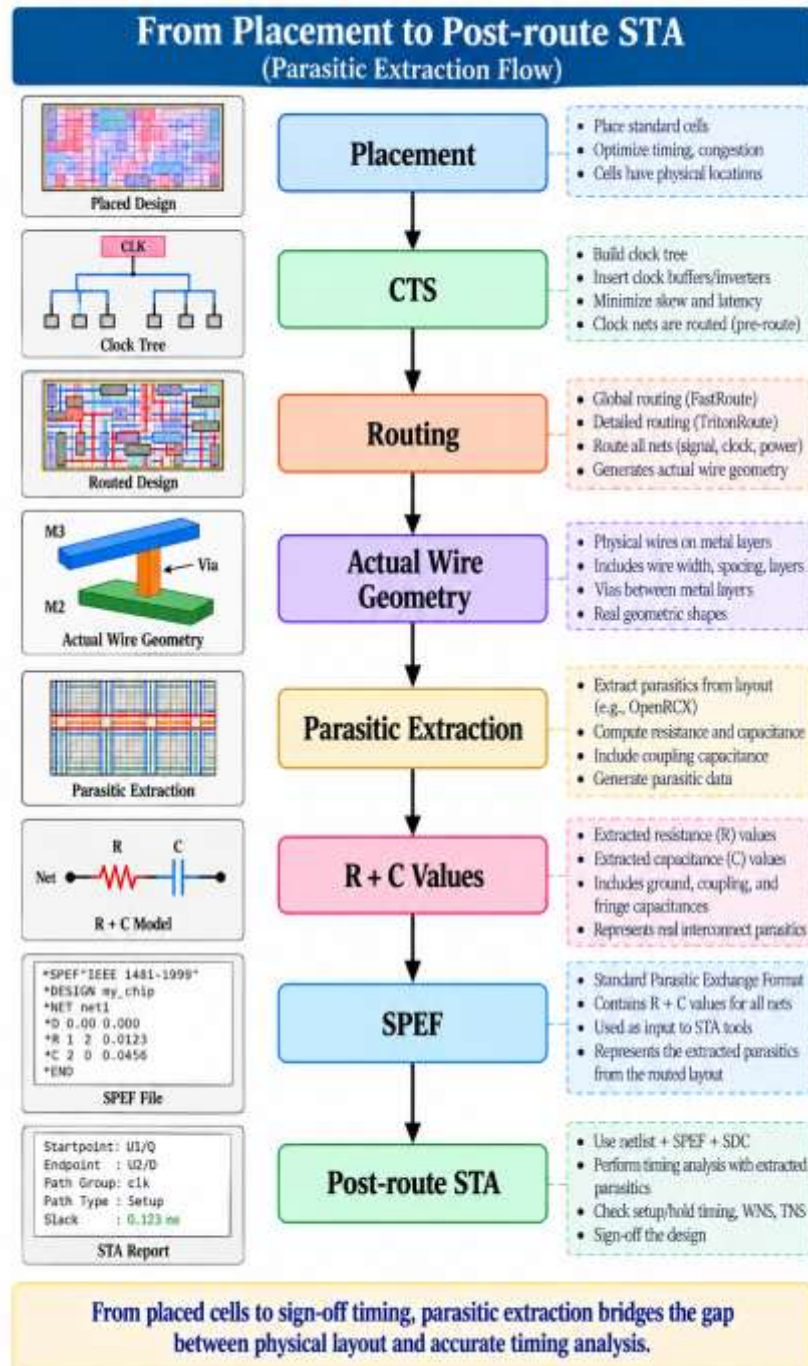
- Resistance  $R$
- Capacitance  $C$
- Coupling capacitance

These are called **parasitics**.

## 5. WHY PARASITIC EXTRACTION IS REQUIRED

Before routing, wire characteristics are estimated.

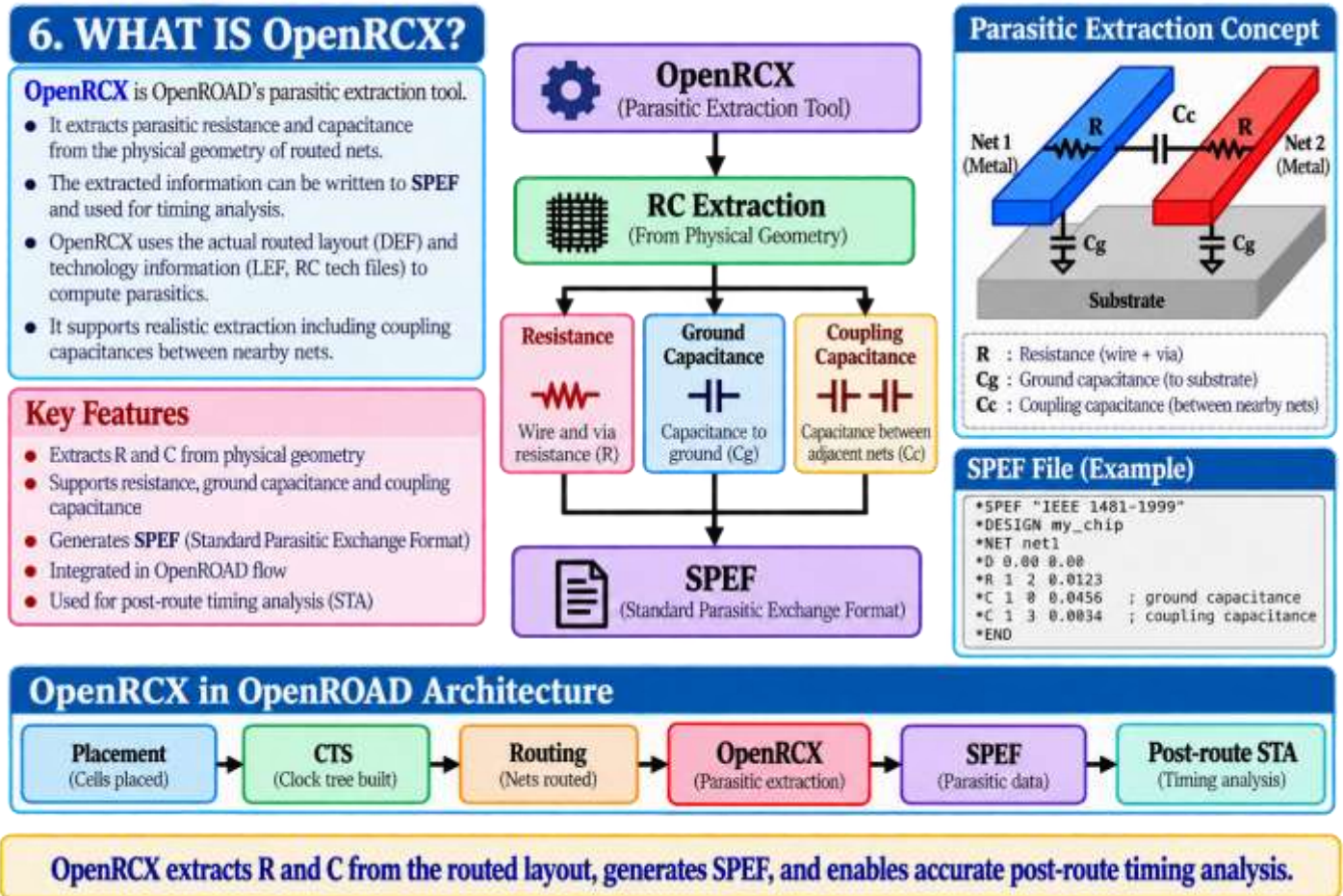
After routing, the actual physical geometry is known.



Post-route parasitics provide a more realistic representation of the physical interconnect.

## 6. WHAT IS OpenRCX?

In the current OpenROAD architecture:



The current OpenROAD extraction module is **rcx**.

## 7. WHAT IS RC EXTRACTION?

RC extraction determines the electrical characteristics of physical interconnects.

For each routed net, extraction can determine:

**Wire length**

- |
- └─ **Resistance**
- |
- └─ **Ground capacitance**
- |
- └─ **Coupling capacitance**

These values are required for accurate timing analysis.

## 8. RESISTANCE

Resistance opposes current flow.

The resistance of a uniform metal wire can be approximated as:

$$R = \rho \frac{L}{A}$$

where:

- $R$ = resistance
- $\rho$ = resistivity
- $L$ = wire length
- $A$ = cross-sectional area

Therefore:

Wire length ↑



Resistance ↑

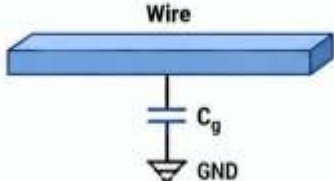
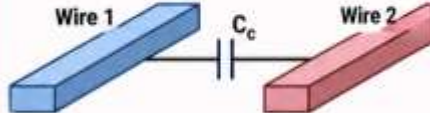
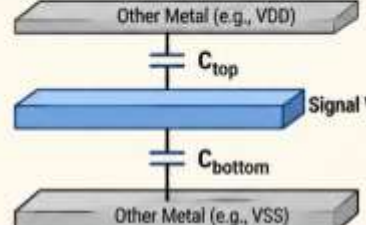
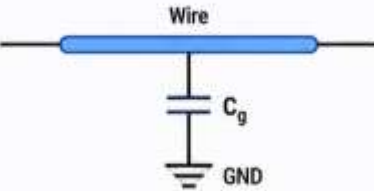
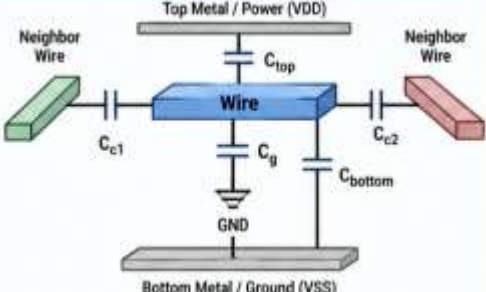
and:

Wire width ↑



Resistance ↓

## 9. CAPACITANCE

|                                                                                                                                                                                                                                                                                       |                                                                                                                                                                                                                                                                                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <h3>1. Capacitance to Ground (<math>C_g</math>)</h3> <p>A wire has capacitance to the substrate/ground through the dielectric.</p>  <p>Due to the dielectric between the wire and the substrate.</p> | <h3>2. Coupling Capacitance (<math>C_c</math>)</h3> <p>A wire has capacitance to neighboring wires due to the electric field between them.</p>  <p>Causes crosstalk and noise between adjacent wires.</p> | <h3>3. Capacitance to Other Conductors</h3> <p>A wire can also have capacitance to other conductors such as power/ground rails or nearby metal layers (above/below).</p>                                                                                                                                                                                                                                                                                                                                                                                                             |
| <h3>Simplified Representation</h3>                                                                                                                                                                   | <h3>Complete Capacitance Model (Simplified)</h3>                                                                                                                                                         | <h3>Key Points</h3> <ul style="list-style-type: none"> <li>A wire has capacitance to:             <ul style="list-style-type: none"> <li>Ground (<math>C_g</math>)</li> <li>Neighboring wires (<math>C_c</math>)</li> <li>Other conductors (e.g., power/ground)</li> </ul> </li> <li>The total capacitance is the sum of all these components.</li> <li>This capacitance contributes to the electrical load seen by the driving cell.</li> <li>Higher capacitance results in:             <ul style="list-style-type: none"> <li>Increased delay</li> <li>Higher power consumption</li> <li>Potential signal integrity issues (crosstalk, noise)</li> </ul> </li> </ul> |

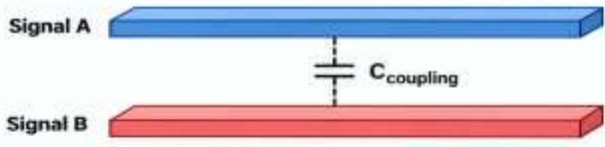
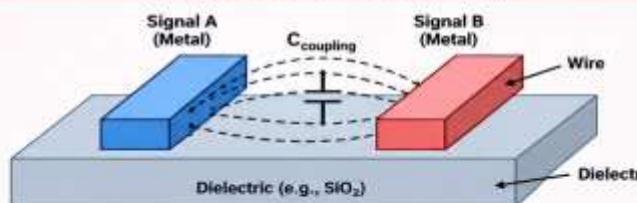
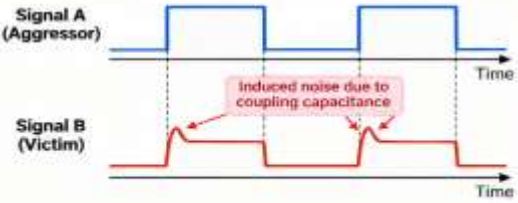
A wire has capacitance to:

- Ground
- Neighboring wires
- Other conductors

The capacitance contributes to the electrical load seen by the driving cell.

## 10. COUPLING CAPACITANCE

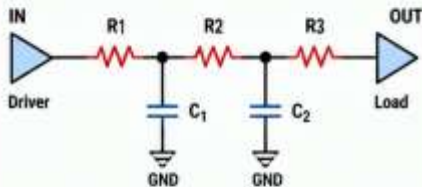
Two neighboring wires can have **coupling capacitance**. When one signal changes, it can influence the neighboring signal through coupling capacitance. This can contribute to crosstalk, delay variation, noise, and timing uncertainty.

|                                                                                                                                                                                                                                                      |                                                                                                                                                                                                                                                                                                                                                                                                |                                                                                                                                                                                                                                                                                                                                                                                                                           |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <h3>1. Conceptual Representation</h3>  <p>Two neighboring wires have coupling capacitance (<math>C_{coupling}</math>) due to the electric field between them.</p> | <h3>2. Physical View (Cross-section)</h3>                                                                                                                                                                                                                                                                  |                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <h3>3. Signal Influence (Waveform View)</h3>                                                                                                                      | <h3>4. Effects of Coupling Capacitance</h3> <ul style="list-style-type: none"> <li><b>Crosstalk</b>: Unwanted signal is induced on neighboring nets.</li> <li><b>Delay variation</b>: Can cause variation in signal delay.</li> <li><b>Noise</b>: Induces voltage noise on the victim net.</li> <li><b>Timing uncertainty</b>: Can lead to setup/hold violations in critical paths.</li> </ul> | <h3>5. Key Takeaways</h3> <ul style="list-style-type: none"> <li>• <b>Coupling capacitance</b> exists between neighboring wires.</li> <li>• A signal transition on one wire can influence the adjacent wire.</li> <li>• It can cause <b>crosstalk, delay variation, noise, and timing uncertainty</b>.</li> <li>• Proper spacing, shielding, and routing techniques are used to minimize coupling capacitance.</li> </ul> |

## 11. RC NETWORK

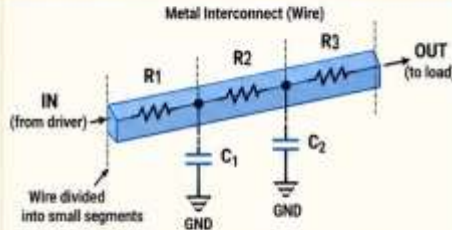
- A real interconnect is usually represented as a **distributed RC network**.
- OpenRCX** extracts such parasitic information from physical routing geometry.

### 1. Conceptual RC Network



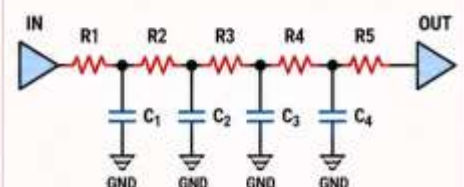
A long interconnect is modeled as a series of resistances (R) and capacitances (C) distributed along the wire.

### 2. Physical Interpretation



Each small segment of the wire has resistance, and each node has capacitance to ground (and also coupling capacitance to neighboring wires).

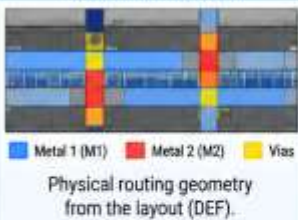
### 3. RC Network with More Segments



In practice, the interconnect is divided into many small segments to accurately model the distributed parasitics.

### 4. Parasitic Extraction Using OpenRCX

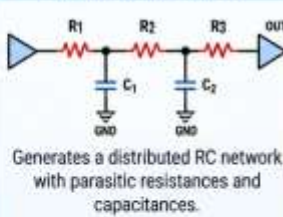
#### 1. Routed Layout (DEF)



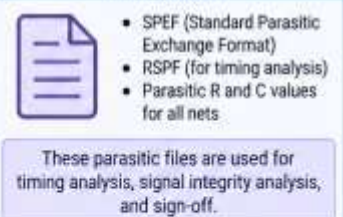
#### 2. OpenRCX Extraction



#### 3. Extracted RC Network



#### 4. Output Files



OpenRCX extracts accurate parasitic information (R, C, and coupling capacitance) from physical routing geometry, which is used for timing, SI, and sign-off analysis.

## 12. EFFECT OF WIRE LENGTH

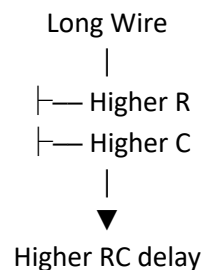
As wire length increases:

$$R \propto L$$

and approximately:

$$C \propto L$$

Therefore:



## 13. RC DELAY

A simplified RC delay relationship is:

$$t_d \approx RC$$

For example:

$$R = 100 \, \Omega$$

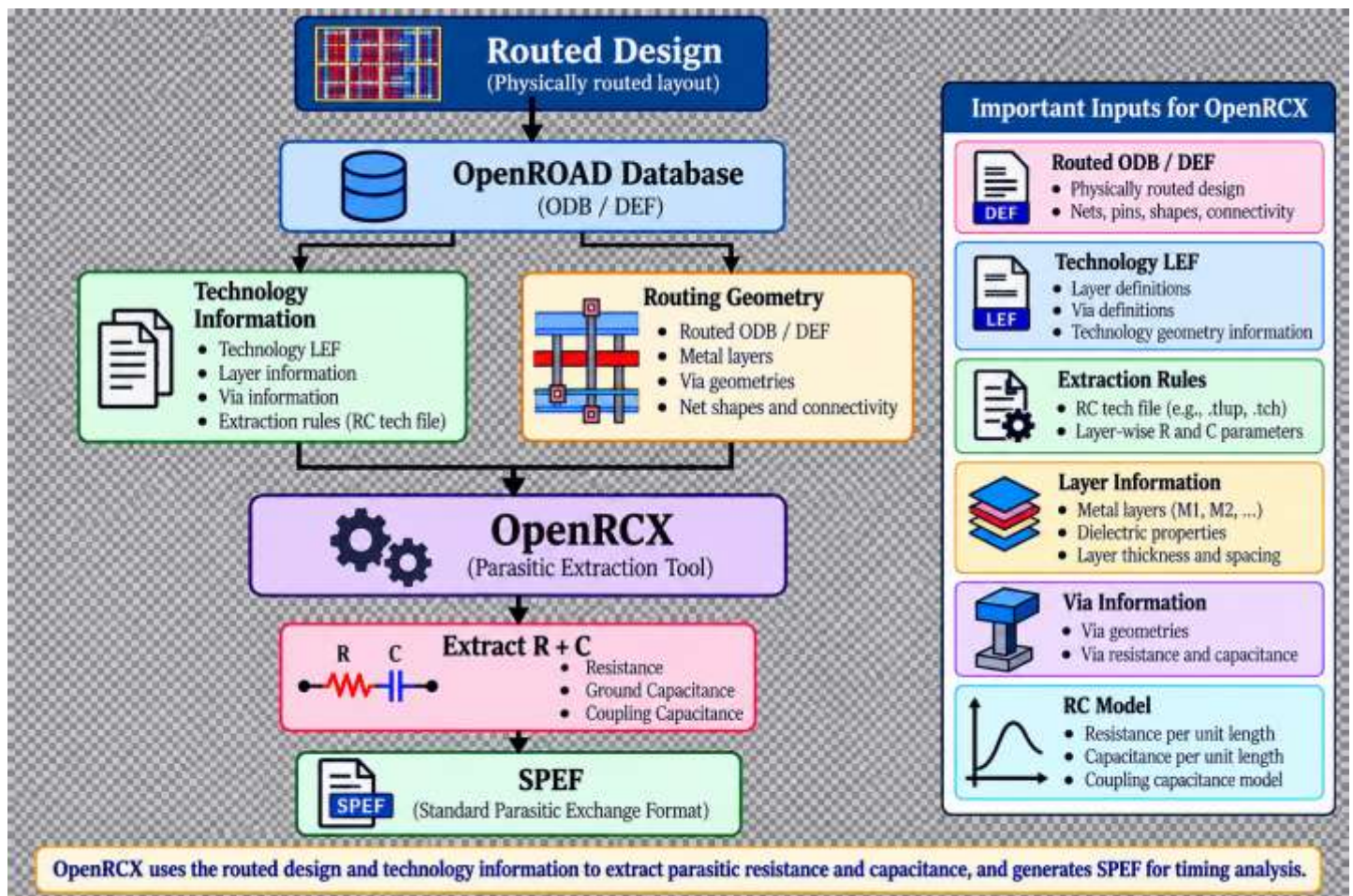
$$C = 20 \, \text{fF}$$

$$\begin{aligned}
 t &\approx RC \\
 &= 100 \times 20 \, \text{fF} \\
 &= 2 \, \text{ps}
 \end{aligned}$$

Actual interconnect delay is more complex because real nets have distributed resistance, capacitance, driver characteristics, loads, and coupling effects.

#### 14. OPENRCX INPUTS

A typical OpenRCX extraction flow requires:



#### 15. EXTRACTION RULES

OpenRCX uses technology-specific extraction rules to determine resistance and capacitance.

These rules describe characteristics such as:

- Metal layers
- Metal widths
- Metal spacing
- Dielectric properties
- Resistance values

- Capacitance behavior
- Via characteristics

A typical extraction-rules file may be technology-specific.

For example:

- rcx\_rules.txt

The exact format and contents depend on the technology/PDK.

## 16. SPEF

SPEF stands for:

### Standard Parasitic Exchange Format

It is a standard format used to represent extracted parasitic information.

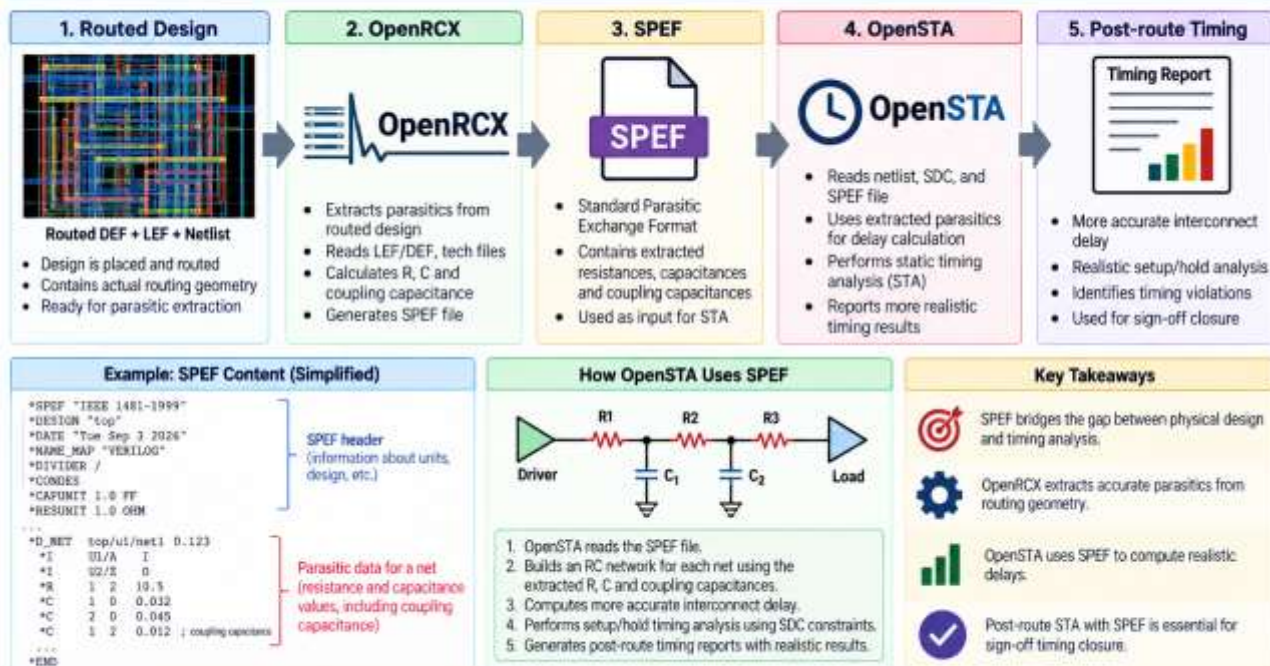
A SPEF file can contain:

- Nets
- Nodes
- Resistance
- Ground capacitance
- Coupling capacitance

The exact contents depend on the extracted design and technology.

## 17. SPEF IN POST-ROUTE STA

OpenSTA uses the extracted parasitics from the SPEF file to calculate more realistic interconnect delay, providing accurate post-route timing analysis.



SPEF enables OpenSTA to consider real interconnect parasitics, resulting in accurate and reliable post-route timing analysis.

## **18. PROCEDURE**

### **Step 1 – Check OpenROAD Version**

Run:

- openroad -version

Start OpenROAD:

- openroad

Check:

- version

### **19. STEP 2 – Check OpenRCX Commands**

Inside OpenROAD:

- help set\_extraction\_rules\_file

Check the extraction command:

- help extract\_parasitics

Check SPEF output:

- help write\_spef

The exact options should always be checked against the installed OpenROAD version.

### **20. STEP 3 – Create Experiment Directory**

- mkdir -p ~/openroad\_exp13
- cd ~/openroad\_exp13

### **21. STEP 4 – Obtain the Routed Design**

Use the routed database generated in Experiment-11:

- routed.odb

The design should already contain:

- Floorplan
- Power grid
- Placement
- CTS
- Signal routing

## **22. STEP 5 – Load Routed Database**

Start OpenROAD:

- openroad

Load the database:

- read\_db routed.odb

Check:

- report\_design\_area

and:

- check\_placement

## **23. STEP 6 – Inspect Routed Design**

Launch the GUI:

- gui::show

Inspect:

- Metal layers
- Vias
- Signal nets
- Clock network
- Power network
- Routing congestion

The design should contain actual physical routing before parasitic extraction.

## **24. STEP 7 – Load Extraction Rules**

OpenRCX requires technology-specific extraction rules.

Representative command:

- set\_extraction\_rules\_file <rcx\_rules\_file>

For example:

- set\_extraction\_rules\_file rcx\_rules.txt

The actual file name and location depend on the technology being used.

## **25. STEP 8 – Define Process Corner**

OpenRCX can use process-corner information for extraction.

A representative command is:

- define\_process\_corner -ext\_model\_index 0 X

The exact syntax and process-corner setup depend on the extraction-rule file and OpenRCX version.

Use:

- help define\_process\_corner

to check the supported syntax.

## **26. STEP 9 – Perform Parasitic Extraction**

Run:

- `extract_parasitics -ext_model_file <rcx_rules_file>`

Depending on the extraction setup, the extraction command may instead use the extraction rules configured previously.

Always verify:

- `help extract_parasitics`

## **27. STEP 10 – Write SPEF**

After extraction:

- `write_spef design.spef`

The resulting file is:

- `design.spef`

Check that it exists:

- `ls -lh design.spef`

## **28. STEP 11 – Inspect SPEF**

Use:

- `gvim design.spef`

or:

- `head -50 design.spef`

Look for sections describing:

- Design
- Ports
- Nets
- Resistance
- Capacitance
- Coupling capacitance

## **29. STEP 12 – Read SPEF for STA**

The extracted SPEF can be read into the timing-analysis environment.

Representative command:

- `read_spef design.spef`

Then run:

- `report_checks -path_delay max`

and:

- `report_checks -path_delay min`

### **30. STEP 13 – Analyze Post-Route Timing**

Record:

- Setup WNS
- Setup TNS
- Hold WNS
- Hold TNS
- Critical Path

### **31. STEP 14 – Analyze Critical Nets**

Identify timing-critical nets and examine:

- Wire length
- Metal layers
- Number of vias
- Resistance
- Capacitance
- Coupling capacitance

### **32. COMPLETE REPRESENTATIVE SCRIPT**

Create:

- `gvim exp13_rcx.tcl`

Representative flow:

`read_db routed.odb`

`report_design_area`

`check_placement`

`set_extraction_rules_file <rcx_rules_file>`

`define_process_corner -ext_model_index 0 X`

`extract_parasitics -ext_model_file <rcx_rules_file>`

write\_spef design.spef

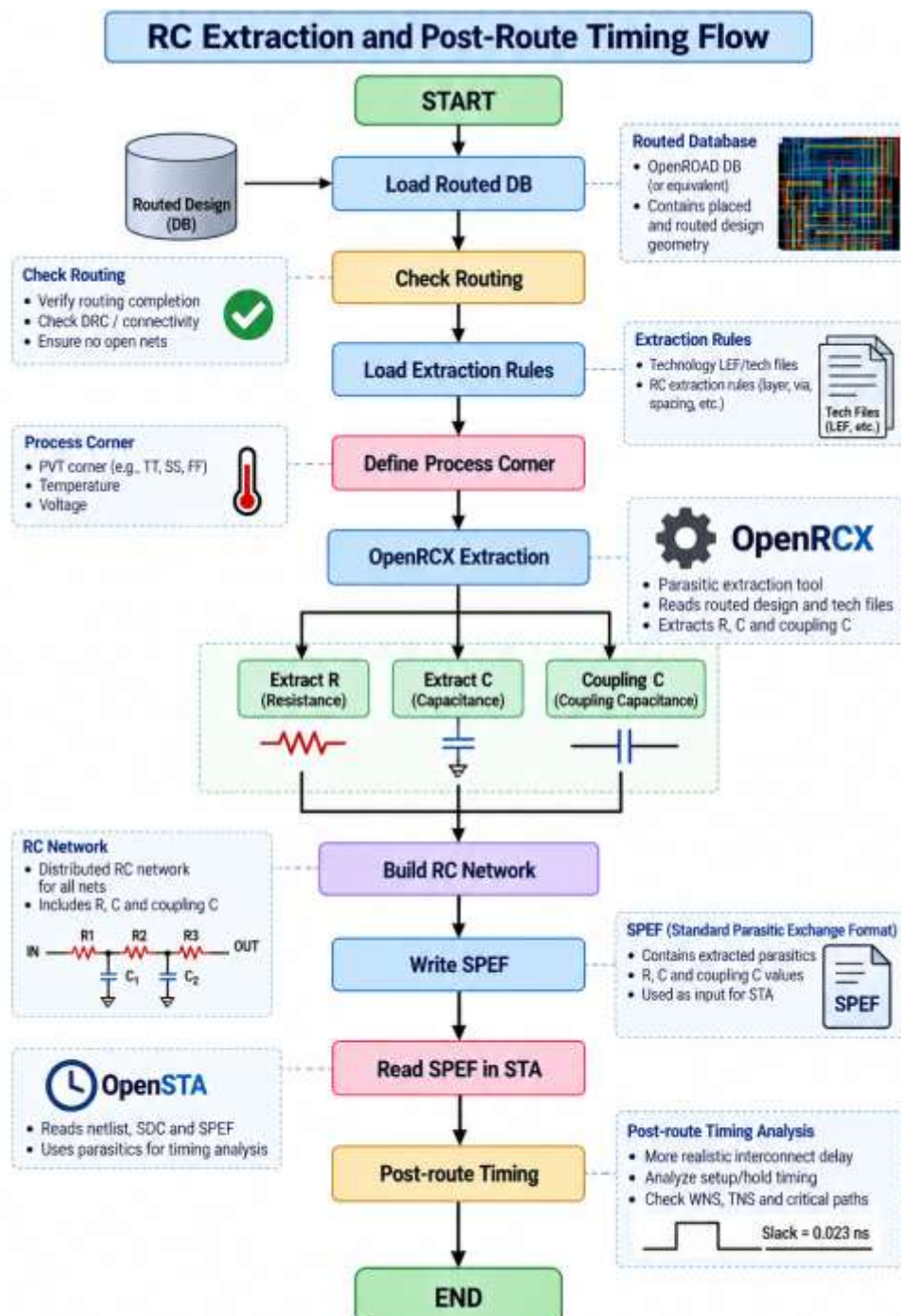
write\_db rcx.odb

Run:

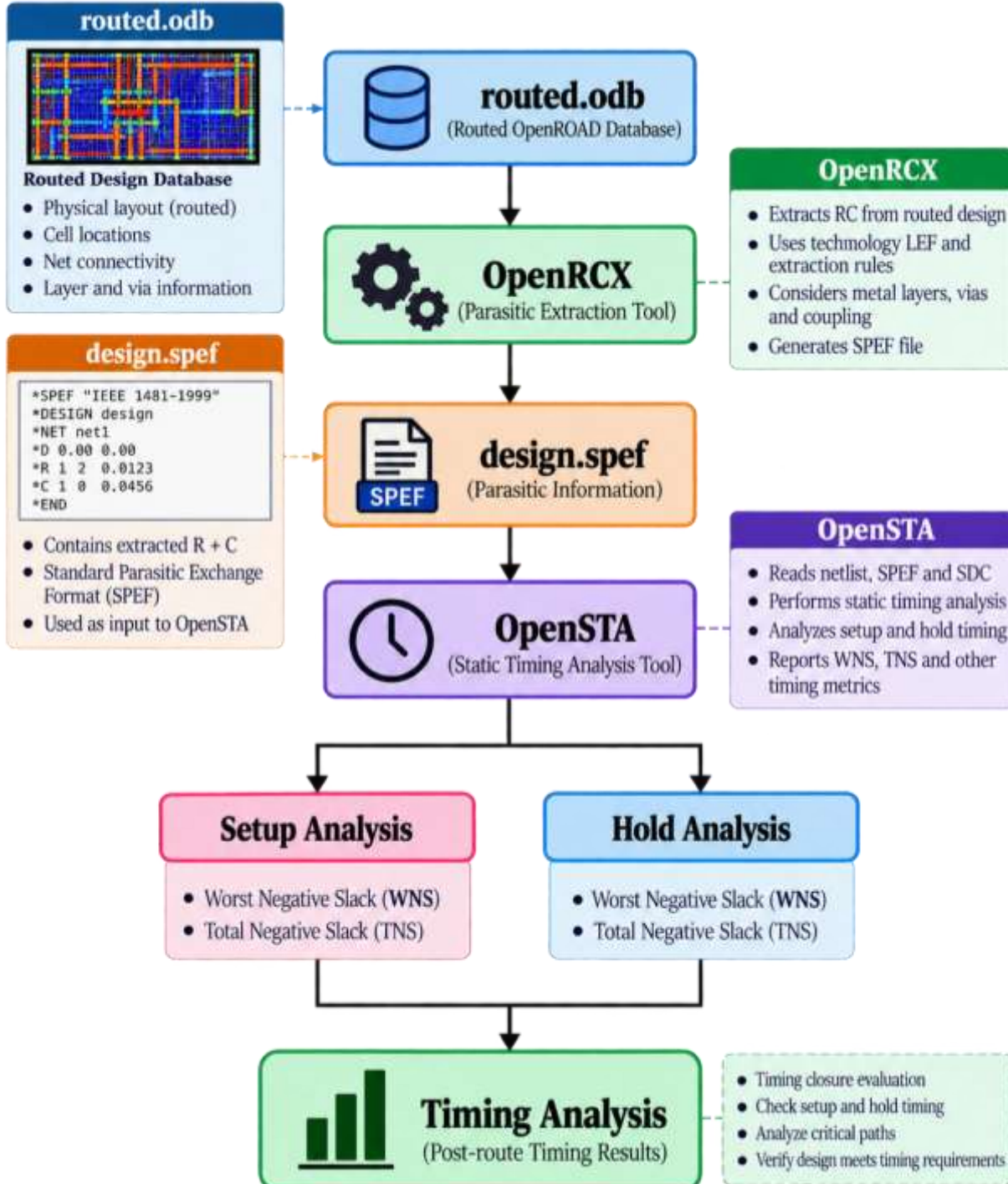
- openroad exp13\_rcx.tcl

**Important:** The exact extraction-rule and process-corner syntax is technology-dependent. Verify the commands in your installed OpenROAD version using help.

### 33. ALTERNATIVE POST-ROUTE STA FLOW

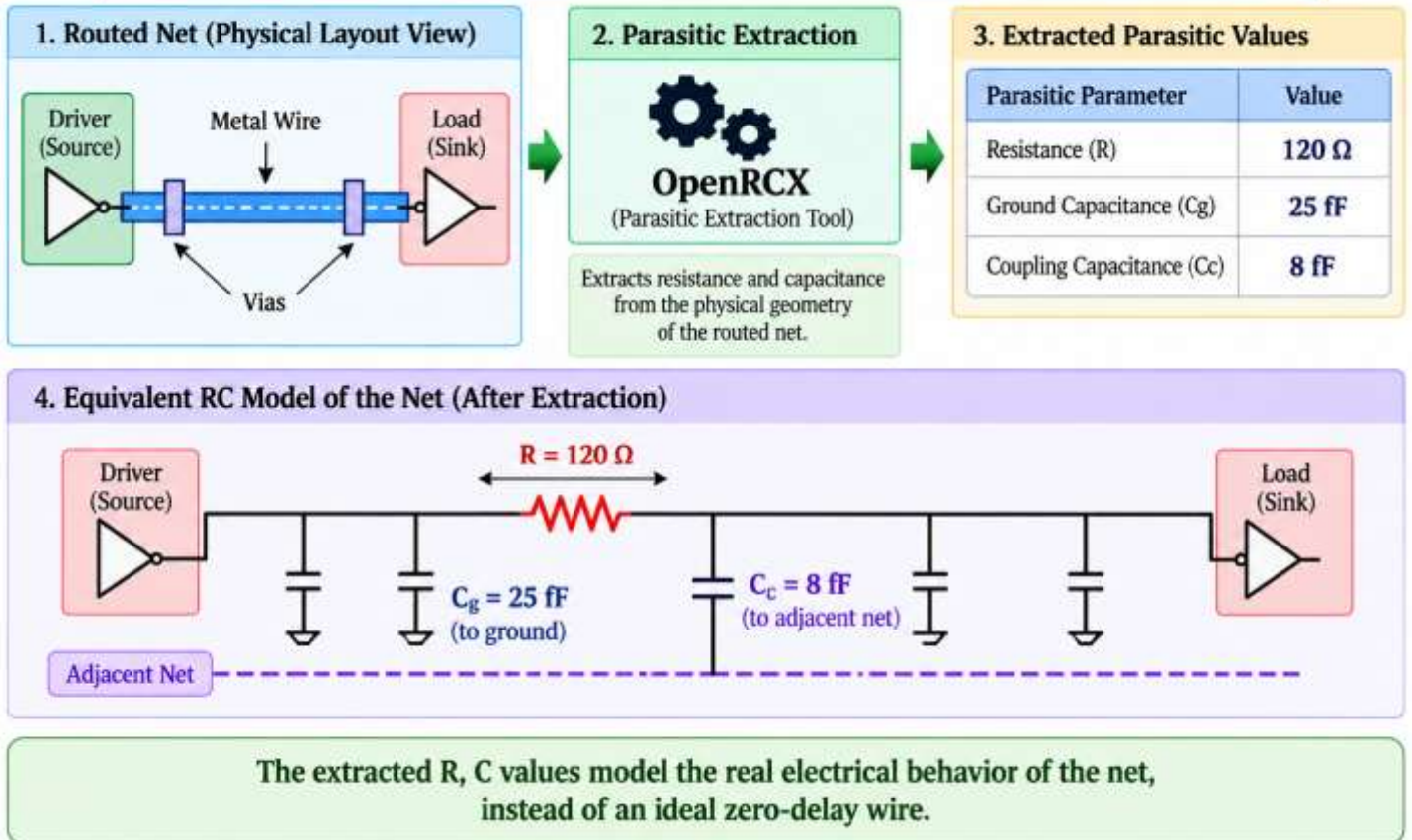


### 34. PARASITIC EXTRACTION FLOW



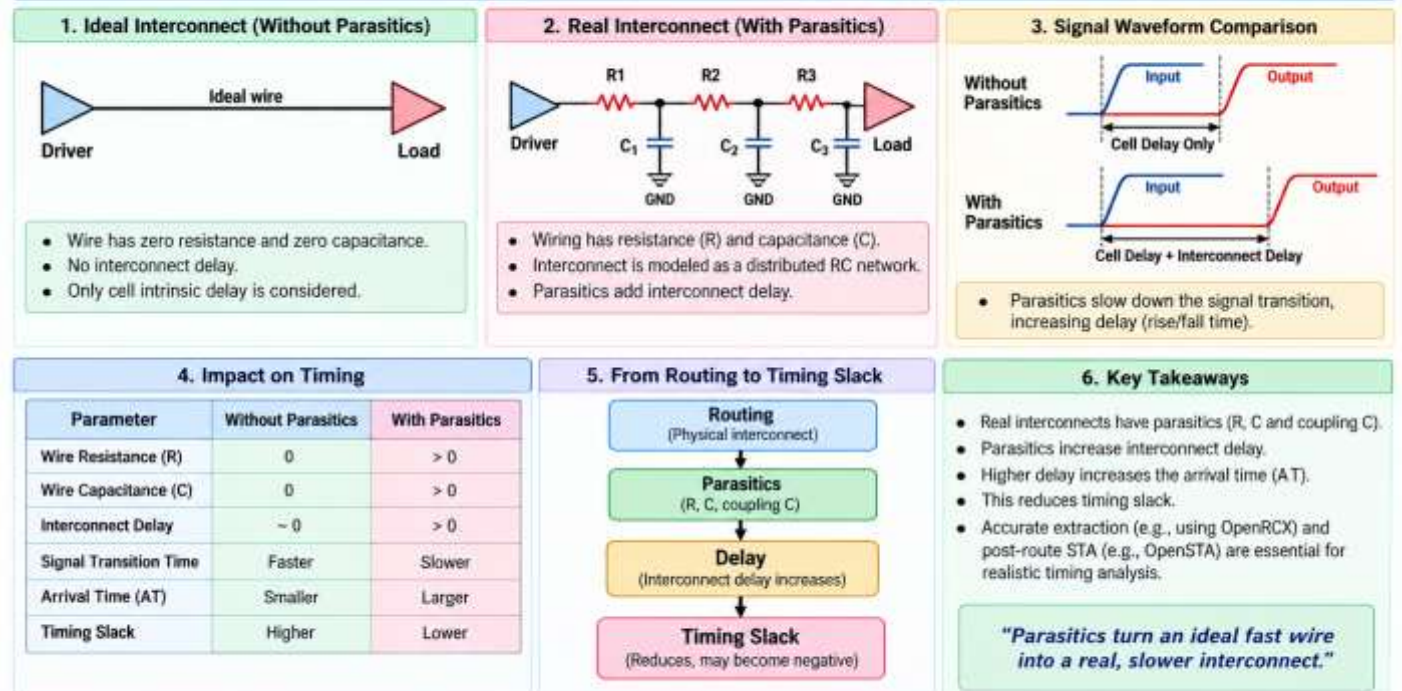
### 35. RC EXTRACTION EXAMPLE

#### Example: Parasitic Extraction of a Routed Net Using OpenRCX



### 36. EFFECT OF PARASITICS ON TIMING

Parasitics introduced by interconnects (R, C and coupling C) increase the signal delay, which reduces the timing slack.



### 37. PRE-ROUTE VS POST-ROUTE TIMING

| Parameter          | Pre-route             | Post-route |
|--------------------|-----------------------|------------|
| Wire information   | Estimated             | Extracted  |
| Resistance         | Estimated/model-based | Extracted  |
| Capacitance        | Estimated/model-based | Extracted  |
| Coupling           | Limited/estimated     | Extracted  |
| Timing accuracy    | Lower                 | Higher     |
| Physical realism   | Lower                 | Higher     |
| Sign-off relevance | Lower                 | Higher     |

### 38. OBSERVATION TABLE

| S.No. | Parameter            | Observation |
|-------|----------------------|-------------|
| 1     | Design Name          |             |
| 2     | OpenROAD Version     |             |
| 3     | Routed Database      |             |
| 4     | Extraction Rules     |             |
| 5     | Process Corner       |             |
| 6     | Number of Nets       |             |
| 7     | Total Wirelength     |             |
| 8     | Extracted Resistance |             |
| 9     | Ground Capacitance   |             |
| 10    | Coupling Capacitance |             |
| 11    | SPEF File            |             |
| 12    | Pre-route WNS        |             |
| 13    | Post-route WNS       |             |
| 14    | Pre-route TNS        |             |

| S.No. | Parameter      | Observation |
|-------|----------------|-------------|
| 15    | Post-route TNS |             |
| 16    | Timing Status  | PASS / FAIL |

### 39. ANALYSIS

### 40. EXPECTED OUTPUT

### 41. IMPORTANT OPENROAD COMMANDS

| Command                   | Purpose                            |
|---------------------------|------------------------------------|
| read_db                   | Read routed OpenROAD database      |
| set_extraction_rules_file | Set RC extraction rules            |
| define_process_corner     | Define extraction process corner   |
| extract_parasitics        | Extract RC parasitics              |
| write_spef                | Write extracted parasitics to SPEF |
| read_spef                 | Read SPEF for timing analysis      |
| report_checks             | Analyze timing                     |
| write_db                  | Save database                      |
| gui::show                 | Open GUI                           |
| help extract_parasitics   | Show extraction options            |
| help write_spef           | Show SPEF output options           |

#### **48. RESULT**

#### **49. CONCLUSION**

### EXPERIMENT-14

#### Check Antenna Rules Violations Using Antenna Rule Checker and Generate a Report

**Contact Hours:** 2 Hours

#### 1. AIM

To check **antenna rule violations** in a routed ASIC design using the **Antenna Rule Checker in OpenROAD**, analyze the violating nets and metal layers, and generate an antenna violation report.

#### 2. OBJECTIVES

After completing this experiment, the student will be able to:

- Understand the **antenna effect** in semiconductor fabrication.
- Understand why long metal interconnects can damage MOS gate oxide.
- Understand antenna-rule ratios used by foundries.
- Run the OpenROAD **Antenna Rule Checker**.
- Check antenna violations in a routed design.
- Identify violating nets and layers.
- Generate and analyze an antenna report.
- Understand common antenna-fixing techniques.
- Distinguish antenna violations from conventional DRC violations.

#### 3. SOFTWARE / TOOLS REQUIRED

| S. No. | Software / Tool      | Purpose                              |
|--------|----------------------|--------------------------------------|
| 1      | Linux / Ubuntu       | Operating environment                |
| 2      | OpenROAD             | Physical design and antenna analysis |
| 3      | Antenna Rule Checker | Detection of antenna violations      |
| 4      | OpenDB               | Design database                      |
| 5      | Yosys                | RTL synthesis                        |
| 6      | OpenSTA              | Timing analysis                      |
| 7      | LEF                  | Physical technology/cell information |
| 8      | Liberty .lib         | Cell timing information              |
| 9      | DEF / ODB            | Placed and routed design             |
| 10     | PDK antenna rules    | Technology-specific antenna limits   |

## 4. THEORY

### 4.1 What is the Antenna Effect?

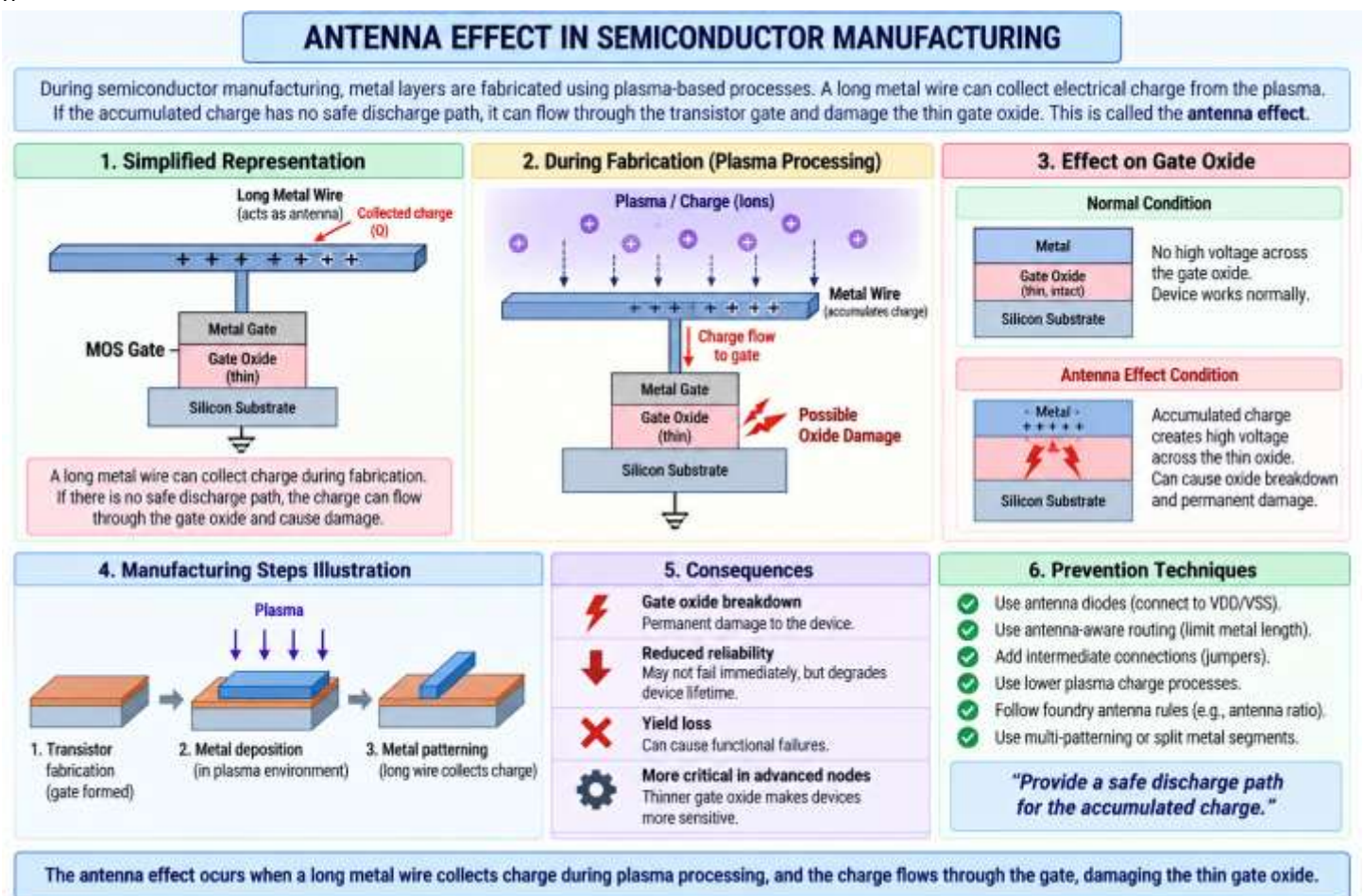
During semiconductor manufacturing, metal layers are fabricated through several plasma-based processing steps.

A long metal wire connected to a transistor gate can collect electrical charge during fabrication.

If the accumulated charge has no safe discharge path, it can flow through the transistor gate and damage the **thin gate oxide**.

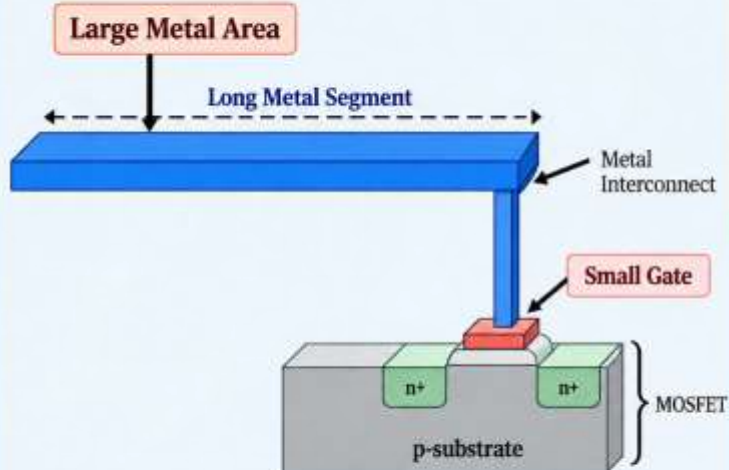
This phenomenon is called the **antenna effect**.

n



## 4.2 Why Does Antenna Violation Occur?

### Example: Long Metal Connected to a Small Gate



### Why It Happens?

- During fabrication, the metal interconnect is exposed to plasma (e.g., in etch processes).
- A long metal segment can collect charge from the plasma.
- This charge can discharge through the gate oxide, damaging the thin oxide layer of the small transistor gate.
- If the ratio of exposed metal area to gate area is too large, it is reported as an **antenna violation**.

### Antenna Ratio Rule

$$\text{Antenna Ratio} = \frac{\text{Exposed Metal Area (A}_{\text{metal}})}{\text{Gate Area (A}_{\text{gate}})}$$

If Antenna Ratio > Technology-Specific Limit  
**→ Antenna Violation**

A long metal segment connected directly to a small transistor gate can cause excessive charge accumulation, and if the antenna ratio exceeds the allowed limit, an antenna violation occurs.

### 4.3 Antenna Ratio

A simplified antenna ratio can be represented as:

$$\text{Antenna Ratio} = \frac{\text{Metal Area}}{\text{Gate Area}}$$

For example, suppose:

Metal area = 1000  $\mu\text{m}^2$

Gate area = 10  $\mu\text{m}^2$

Then:

$$\text{Antenna Ratio} = \frac{1000}{10} = 100$$

If the technology permits a maximum ratio of 80, the net violates the antenna rule.

**Important:** Actual antenna rules are technology/PDK dependent. Foundries may define different area, sidewall, perimeter, partial-area, and layer-specific rules.

### 4.4 Antenna Rules

A foundry provides antenna rules that specify the maximum permitted amount of conductor connected to sensitive transistor terminals during manufacturing.

Rules can depend on:

- Metal layer
- Metal area
- Metal perimeter
- Gate area

- Sidewall area
- Via contribution
- Layer combination
- Process step
- Diode protection

#### 4.5 Antenna Violation

An antenna violation occurs when the calculated antenna value exceeds the allowed value specified by the technology rules.

Example:

Allowed Ratio = 80

Actual Ratio = 125

$125 > 80$



ANTENNA VIOLATION

#### 4.6 Antenna Rule Checker in OpenROAD

OpenROAD contains an **Antenna Rule Checker** that analyzes the routed design against the antenna rules available for the technology.

The current OpenROAD antenna module is associated with the ant module, and the main command is:

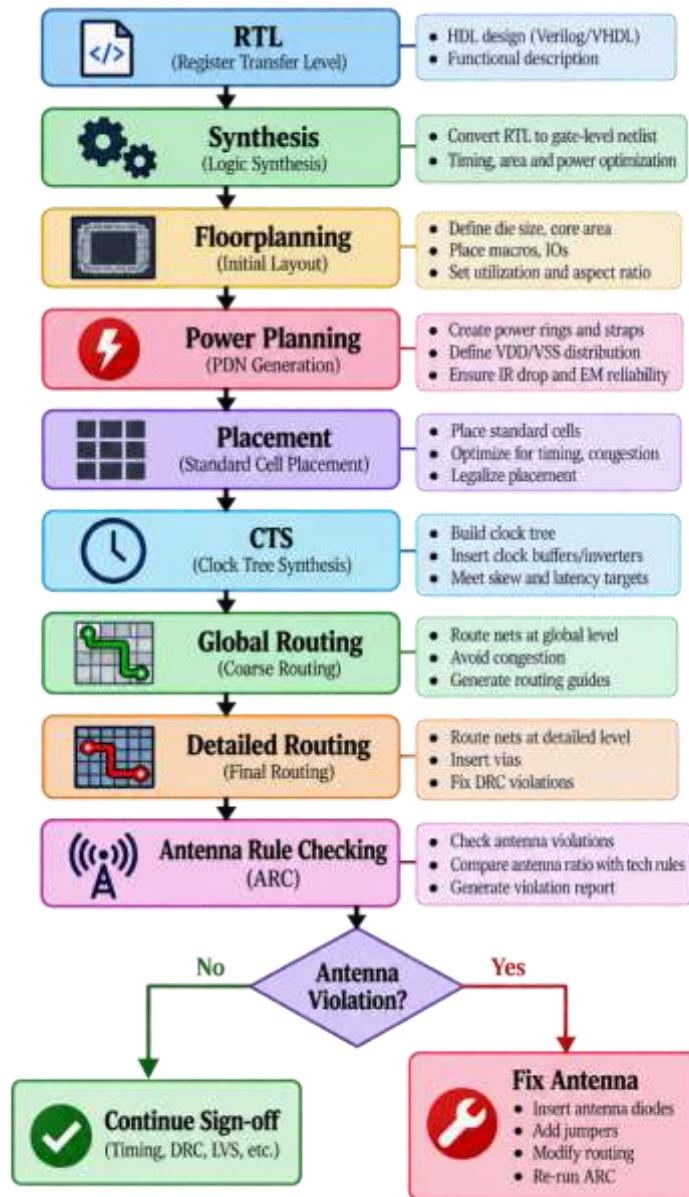
- check\_antennas

Useful commands can be inspected using:

- help check\_antennas

The checker can be used after routing when the design contains actual routed metal and vias.

#### 4.7 Position of Antenna Checking in ASIC Flow



#### 4.8 Why Antenna Checking is Performed After Routing

Antenna rules depend strongly on the physical interconnect.

Before routing:

Cell — Cell

There is no complete physical metal structure to analyze.

After routing:

Cell — M1 — Via — M2 — Via — M3 — Cell

The checker can analyze:

- Metal area
- Metal perimeter
- Vias
- Connected gate area
- Layer information
- Net connectivity

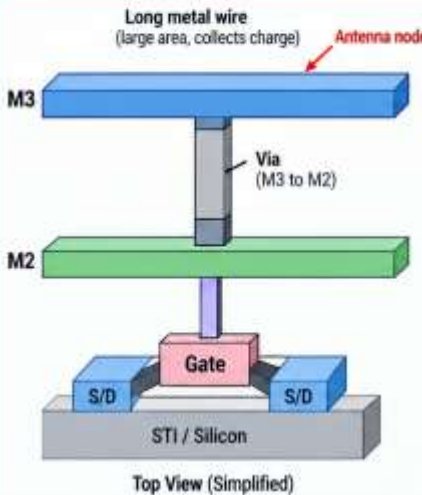
Therefore, antenna checking is normally performed on the routed design.

#### 4.9 Antenna Violation Example

### 4.9 ANTENNA VIOLATION EXAMPLE

A long metal interconnect connected to a transistor gate can collect charge during plasma processing. If the charge has no safe discharge path, it can flow through the gate oxide and cause damage. This is checked using antenna rules, and if the metal area is too large, an antenna violation occurs.

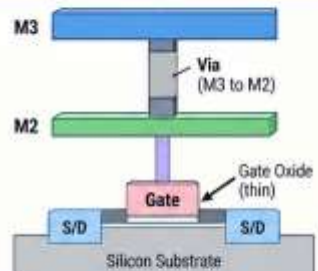
#### 1. Layout Example



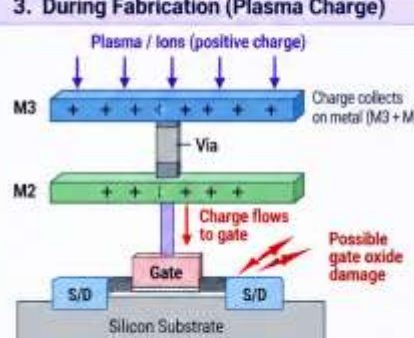
Top View (Simplified)

Metal layers M3 and M2 are connected through a via and drive the gate of a transistor. If the metal area is large enough, it can collect charge during fabrication, leading to an antenna violation.

#### 2. Cross-section View



#### 3. During Fabrication (Plasma Charge)



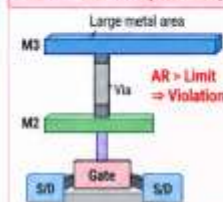
#### 4. Antenna Rule Check

$$\text{Antenna Ratio (AR)} = \frac{\text{Metal Area (connected to gate)}}{\text{Gate Area (of the transistor)}}$$

- If  $AR > \text{technology limit} \rightarrow$  Antenna violation
- The metal area includes all layers (M1, M2, M3, ...) connected to the gate.
- Each foundry provides antenna rule limits for different layers and processes.

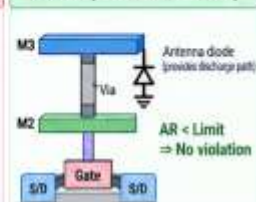
#### 5. Example: Violation vs Fix

Antenna Violation (Before Fix)



AR > Limit  
 $\Rightarrow$  Violation

After Fix (Antenna Diode Added)



AR < Limit  
 $\Rightarrow$  No violation

#### 6. Key Takeaways

- Long metal wires connected to gates can collect plasma charge.
- If no safe discharge path exists, the charge can damage the thin gate oxide.
- Antenna rules check the metal-to-gate area ratio.
- Fixes include antenna diodes, metal segmentation, or adding jumpers.
- Running an antenna rule checker (ARC) after routing helps identify violations early.

Antenna violation occurs when the metal area connected to a gate is too large, causing excessive charge during fabrication, which can damage the gate oxide.

Suppose:

Metal Area =  $900 \mu\text{m}^2$

Gate Area =  $5 \mu\text{m}^2$

Then:

$$\text{Antenna Ratio} = \frac{900}{5} = 180$$

If:

Maximum Allowed Ratio = 100

then:

Actual Ratio > Allowed Ratio



Antenna Violation

#### 4.10 Common Antenna Fixing Techniques

Although this experiment focuses on **checking and reporting**, understanding the repair methods is important.

##### 1. Antenna Diode Insertion

A diode can provide a discharge path for accumulated charge.

##### 2. Metal Jumper

The connection can be moved to a higher metal layer through a via/jumper structure.

This can reduce the antenna contribution associated with lower layers during particular manufacturing steps.

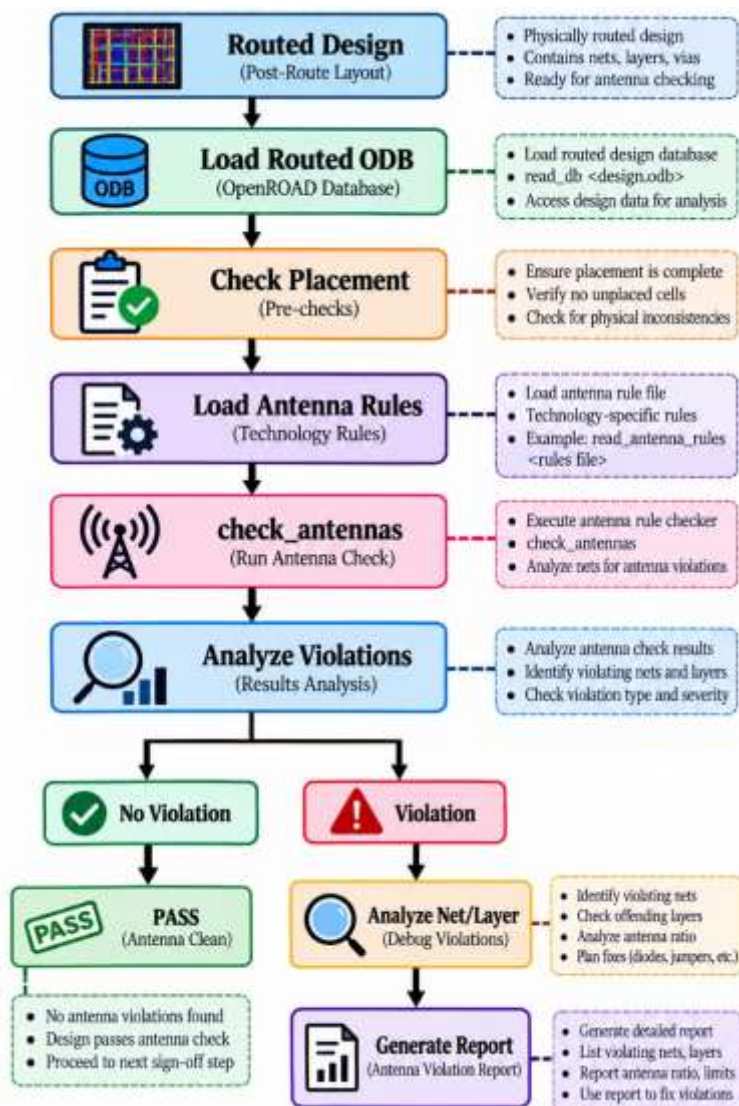
##### 3. Routing Modification

The router may be able to modify the route to reduce the violating conductor contribution

##### 4. Reduce Metal Exposure

If possible, reducing the amount of exposed metal connected to the gate can reduce the antenna ratio.

#### 5. EXPERIMENT FLOW



## 6. PROCEDURE

Step 1: Check OpenROAD Installation

Open a Linux terminal and execute:

- `openroad -version`

Expected output will display the installed OpenROAD version.

### Step 2: Create Experiment Directory

- `mkdir -p ~/openroad_exp14`
- `cd ~/openroad_exp14`

### Step 3: Start OpenROAD

- `openroad`

You should see the OpenROAD Tcl prompt:

- `OpenROAD>`

### Step 4: Check Antenna Command

At the OpenROAD prompt:

- `help check_antennas`

This displays the available antenna-checking options for your installed OpenROAD version.

### Step 5: Load the Routed Design

Use the routed database generated in the previous experiment:

- `read_db routed.odb`

Alternatively, if your routed design has another name:

- `read_db <routed_design>.odb`

### Step 6: Check Design Placement

Run:

- `check_placement`

This verifies that the physical placement is valid before proceeding with analysis.

### Step 7: Check Antenna Violations

Run:

- `check_antennas`

The checker analyzes the routed design for antenna-rule violations.

### Step 8: Run a Detailed Antenna Check

For additional information, use:

- `check_antennas -verbose`

The exact available options can vary with the OpenROAD version, so always verify using:

- `help check_antennas`

### Step 9: Check a Specific Net

If the installed version supports the `-net` option, a particular net can be checked using:

- `check_antennas -net <NET_NAME>`

**For example:**

- `check_antennas -net net_123`

Use the exact net name present in your design.

## 7. CREATE AN ANTENNA CHECK SCRIPT

Create a Tcl script:

- `gvim exp14_antenna.tcl`

**Enter:**

- `read_db routed.odb`
- `check_placement`
- `check_antennas -verbose`

**Save and exit GVim.**

## 8. RUN THE SCRIPT

Execute:

- `openroad exp14_antenna.tcl`

To save the output to a report:

- `openroad exp14_antenna.tcl > antenna_report.txt`

## 9. VIEW THE REPORT

Use:

- `gvim antenna_report.txt`

or:

- `cat antenna_report.txt`

You can search the report for relevant terms in GVim:

- /antenna

or:

- /violation

## 10. ANALYSIS OF ANTENNA REPORT

### 11. OBSERVATION TABLE

| Parameter                    | Observation        |
|------------------------------|--------------------|
| Design Name                  |                    |
| OpenROAD Version             |                    |
| Routed Database              | routed.odb         |
| Number of Nets Checked       |                    |
| Number of Antenna Violations |                    |
| Violating Net(s)             |                    |
| Violating Layer(s)           |                    |
| Maximum Antenna Value/Ratio  |                    |
| Antenna Rule Limit           |                    |
| Diode Protection             |                    |
| Report File                  | antenna_report.txt |
| Overall Status               | PASS / FAIL        |

## 12. ANTENNA VS DRC

Antenna checking is related to physical verification but is not exactly the same as conventional geometric DRC.

| DRC                                  | Antenna Check                             |
|--------------------------------------|-------------------------------------------|
| Checks geometric/manufacturing rules | Checks charge-related antenna rules       |
| Width, spacing, enclosure, etc.      | Metal/gate relationship and antenna rules |
| Primarily geometric constraints      | Connectivity + layer + technology rules   |
| Can detect shorts/spacings           | Detects potential gate-oxide damage       |

|                                       |                                      |
|---------------------------------------|--------------------------------------|
| <b>DRC</b>                            | <b>Antenna Check</b>                 |
| Used throughout physical verification | Particularly important after routing |

### 13. ANTENNA VS IR DROP

These two analyses address different reliability problems.

| <b>Antenna Analysis</b>                  | <b>IR Drop Analysis</b>                     |
|------------------------------------------|---------------------------------------------|
| Fabrication-related reliability          | Power-delivery reliability                  |
| Concerned with accumulated plasma charge | Concerned with voltage drop                 |
| Associated with gate oxide damage        | Associated with insufficient supply voltage |
| Uses antenna rules                       | Uses PDN/current information                |
| Performed using antenna checker          | Performed using PDNSim/power-grid analysis  |

### 14. EXPECTED OUTPUT

### 15. RESULT

### 18. CONCLUSION

**EXPERIMENT RTL-TO-GDSII FLOW**



**ALL THE BEST FOR EXAM AND YOUR FUTURE**

**KEEP WORKING HARD.....**

**Thank you .....**